

DPTO INFORMÁTICA – IES TRAFALGAR
MÓDULO – PROYECTO INTEGRADO
C.F.G.S. DAW

ECOMMERCE

Autor: Judit Quirós Violero

Fecha: 25/04/2025

Tutor: José Antonio Rodríguez Jiménez

HOJA DE RESUMEN

Título del proyecto: Ecommerce	
Autor: Judit Quirós Violero	Fecha: 25/04/2025
Tutor: José Antonio Rodríguez Jiménez	
Ciclo Formativo: DAW	Curso: 2024/2025
Palabras clave: Laravel, Livewire, ecommerce, MySQL, autenticación, carrito de compras, pedidos, accesibilidad web, PHP, base de datos relacional.	
Resumen del proyecto: Este proyecto consiste en el desarrollo de una tienda online utilizando el framework Laravel junto con Livewire, como parte de un trabajo académico del ciclo de Desarrollo de Aplicaciones Web. El sistema incluye funcionalidades típicas de un ecommerce, como la gestión de productos, carrito de compras, pedidos y control de roles para usuarios y administradores. Se ha implementado una base de datos relacional en MySQL y se han tenido en cuenta aspectos clave como la accesibilidad web. Aunque no se han desarrollado pruebas automatizadas, se reconoce su importancia para garantizar la calidad del sistema, y la posibilidad de implementarlas en un futuro. El objetivo principal es crear una base sólida que pueda ampliarse o adaptarse fácilmente a proyectos reales.	

ÍNDICE

HOJA DE RESUMEN	2
ÍNDICE	3
1. INTRODUCCIÓN	5
1.2 - OBJETIVOS DEL PROYECTO	5
2. ESTUDIO DE VIABILIDAD	6
2.1 - VIABILIDAD TÉCNICA	6
2.2 - VIABILIDAD ECONÓMICA	7
2.3 - VIABILIDAD OPERATIVA	8
3. ALTERNATIVAS Y SELECCIÓN DE LA SOLUCIÓN	9
3.1 - POSIBLES ALTERNATIVAS	9
4. ANÁLISIS Y DISEÑO DE LA SOLUCIÓN ADOPTADA	11
4.1 - ESTRUCTURA DE LA BASE DE DATOS	11
4.2 - FUNCIONALIDADES PRINCIPALES	11
4.3 - CONTROL DE ROLES Y DOBLE AUTENTICACIÓN	13
4.4 - FRONTEND Y HERRAMIENTAS ADICIONALES	13
5. IMPLEMENTACIÓN	14
5.1 - ESTRUCTURA DEL PROYECTO	14
5.1.1 - CARPETAS IMPORTANTES	14
5.2 - MODELOS	15
5.2.1 - CART.PHP	15
5.2.2 - CARTITEM.PHP	16
5.2.3 - IMAGE.PHP	17
5.2.4 - ORDER.PHP	18
5.2.5 - ORDERITEM.PHP	19
5.2.6 - PRODUCT.PHP	21
5.2.7 - SIZE.PHP	22
5.2.8 - USER.PHP	22
5.3 - SEEDERS	24
5.3.1 - DATABASESEEDER.PHP	24
5.3.3 - PRODUCTSEEDER.PHP	24
5.3.3 - IMAGESEEDER.PHP	26
5.4 - COMPONENTES BLADE Y VISTAS BASE	27
5.4.1 - BASE.BLADE.PHP	27
5.4.2 - HEADER.BLADE.PHP Y SIDEBAR.BLADE.PHP	31
5.5 - COMPONENTES LIVEWIRE	33
5.5.1 - ADDTOCART.PHP	33

5.5.2 - ADMIN DASHBOARD.PHP	36
5.5.3 - INDEX PAGE.PHP	39
5.5.4 - ORDER CONFIRMATION.PHP	42
5.5.5 - ORDER CRUD.PHP	45
5.5.6 - PRODUCT CRUD.PHP	51
5.5.7 - PRODUCT SHOW.PHP	65
5.5.8 - SEE MY ORDERS.PHP	67
5.5.9 - SHOW CART.PHP	69
5.5.10 - SHOW STORE PROD.PHP	74
5.5.11 - USER CRUD.PHP	79
5.6 - GESTIÓN DE USUARIOS Y ROLES	84
5.6.1 - ROLES DEFINIDOS	84
5.6.2 - MIDDLEWARE PARA EL CONTROL DE ACCESO	85
5.6.3 - REDIRECCIÓN POST-LOGIN	86
5.7 - DEFINICIÓN DE RUTAS	86
5.7.1 - RUTAS PÚBLICAS	86
5.7.2 - RUTAS PROTEGIDAS PARA USUARIOS AUTENTICADOS	87
5.7.3 - RUTAS PARA USUARIOS NORMALES (ROL = 0)	88
5.7.4 - RUTAS PARA ADMINISTRADORES (ROL = 1)	88
5.7.6 - OTRAS RUTAS	89
5.8 - SISTEMA DE CORREO ELECTRÓNICO	90
5.8.1 - NUEVO PEDIDO	90
5.8.2 - CÓDIGO DE VERIFICACIÓN	91
5.9 - AUTENTICACIÓN Y VERIFICACIÓN EN 2 PASOS	92
5.10 - PRÁCTICAS DE ACCESIBILIDAD IMPLEMENTADAS	95
6. PRUEBAS	96
6.2 - DE ACCESIBILIDAD	97
6.2.1 - HERRAMIENTAS	97
6.2.2 - DOCUMENTACIÓN	98
7. COSTES/PRESUPUESTO	99
8. CONCLUSIONES	100
9. BIBLIOGRAFÍA	101
10. GLOSARIO	102
11. ANEXOS	103
11.1 - PROTOTIPO DE LA WEB	103
11.2 - BASE DE DATOS	103
11.2.1 - DIAGRAMA RELACIONAL	104
11.2.2 - GRAFO	104

1. INTRODUCCIÓN

El presente proyecto consiste en el desarrollo de una tienda online de ropa como ejemplo clásico de ecommerce, por la variedad de funcionalidades que integra: desde la gestión de usuarios y productos hasta el control de stock, pedidos y roles de acceso.

Esta elección permite poner en práctica los conocimientos aprendidos en el curso, y usar tecnologías modernas como Laravel con Livewire, Blade, Bootstrap, Eloquent... Aunque el enfoque del proyecto es académico, su estructura está pensada como base para posibles aplicaciones comerciales reales en distintos sectores del ecommerce.

1.1 - OBJETIVOS DEL PROYECTO

El objetivo de este proyecto es aplicar los conocimientos adquiridos a lo largo del ciclo formativo para construir una tienda online completamente funcional. Se busca desarrollar una plataforma dinámica, escalable y mantenible, que aproveche las capacidades de Laravel y Livewire para ofrecer un rendimiento óptimo y una experiencia de usuario fluida.

Para alcanzar este fin, se plantea la implementación de un sistema de autenticación de usuarios, la creación de un CRUD completo para productos con soporte para subida de imágenes, y el desarrollo de un carrito de compra funcional. Asimismo, se contempla la gestión de pedidos, la incorporación de un sistema de control de roles que distinga entre administradores y usuarios normales.

El proyecto también persigue garantizar la accesibilidad de la aplicación conforme a los criterios WCAG, y documentar todas las decisiones técnicas y funcionales tomadas a lo largo del desarrollo.

2. ESTUDIO DE VIABILIDAD

2.1 - VIABILIDAD TÉCNICA

Para evaluar la viabilidad técnica del proyecto, se han analizado los recursos tecnológicos disponibles, así como la compatibilidad de los mismos con los requisitos del sistema.

El desarrollo del ecommerce se ha llevado a cabo utilizando el framework Laravel, una herramienta robusta y ampliamente utilizada en el desarrollo de aplicaciones web en PHP. Además, se ha empleado Livewire para gestionar componentes interactivos en el lado del cliente sin necesidad de escribir JavaScript, lo cual permite mantener la lógica del frontend en el propio backend.

Se ha optado por MySQL como sistema gestor de base de datos relacional, debido a su integración nativa con Laravel y su rendimiento eficiente en entornos de tamaño medio. El servidor local se ha configurado mediante XAMPP, que proporciona Apache, PHP y MySQL de manera integrada, permitiendo así un entorno de desarrollo estable.

El proyecto ha sido desarrollado y probado en un entorno controlado, lo que ha permitido comprobar la compatibilidad entre las tecnologías utilizadas. Asimismo, los requerimientos del sistema no superan las capacidades del hardware y software disponible para el desarrollo del proyecto.

2.2 - VIABILIDAD ECONÓMICA

Todo el desarrollo se ha llevado a cabo en un entorno local utilizando herramientas de código abierto, lo que ha permitido prescindir de cualquier inversión en licencias o plataformas de pago. Esto incluye el uso de tecnologías como **Laravel**, **Livewire**, **MySQL**, y servidores locales como **XAMPP**.

En caso de que en un futuro el proyecto fuera desplegado en un entorno de producción, la inversión sería mínima ya que solo habría que considerar los costes asociados a la contratación de un servicio de **hosting** y la compra de un **dominio web**, cuyos precios en el mercado son muy accesibles. Actualmente, existen opciones confiables que ofrecen ambos servicios por una tarifa anual media inferior a los **50 €**, lo que representa una inversión perfectamente asumible para asegurar la disponibilidad pública del sitio.

Aunque el proyecto tiene un carácter académico y no ha implicado costes reales, en un entorno profesional sería necesario realizar una estimación económica basada en el tiempo invertido y el coste de la mano de obra.

Si se estima que para llevar a cabo el desarrollo completo del ecommerce, se han dedicado aproximadamente 100 horas de trabajo. Considerando una tarifa media de 15 €/hora para un desarrollador web junior o freelance, el coste total del desarrollo ascendería a unos 1.500 €.

2.3 - VIABILIDAD OPERATIVA

El sistema ha sido diseñado para garantizar una experiencia de uso fluida, funcional y segura, tanto para usuarios administradores como para clientes finales. Se ha priorizado la usabilidad mediante una interfaz clara y responsiva, construida con **Bootstrap**, lo que permite una correcta visualización y navegación desde distintos dispositivos y resoluciones.

El desarrollo se ha llevado a cabo en un entorno de trabajo optimizado, utilizando herramientas modernas como **Visual Studio Code** sobre **Windows 11**, y realizando pruebas de accesibilidad en la última versión estable de Google Chrome. Esto asegura una compatibilidad sólida con los estándares actuales del desarrollo web.

El sistema incorpora funcionalidades clave como el redireccionamiento automático según el rol del usuario, mejorando la organización del flujo de navegación y reforzando la seguridad al restringir el acceso a determinadas áreas.

Asimismo, la estructura del proyecto facilita su mantenimiento y escalabilidad, permitiendo añadir nuevas funcionalidades en el futuro sin comprometer el rendimiento o la estabilidad general del sistema.

3. ALTERNATIVAS Y SELECCIÓN DE LA SOLUCIÓN

Para el desarrollo de este proyecto se optó por utilizar **Laravel** como framework principal del backend, complementado con **Livewire** para lograr interactividad sin necesidad de recurrir a frameworks JavaScript complejos. En el frontend, se utilizó **Bootstrap** para la maquetación responsiva y rápida de las vistas.

Como sistema de gestión de bases de datos se empleó **MySQL**, ampliamente utilizado y compatible con Laravel, y se trabajó en un entorno local mediante **XAMPP**, que facilitó el montaje rápido del servidor web y la base de datos sin costes adicionales ni configuraciones complejas.

Durante el desarrollo, se utilizó el comando **php artisan serve** para iniciar un servidor local y visualizar la aplicación en el navegador.

Esta combinación se ajustó perfectamente tanto a los contenidos del curso como a los objetivos del proyecto, ofreciendo un entorno robusto, bien documentado y fácil de mantener.

No obstante, se consideraron otras alternativas tecnológicas antes de tomar esta decisión:

- **WordPress + WooCommerce:** son rápidos de implementar y ofrecen soluciones listas para usar, pero presentan limitaciones importantes a la hora de personalizar funcionalidades específicas del negocio, especialmente si se requiere un control más detallado del backend.
- **Proyecto con Maven, Spring Boot y Tomcat:** se consideró la posibilidad de desarrollar el sistema utilizando Java, con gestión de dependencias mediante Maven, estructura modular con Spring Boot y despliegue en Tomcat. Sin embargo, esta alternativa implicaba una configuración inicial más compleja y una curva de aprendizaje más elevada. Además, requería implementar manualmente elementos como la autenticación y la gestión de sesiones, lo

que hacía más difícil y largo el desarrollo para un proyecto de carácter académico y limitado en tiempo.

- **Symfony:** se evaluó como una alternativa dentro del ecosistema PHP, pero es más complejo de configurar y utilizar en comparación con Laravel. Además es una tecnología que vimos durante el curso, donde se detectaron gran cantidad de errores y una curva de aprendizaje más alta. Por ello, se descartó en favor de una solución más accesible.

Herd: aunque Herd facilita el desarrollo local al integrar PHP y Nginx, permitiendo el despliegue automático de la aplicación usando dominios personalizados del tipo nombredelproyecto.test. Además esto elimina la necesidad de configurar manualmente servidores como Apache o gestionar entornos complejos. Sin embargo, en este proyecto se prefirió XAMPP por su mayor familiaridad y porque XAMPP ya lo tenía instalado en el ordenador y Herd tenía que instalarlo.

4. ANÁLISIS Y DISEÑO DE LA SOLUCIÓN ADOPTADA

4.1 – ESTRUCTURA DE LA BASE DE DATOS

Se diseñó una base de datos relacional en MySQL compuesta por las siguientes tablas:

- **users:** contiene los datos básicos del usuario y su rol.
- **products:** almacena la información del producto (nombre, descripción, color, estilo, categoría, género, precio y descuento).
- **images:** almacena múltiples imágenes por producto.
- **sizes y product_size:** permiten manejar múltiples tallas por producto con su respectivo stock.
- **carts y cart_items:** gestionan el carrito de cada usuario con sus productos seleccionados y cantidades.
- **orders y order_items:** gestionan los pedidos realizados, permitiendo ver productos, cantidades, tallas y precios, además del estado del pedido.

Además, Laravel genera otras tablas por defecto para el manejo de sesiones, contraseñas o tokens, que no están directamente relacionadas con las funcionalidades principales del proyecto.

4.2 – FUNCIONALIDADES PRINCIPALES

- Registro, login y autenticación segura gestionados con Laravel.
- Control de roles de usuario (administrador = 1 y cliente = 0) con redirección automática a paneles específicos tras el login.
- Vista del catálogo de productos con filtros por diferentes atributos (talla, categoría, etc.) para facilitar la búsqueda.

- Carrito de compras dinámico usando Livewire:
 - Visualización en tiempo real de los productos agregados al carrito.
 - Posibilidad de seleccionar talla, modificar cantidad y eliminar productos sin recargar la página.
 - Vista persistente del resumen del carrito con total actualizado y botón de pedido siempre visible para mejorar la experiencia de compra.
- Vista donde se muestran los detalles del pedido y se confirman (o no).
- Panel de administración completo para usuarios con rol administrador, que incluye:
 - Crear, editar y eliminar productos con opción para subir múltiples imágenes.
 - Visualización y búsqueda avanzada de productos por ID o nombre.
 - Gestión de pedidos por usuario con detalles completos.
 - Visualización y búsqueda de pedidos por estado, nombre del usuario que realizó el pedido e intervalo de fechas en el que se realizó el pedido.
 - Eliminación de productos incluidos en un pedido.
 - Cálculo del dinero a devolver si el pedido está pagado.
 - Visualización y búsqueda de usuarios por nombre y rol.
 - Editar o eliminar usuarios.
- Vista para los usuarios normales de su historial de pedidos.
- Configuración de perfil (Creada y gestionada por Laravel).
- Vistas que le dan al sitio web más realismo como sobre nosotros, contacto, aviso legal...)
- Un mapa web con enlaces a todas las páginas del sitio web.
- Interfaz responsive y accesible, desarrollada con Bootstrap para garantizar compatibilidad con distintos dispositivos y tamaños de pantalla.
- Validación de formularios y manejo de errores para asegurar la integridad de los datos ingresados.
- Arquitectura modular que facilita futuras ampliaciones o integración de nuevas funcionalidades.

4.3 – CONTROL DE ROLES Y DOBLE AUTENTICACIÓN

El sistema implementa un control de roles que diferencia entre usuarios normales y administradores. Tras el inicio de sesión, se aplica una segunda capa de seguridad mediante un proceso de doble autenticación, en el cual el usuario debe ingresar un código enviado a su correo electrónico. Una vez verificado, cada usuario es redirigido automáticamente a su panel correspondiente según su rol. Esta medida mejora tanto la seguridad del sistema como la experiencia de navegación, al restringir el acceso a secciones no autorizadas y garantizar que solo usuarios legítimos accedan a funciones específicas.

4.4 – FRONTEND Y HERRAMIENTAS ADICIONALES

En las vistas que he creado yo misma, he usado principalmente **Bootstrap** y **CSS** puro para diseñar y maquetar, buscando un control más directo sobre el estilo y asegurando buenas prácticas de accesibilidad, como el uso adecuado de etiquetas, contraste de colores y navegación mediante teclado.

Las vistas que vienen por defecto con **Laravel**, emplean tecnologías diferentes como **Tailwind CSS** junto con **Flux 1.4** para el manejo de estilos y diseño. Además, **Laravel** utiliza **Vite** para compilar y optimizar los archivos de frontend.

También se incluye **Laravel Volt** en algunas rutas, que es una herramienta visual moderna para facilitar el desarrollo con **Livewire**, aunque no se ha utilizado mucho en este proyecto.

5. IMPLEMENTACIÓN

La implementación de este proyecto ecommerce abarca desde la configuración de la base de datos hasta el desarrollo de la interfaz y la lógica del sistema, integrando funcionalidades clave de forma segura, eficiente y fácil de usar. Se diseñaron tablas para gestionar usuarios, productos y pedidos, y se construyeron vistas personalizadas con Bootstrap y CSS puro, en contraste con las vistas por defecto de Laravel. Además, se utilizó Livewire para crear componentes dinámicos que mejoran la experiencia del usuario.

5.1 – ESTRUCTURA DEL PROYECTO

5.1.1 – CARPETAS IMPORTANTES

- Componentes middleware:
 - /app/Http/Middleware
- Componentes livewire, junto con sus vistas:
 - /app/Livewire
 - /resources/views/livewire
- Mails:
 - /app/Mail
- Modelos:
 - /app/Models
- Migraciones:
 - /database/migrations
- Inserciones a la base de datos:
 - /database/seeders
- Vistas:
 - Componentes: /resources/views/components
 - Mensaje de los emails: /resources/views/emails
 - Usuarios: /resources/views/users
 - Administrador: /resources/views/admin

- Autenticación: /resources/livewire/auth
- Rutas:
 - Generales: /routes/web.php
 - Autenticación: /routes/auth.php
- Donde se guardan las imágenes de la web:
 - Las que se cargan en el seed: public/storage/fotos_productos
 - Las que se añaden desde la web: public/storage/products

5.2 - MODELOS

5.2.1 - CART.PHP

- Campos asignables:
 - **user_id**: Referencia al usuario dueño del carrito. Se usa para asignar el carrito al usuario correspondiente. `protected $fillable = ['user_id'];`
- Relaciones:
 - **items()**: Es una relación uno a muchos (hasMany), que se relaciona con el modelo de CartItem, cuya clave foránea es “cart_id” en la tabla cart_items. En resumen: un carrito puede contener varios items (productos añadidos).

```
public function items(): HasMany
{
    return $this->hasMany(related: CartItem::class);
}
```

- **user()**: Es una relación inversa uno a uno (belongsTo), que se relaciona con el modelo de User, cuya clave foránea es “user_id” en la tabla carts. En resumen: Cada carrito pertenece a un único usuario.

```
public function user(): BelongsTo
{
    return $this->belongsTo(related: User::class);
}
```

- Métodos personalizados:
 - **addProduct(int \$productId, int \$sizeId, int \$quantity = 1):** Añade un producto al carrito con la talla indicada. Si ya existe ese producto con esa talla, simplemente aumenta la cantidad, y si no está, se crea un nuevo carrito (CartItem).

```
public function addProduct(int $productId, int $sizeId, int $quantity = 1): void
{
    $cartItem = $this->items()
        ->where(column: 'product_id', operator: $productId)
        ->where(column: 'size_id', operator: $sizeId)
        ->first();

    if ($cartItem) {
        $cartItem->quantity += $quantity;
        $cartItem->save();
    } else {
        $this->items()->create(attributes: [
            'product_id' => $productId,
            'size_id' => $sizeId,
            'quantity' => $quantity,
        ]);
    }
}
```

5.2.2 - CARTITEM.PHP

- Campos asignables:
 - **cart_id:** Referencia al carrito al que pertenece este ítem.
 - **product_id:** ID del producto añadido al carrito.
 - **size_id:** ID de la talla seleccionada para ese producto.
 - **quantity:** Cantidad de unidades del producto con esa talla.

```
protected $fillable = ['cart_id', 'product_id', 'size_id', 'quantity'];
```

- Relaciones:
 - **cart():** Es una relación inversa uno a muchos (belongsTo), que se relaciona con el modelo de Cart, cuya clave foránea es “cart_id” en la tabla cart_items. En resumen: el ítem pertenece a un carrito específico.

```
public function cart(): BelongsTo
{
    return $this->belongsTo(related: Cart::class);
}
```


- **product()**: Es una relación inversa uno a muchos (belongsTo), que se relaciona con el modelo de Product, cuya clave foránea es “product_id” en la tabla cart_items. En resumen: el ítem del carrito está relacionado con un producto de la tienda.

```
public function product(): BelongsTo
{
    return $this->belongsTo(related: Product::class);
}
```

- **size()**: Es una relación inversa uno a muchos (belongsTo), que se relaciona con el modelo de Size, cuya clave foránea es “size_id” en la tabla cart_items. En resumen: el ítem del carrito está asociado a una talla específica del producto.

```
public function size(): BelongsTo
{
    return $this->belongsTo(related: Size::class);
}
```

5.2.3 - IMAGE.PHP

- Campos asignables:
 - **product_id**: ID del producto al que pertenece la imagen.
 - **url**: Ruta o dirección de la imagen almacenada.

```
protected $fillable = ['product_id', 'url'];
```

- Relaciones:
 - **product()**: Es una relación inversa uno a muchos (belongsTo), que se relaciona con el modelo de Product, cuya clave foránea es “product_id” en la tabla Images. En resumen: cada imagen está asociada a un único producto.

```
public function product(): BelongsTo
{
    return $this->belongsTo(related: Product::class);
}
```

5.2.4 - ORDER.PHP

- Traits usados:

- **HasFactory**: Habilita el uso de factorías para generar instancias de prueba. `use HasFactory;`

- Campos asignables:

- **user_id**: ID del usuario que realizó el pedido.
- **total**: Total monetario del pedido (considerando descuentos).
- **status**: Estado actual del pedido (por ejemplo: pendiente, enviado, cancelado).

```
protected $fillable = [  
    'user_id',  
    'total',  
    'status',  
];
```

- Relaciones:

- **user()**: Es una relación inversa uno a muchos (belongsTo), que se relaciona con el modelo de User, cuya clave foránea es “user_id” en la tabla orders. En resumen: el pedido pertenece a un usuario específico.

```
public function user(): BelongsTo  
{  
    return $this->belongsTo(related: User::class);  
}
```

- **items()**: Es una relación uno a muchos (hasMany), que se relaciona con el modelo de OrderItem, cuya clave foránea es “order_id” en la tabla order_items. En resumen: el pedido contiene múltiples productos, incluyendo cantidad, talla y precio unitario.

```
public function items(): HasMany  
{  
    return $this->hasMany(related: OrderItem::class);  
}
```

- Métodos personalizados:

- **createFromCartItems(\$user, \$cartItems)**: Crea un pedido nuevo a partir de los ítems del carrito de un usuario. Calcula el total (aplicando descuentos), registra cada ítem como OrderItem, actualiza el stock en la tabla pivot product_size y elimina los ítems del carrito. Todo se hace dentro de una transacción para garantizar la integridad de los datos.

```
public static function createFromCartItems($user, $cartItems): mixed
{
    return DB::transaction(callback: function () use ($user, $cartItems): Order {

        // Calculamos el total con descuentos
        $total = 0;
        foreach ($cartItems as $item) {
            $price = $item->product->discount > 0
                ? round(num: $item->product->price * (1 - $item->product->discount / 100), precision: 2)
                : $item->product->price;

            $total += $price * $item->quantity;
        }

        // Creamos el pedido
        $order = self::create(attributes: [
            'user_id' => $user->id,
            'total' => $total,
            'status' => 'pendiente',
        ]);

        // Creamos los order_items y actualizamos stock
        foreach ($cartItems as $item) {
            $price = $item->product->discount > 0
                ? round(num: $item->product->price * (1 - $item->product->discount / 100), precision: 2)
                : $item->product->price;

            OrderItem::create(attributes: [
                'order_id' => $order->id,
                'product_id' => $item->product_id,
                'size_id' => $item->size_id,
                'quantity' => $item->quantity,
                'price' => $price,
            ]);

            // Reducir stock en tabla pivot product_size
            DB::table(table: 'product_size')
                ->where(column: 'product_id', operator: $item->product_id)
                ->where(column: 'size_id', operator: $item->size_id)
                ->decrement(column: 'stock', amount: $item->quantity);

            // Eliminar item del carrito
            $item->delete();
        }

        return $order;
    });
}
```

5.2.5 - ORDERITEM.PHP

- Traits usados:

- **HasFactory**: Habilita el uso de factorías para generar instancias de prueba.

```
use HasFactory;
```

- Campos asignables:

- **order_id**: ID del pedido al que pertenece este ítem
- **product_id**: ID del producto comprado
- **size_id**: ID de la talla seleccionada del producto
- **quantity**: Cantidad de este producto en el pedido
- **price**: Precio unitario del producto en el momento del pedido

```
protected $fillable = [  
    'order_id',  
    'product_id',  
    'size_id',  
    'quantity',  
    'price',  
];
```

- Relaciones:

- **order()**: Es una relación inversa uno a muchos (belongsTo), que se relaciona con el modelo de Order, cuya clave foránea es “order_id” en la tabla order_items. En resumen: un producto/ítem pertenece a un pedido específico.

```
public function order(): BelongsTo  
{  
    return $this->belongsTo(related: Order::class);  
}
```

- **product()**: Es una relación inversa uno a muchos (belongsTo), que se relaciona con el modelo de Product, cuya clave foránea es “product_id” en la tabla order_items. En resumen: el ítem del pedido está relacionado con un producto de la tienda.

```
public function product(): BelongsTo  
{  
    return $this->belongsTo(related: Product::class);  
}
```

- **size()**: Es una relación inversa uno a muchos (belongsTo), que se relaciona con el modelo de Size, cuya clave foránea es “size_id” en la tabla order_items. En resumen: el ítem del pedido está asociado a una talla específica del producto.

```
public function size(): BelongsTo  
{  
    return $this->belongsTo(related: Size::class);  
}
```

5.2.6 - PRODUCT.PHP

• Traits usados:

- **HasFactory**: Habilita factorías para instancias de prueba.

```
use HasFactory;
```

• Tabla asociada:

- **products**: Se asocia el modelo a esa tabla de la base de datos.

```
protected $table = 'products';
```

• Campos asignables:

- **name**: Nombre del producto.
- **description**: Descripción detallada del producto.
- **color**: Color del producto.
- **gender**: Género al que va dirigido (ejemplo: hombre, mujer, unisex).
- **style**: Estilo del producto.
- **category**: Categoría a la que pertenece el producto.
- **price**: Precio base del producto.
- **discount**: Porcentaje de descuento aplicado al producto.

```
protected $fillable = [  
    'name',  
    'description',  
    'color',  
    'gender',  
    'style',  
    'category',  
    'price',  
    'discount'  
];
```

• Relaciones:

- **images()**: Es una relación uno a muchos (hasMany), que se relaciona con el modelo de Image, cuya clave foránea es "product_id" en la tabla images. En resumen: un producto puede tener varias imágenes asociadas.

```
public function images(): HasMany  
{  
    return $this->hasMany(related: Image::class);  
}
```

- **sizes()**: Es una relación muchos a muchos (belongsToMany), que se relaciona con el modelo de Size. Tiene una tabla pivote que es product_size, que almacena el stock específico de cada talla a través de withPivot('stock'). En resumen: un producto puede tener varias tallas disponibles.

```
public function sizes(): BelongsToMany  
{  
    // withTimestamps() mantiene los timestamps en la tabla pivote  
    return $this->belongsToMany(related: Size::class)->withPivot(columns: 'stock')->withTimestamps();  
}
```

5.2.7 - SIZE.PHP

- Campos asignables:
 - **name:** Nombre de la talla (por ejemplo, "S", "M", "L", "XL").
- Relaciones:
 - **products():** Es una relación muchos a muchos (belongsToMany), que se relaciona con el modelo de Product. Tiene una tabla pivote que es product_size, que contiene el campo stock, y que representa la cantidad disponible para cada talla-producto. En resumen: una talla puede estar asociada a muchos productos, y cada producto puede tener varias tallas.

```
public function products(): BelongsToMany
{
    return $this->belongsToMany(related: Product::class)->withPivot(columns: 'stock')->withTimestamps();
}
```

5.2.8 - USER.PHP

- Traits usados:
 - **HasFactory:** Permite usar factorías para pruebas y seeders.
 - **Notifiable:** Permite el envío de notificaciones al usuario (por ejemplo, correo electrónico).

```
use HasFactory, Notifiable;
```

- Campos asignables:
 - **name:** Nombre completo del usuario.
 - **email:** Correo electrónico del usuario (utilizado para la autenticación).
 - **password:** Contraseña del usuario (almacenada como hash).
 - **role:** Rol del usuario (por ejemplo, "admin" o "usuario normal").

```
protected $fillable = [
    'name',
    'email',
    'password',
    'role',
];
```

- Campos ocultos (\$hidden):
 - **password:** La contraseña será oculta al serializar el modelo (por ejemplo, en respuestas JSON).
 - **remember_token:** Token usado para mantener la sesión iniciada.

```
protected $hidden = [
    'password',
    'remember_token',
];
```

- **casts():**
 - **email_verified_at:** Se convierte un objeto datetime para manejar la verificación del correo.
 - **password:** Se guarda como un hash automáticamente.

```
protected function casts(): array
{
    return [
        'email_verified_at' => 'datetime',
        'password' => 'hashed',
    ];
}
```

a

- **Relaciones:**
 - **cart():** Es una relación uno a uno (hasOne), que se relaciona con el modelo de Cart, cuya clave foránea es “user_id” en la tabla cart. En resumen: un usuario tiene un solo carrito de compras.

```
public function cart(): HasOne
{
    return $this->hasOne(releted: Cart::class);
}
```

- **order():** Es una relación uno a muchos (hasMany), que se relaciona con el modelo de Order, cuya clave foránea es “user_id” en la tabla order. En resumen: un usuario puede tener muchos pedidos.

```
public function orders(): HasMany
{
    return $this->hasMany(releted: Order::class);
}
```

- **Métodos personalizados:**
 - **initials():** Se creó automáticamente al crear el proyecto. Devuelve las iniciales del nombre completo del usuario.

```
public function initials(): string
{
    return Str::of(string: $this->name)
        ->explode(delimiter: ' ')
        ->map(callback: fn(string $name): Stringable => Str::of(string: $name)->substr(start: 0, length: 1))
        ->implode(value: '');
}
```

- **getOrCreateCart():** Devuelve el carrito del usuario si existe, o lo crea si no lo tiene.

```
public function getOrCreateCart(): Cart
{
    return $this->cart()->firstOrCreate(attributes: []);
}
```

5.3 – SEEDERS

5.3.1 – DATABASESEEDER.PHP

- Propósito: Seeder principal que se encarga de inicializar datos básicos para la aplicación, incluyendo usuarios de prueba y llamar a otros seeders para poblar productos e imágenes.
- Funcionalidad: Crea dos usuarios con roles y datos predeterminados:
 - Usuario normal (role por defecto).
 - Usuario administrador (role = 1) con email propio.

```
// Crear un usuario normal de prueba
User::factory()->create([
    'name' => 'TestUser',
    'email' => 'test@example.com',
    'password' => Hash::make(value: '123456789'),
]);

// Crear un usuario administrador
User::factory()->create([
    'name' => 'Judit',
    'email' => 'juditquirosviolero@gmail.com',
    'password' => Hash::make(value: '123456789'),
    'role' => '1',
]);
```

Llama a los seeders ProductSeeder e ImageSeeder para cargar datos adicionales relacionados con productos y sus imágenes.

```
$this->call(class: [
    ProductSeeder::class,
    ImageSeeder::class,
]);
```

5.3.2 – PRODUCTSEEDER.PHP

- Propósito: Poblar la base de datos con tallas y productos iniciales para la tienda.
- Funcionalidad:
 - Crea las tallas de ropa (XS, S, M, L, XL, XXL) y las tallas de calzado (36 a 45) en la tabla sizes.

```
$clothingSizes = ['XS', 'S', 'M', 'L', 'XL', 'XXL'];
foreach ($clothingSizes as $size) {
    Size::create(attributes: ['name' => $size]);
}

// Crear tallas de Calzado del 36 al 45
for ($i = 36; $i <= 45; $i++) {
    Size::create(attributes: ['name' => (string) $i]);
}
```


- Obtiene todas las tallas de la base de datos y las separa en dos grupos: tallas numéricas (calzado) y no numéricas (ropa).

```
$sizes = Size::all();

// Separar tallas numéricas (Calzado) y no numéricas (ropa)
$shoeSizes = $sizes->filter(callback: fn(Size $s): bool => is_numeric(value: $s->name));
$clothingSizes = $sizes->filter(callback: fn(Size $s): bool => !is_numeric(value: $s->name));
```

- Crea un array grande con muchos productos, cada uno con atributos como nombre, descripción, estilo, género, color, categoría, y precio.

```
$products = [
    ['name' => 'Abrigo largo de pelo', 'description' => 'Abrigo largo de pelo, ideal para invierno.', 'style' => 'Elegante', 'gender' => 'Mujer', 'color' => 'Gris', 'category' => 'Abrigos', 'price' => 89.99],
    ['name' => 'Chaqueta ajustada', 'description' => 'Chaqueta ajustada para oficina o eventos formales.', 'style' => 'Elegante', 'gender' => 'Hombre', 'color' => 'Negro', 'category' => 'Chaquetas', 'price' => 69.99],
    ['name' => 'Botas altas de cuero', 'description' => 'Botas altas de cuero con detalles de hebilla, para el invierno.', 'style' => 'Elegante', 'gender' => 'Mujer', 'color' => 'Negro', 'category' => 'Calzado', 'price' => 99.99],
    ['name' => 'Botas de montaña', 'description' => 'Botas de montaña resistentes y cómodas.', 'style' => 'Deportivo', 'gender' => 'Unisex', 'color' => 'Marrón', 'category' => 'Calzado', 'price' => 84.99],
    ['name' => 'Camisa de lino', 'description' => 'Camisa de lino ligera, perfecta para el verano.', 'style' => 'Casual', 'gender' => 'Hombre', 'color' => 'Blanco', 'category' => 'Camisas', 'price' => 39.99],
    ['name' => 'Camiseta básica', 'description' => 'Camiseta cómoda de algodón 100% para cualquier ocasión casual.', 'style' => 'Casual', 'gender' => 'Unisex', 'color' => 'Azul', 'category' => 'Camisetas', 'price' => 19.99],
    ['name' => 'Camiseta gráfica', 'description' => 'Camiseta con cuello en V y estampado gráfico.', 'style' => 'Casual', 'gender' => 'Unisex', 'color' => 'Blanco', 'category' => 'Camisetas', 'price' => 22.99],
    ['name' => 'Camiseta de rayas', 'description' => 'Camiseta de rayas en tonos clásicos, un must en tu armario.', 'style' => 'Clásico', 'gender' => 'Mujer', 'color' => 'Blanco', 'category' => 'Camisetas', 'price' => 24.99],
    ['name' => 'Chaqueta de cuero', 'description' => 'Chaqueta de cuero negro con detalles metálicos, un clásico.', 'style' => 'Moderno', 'gender' => 'Hombre', 'color' => 'Negro', 'category' => 'Chaquetas', 'price' => 129.99],
    ['name' => 'Chaqueta de lana', 'description' => 'Chaqueta de lana en color gris, perfecta para eventos formales.', 'style' => 'Elegante', 'gender' => 'Mujer', 'color' => 'Gris', 'category' => 'Chaquetas', 'price' => 89.99],
    ['name' => 'Chaqueta impermeable', 'description' => 'Chaqueta impermeable con capucha para lluvia.', 'style' => 'Deportivo', 'gender' => 'Unisex', 'color' => 'Azul Marino', 'category' => 'Chaquetas', 'price' => 74.99],
    ['name' => 'Conjunto casual femenino', 'description' => 'Conjunto casual femenino con blusa blanca ligera y falda midi fluida beige, perfecto para días cálidos.', 'style' => 'Casual', 'gender' => 'Mujer', 'color' => 'Beige', 'category' => 'Conjuntos', 'price' => 59.99],
    ['name' => 'Conjunto deportivo uomo', 'description' => 'Conjunto deportivo compuesto por sudadera con capucha y pantalón deportivo en color verde, cómodo y funcional para entrenamiento.', 'style' => 'Deportivo', 'gender' => 'Hombre', 'color' => 'Verde', 'category' => 'Conjuntos', 'price' => 44.99],
    ['name' => 'Chaleco elegante gris', 'description' => 'Elegante chaleco de vestir en gris oscuro, ideal para eventos y oficinas.', 'style' => 'Elegante', 'gender' => 'Hombre', 'color' => 'Gris', 'category' => 'Chalecos', 'price' => 49.99],
    ['name' => 'Conjunto elegante completo', 'description' => 'Elegante conjunto elegante que incluye chaqueta, chaleco y pantalón de vestir en gris oscuro, ideal para eventos y oficinas.', 'style' => 'Elegante', 'gender' => 'Hombre', 'color' => 'Gris', 'category' => 'Conjuntos', 'price' => 149.99],
    ['name' => 'Falda larga beige', 'description' => 'Falda larga en tela fluida, perfecta para el verano o el estilo bohemio.', 'style' => 'Casual', 'gender' => 'Mujer', 'color' => 'Beige', 'category' => 'Faldas', 'price' => 34.99],
    ['name' => 'Falda floral', 'description' => 'Falda plisada midi con diseño floral.', 'style' => 'Bohemio', 'gender' => 'Mujer', 'color' => 'Rosa', 'category' => 'Faldas', 'price' => 39.99],
    ['name' => 'Jersey grueso', 'description' => 'Jersey de lana gruesa para días fríos.', 'style' => 'Casual', 'gender' => 'Mujer', 'color' => 'Gris Claro', 'category' => 'Jerseys', 'price' => 49.99],
    ['name' => 'Jersey fino beige', 'description' => 'Jersey de punto fino para uso diario.', 'style' => 'Casual', 'gender' => 'Unisex', 'color' => 'Beige', 'category' => 'Jerseys', 'price' => 39.99],
    ['name' => 'Pantalón cargo', 'description' => 'Pantalón cargo cómodo y práctico, con múltiples bolsillos.', 'style' => 'Casual', 'gender' => 'Unisex', 'color' => 'Gris', 'category' => 'Pantalones', 'price' => 34.99],
    ['name' => 'Pantalón deportivo slim', 'description' => 'Pantalón deportivo de tela con corte slim para entrenar.', 'style' => 'Deportivo', 'gender' => 'Hombre', 'color' => 'Azul', 'category' => 'Pantalones', 'price' => 29.99],
    ['name' => 'Pantalón elegante negro', 'description' => 'Pantalón elegante en color negro, ideal para oficina o eventos elegantes.', 'style' => 'Elegante', 'gender' => 'Hombre', 'color' => 'Negro', 'category' => 'Pantalones', 'price' => 44.99],
    ['name' => 'Pantalón palazzo', 'description' => 'Pantalón palazzo fluido y elegante.', 'style' => 'Elegante', 'gender' => 'Mujer', 'color' => 'Negro', 'category' => 'Pantalones', 'price' => 41.99],
    ['name' => 'Pantalones cortos', 'description' => 'Pantalones cortos de mezclilla desgastada.', 'style' => 'Casual', 'gender' => 'Unisex', 'color' => 'Azul Claro', 'category' => 'Pantalones', 'price' => 24.99],
    ['name' => 'Sandalias de cuero', 'description' => 'Sandalias de cuero con tiras finas.', 'style' => 'Casual', 'gender' => 'Mujer', 'color' => 'Marrón', 'category' => 'Calzado', 'price' => 34.99],
    ['name' => 'Sudadera blanca', 'description' => 'Sudadera cómoda con capucha, ideal para el invierno.', 'style' => 'Deportivo', 'gender' => 'Hombre', 'color' => 'Blanco', 'category' => 'Sudaderas', 'price' => 39.99],
    ['name' => 'Sudadera oscura', 'description' => 'Sudadera oscura con bolsillo canguro.', 'style' => 'Urbano', 'gender' => 'Unisex', 'color' => 'Gris Oscuro', 'category' => 'Sudaderas', 'price' => 44.99],
    ['name' => 'Conjunto urbano negro', 'description' => 'Sudadera oscura negra y pantalón jogger ajustado en tejido cómodo, ideal para un look urbano moderno.', 'style' => 'Urbano', 'gender' => 'Unisex', 'color' => 'Negro', 'category' => 'Conjuntos', 'price' => 69.99],
    ['name' => 'Vestido corto de verano', 'description' => 'Vestido corto de verano, cómodo y fresco para el calor.', 'style' => 'Casual', 'gender' => 'Mujer', 'color' => 'Azul', 'category' => 'Vestidos', 'price' => 34.99],
    ['name' => 'Vestido de playa moderno', 'description' => 'Vestido de playa moderno, ideal para la playa.', 'style' => 'Bohemio', 'gender' => 'Mujer', 'color' => 'Blanco', 'category' => 'Vestidos', 'price' => 29.99],
    ['name' => 'Vestido de playa', 'description' => 'Vestido de playa, ideal para la playa.', 'style' => 'Bohemio', 'gender' => 'Mujer', 'color' => 'Blanco', 'category' => 'Vestidos', 'price' => 29.99],
    ['name' => 'Vestido largo de noche beige', 'description' => 'Vestido largo de noche con encaje y detalles brillantes.', 'style' => 'Elegante', 'gender' => 'Mujer', 'color' => 'Beige', 'category' => 'Vestidos', 'price' => 89.99],
    ['name' => 'Vestido largo de noche turquesa', 'description' => 'Vestido largo de noche con encaje y detalles brillantes.', 'style' => 'Elegante', 'gender' => 'Mujer', 'color' => 'Turquesa', 'category' => 'Vestidos', 'price' => 89.99],
    ['name' => 'Calzado de cuero negro', 'description' => 'Calzado de cuero negro de alta calidad, ideales para ocasiones formales.', 'style' => 'Elegante', 'gender' => 'Unisex', 'color' => 'Negro', 'category' => 'Calzado', 'price' => 79.99],
    ['name' => 'Calzado deportivo blanco', 'description' => 'Calzado deportivo y, para el día a día.', 'style' => 'Deportivo', 'gender' => 'Unisex', 'color' => 'Blanco', 'category' => 'Calzado', 'price' => 59.99],
    ['name' => 'Calzado casual negro', 'description' => 'Calzado para salir, tipo converse.', 'style' => 'Casual', 'gender' => 'Unisex', 'color' => 'Negro', 'category' => 'Calzado', 'price' => 39.99],
];
```

- Crea un array de descuentos que se asignarán a algunos productos, en orden.
- Itera sobre los productos, asigna un descuento de la lista si queda alguno y crea cada producto en la base de datos.

```
$index = 0;

foreach ($products as $productData) {
    $discount = null;

    // Asignar descuento si hay disponible en la lista
    if ($index < count(value: $discounts)) {
        $discount = $discounts[$index];
        $index++;
    }

    $product = Product::create(attributes: [
        'name' => $productData['name'],
        'description' => $productData['description'],
        'color' => $productData['color'],
        'gender' => $productData['gender'],
        'style' => $productData['style'],
        'category' => $productData['category'],
        'price' => $productData['price'],
        'discount' => $discount,
    ]);
}
```

- Según la categoría del producto, asigna un grupo de tallas (calzado o ropa).

```
$sizePool = $productData['category'] === 'Calzado' ? $shoeSizes : $clothingSizes;
```

- Asigna algunas tallas a cada producto, eligiendo entre 1 y todas las tallas disponibles al azar para ese producto.

```
$assignedSizes = $sizePool->random(number: rand(min: 1, max: $sizePool->count()));
```

- Las tallas se almacenan en la tabla sizes con un stock aleatorio de entre 5 y 50, y se relacionan con la tabla de productos

```
foreach ($assignedSizes as $size) {
    $product->sizes()->attach(ids: $size->id,
        'stock' => rand(min: 5, max: 50),
    );
}
```

5.3.3 - IMAGESEEDER.PHP

- Propósito: Asignar imágenes relacionadas a los productos ya creados.
- Funcionalidad: Define un array de imágenes donde cada elemento tiene:
 - product_id: referencia al producto al que pertenece la imagen.
 - url: ruta relativa al archivo de imagen.

```
$images = [
    // 1. Abrigo largo mujer camel
    ['product_id' => 1, 'url' => 'fotos_productos/abrigo_largo_muj_camel_1.png'],
    ['product_id' => 1, 'url' => 'fotos_productos/abrigo_largo_muj_camel.png'],

    // 2. Blazer hombre negro
    ['product_id' => 2, 'url' => 'fotos_productos/blazer_hom_negro_1.png'],
    ['product_id' => 2, 'url' => 'fotos_productos/blazer_hom_negro.png'],

    // 3. Botas
    ['product_id' => 3, 'url' => 'fotos_productos/botas_altas_muj_negro.jpg'],
    ['product_id' => 4, 'url' => 'fotos_productos/botas_montaña_uni_marron.png'],
```

...

```
    // 32. Zapatos formal unisex negro
    ['product_id' => 34, 'url' => 'fotos_productos/zapatos_formal_uni_negro_1.jpg'],
    ['product_id' => 34, 'url' => 'fotos_productos/zapatos_formal_uni_negro.jpg'],

    // 33. Zapatos deporte unisex blanco
    ['product_id' => 35, 'url' => 'fotos_productos/zapatos_depor_uni_blanco.jpg'],

    // 34. Zapatos casual unisex naranja
    ['product_id' => 36, 'url' => 'fotos_productos/zapatos_casual_uni_naranja.png'],
];
```

Inserta cada imagen en la tabla images usando el modelo Image.

```
foreach ($images as $img) {
    Image::create(attributes: [
        'product_id' => $img['product_id'],
        'url' => $img['url'],
    ]);
}
```

- Detalles técnicos
 - Las imágenes se guardan en la carpeta fotos_productos dentro del proyecto.
 - Se crean múltiples imágenes por producto para ofrecer distintas vistas.

5.4 – COMPONENTES BLADE Y VISTAS BASE

Este apartado describe los componentes Blade que forman la estructura base de las vistas del proyecto. Estos archivos no contienen lógica dinámica como los componentes Livewire, pero son fundamentales para organizar y mantener una maquetación coherente y reutilizable en todo el ecommerce.

Se utilizan principalmente dos plantillas:

- `resources/views/component/layout/base.blade.php`: que actúa como layout general de la página web, e incluye en su interior elementos comunes como la cabecera, el menú de navegación, el pie de página, y un slot donde se insertan dinámicamente los componentes Livewire o el contenido específico de cada vista.
- `resources/views/component/layouts/app`: contiene dos archivos Blade que funcionan como *layouts* para las vistas accesibles desde el perfil de usuario. Estos layouts son:
 - `sidebar.blade.php`: estructura visual de la barra lateral del sitio.
 - `header.blade.php`: estructura visual del encabezado del panel de usuario.

Estos layouts permiten separar la presentación de la lógica de negocio y facilitan el mantenimiento de una experiencia de usuario consistente a lo largo del sitio.

5.4.1 – BASE.BLADE.PHP

- Estructura General:
 - Head: Incluye los metadatos, el título de la pestaña (dinámico con `{{ $metaTitle }}`), y los enlaces a los recursos de Bootstrap y Bootstrap Icons.

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Nombre Tienda - {{ $metaTitle }}</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet" />
  <link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.10.5/font/bootstrap-icons.css" rel="stylesheet" />
```

- Estilos CSS personalizados: Se definen estilos globales como el ancho del contenedor, el diseño responsive, y los bordes para inputs y formularios, manteniendo buenas prácticas de accesibilidad visual.

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  width: 100% !important;
  max-width: 100% !important;
}

.container {
  width: 95%;
  max-width: 100% !important;
  margin: auto;
  padding: 0;
}

.btn-sm {
  font-size: small;
}

textarea,
input,
select {
  border: 1px solid #000 !important;
}

.btn:focus,
textarea:focus,
input:focus,
select:focus {
  outline: none !important;
  border: 1px solid black !important;
}

.carousel-control-prev:focus-visible,
.carousel-control-next:focus-visible {
  outline: none;
  border: 1px solid #000;
  border-radius: 0.5rem;
  box-shadow: 0 0 2px #000;
}

@media (max-width: 950px) {
  .container {
    max-width: 100%;
    font-size: small;
  }
}
```

- Cuerpo (<body>): El contenido se estructura de la siguiente manera:

```
<body class="d-flex flex-column min-vh-100">
```

- @include('components.layouts.header'): Se carga el encabezado del sitio.

```
@include(view: 'components.layouts.header')
```

- @include('components.layouts.nav'): Se incluye la barra de navegación con enlaces a las secciones principales.

```
@include(view: 'components.layouts.nav')
```

- <main>: Sección principal de la página que muestra un título dinámico (\$tituloSeccion) y el contenido específico mediante {{ \$slot }}.

```
<main class="flex-grow-1 py-4">
  <div class="container">
    <h2 class="text-center fw-bold">{{ $tituloSeccion ?? 'Bienvenido a nuestra tienda' }}</h2>
    <br>
    {{ $slot }}
  </div>
</main>
```

- @include('components.layouts.footer'): Incluye el pie de página con enlaces legales y redes sociales.

```
@include(view: 'components.layouts.footer')
```

- `<script>`: etiqueta que carga un archivo JavaScript de Bootstrap Bundle. Este script permite que los componentes interactivos de Bootstrap como el carrusel y los modales.

```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
```

- Componentes incluidos:
 - `components/layouts/header.blade.php`: Encabezado superior que contiene el nombre de la tienda, el icono del logo, y enlaces a inicio de sesión, registro o perfil, dependiendo del estado de autenticación del usuario.

```
<header class="bg-light py-2 border-bottom">
  <div class="container d-flex justify-content-between align-items-center">
    <!-- logo + nombre -->
    <div class="d-flex align-items-center gap-2">
      @include(view: 'components.app-logo-icon')
      <h1 class="h4 mb-0">Nombre Tienda</h1>
    </div>
    <!-- login / registro -->
    @if (Route::has(name: 'login'))
      <div>
        @auth
          <a href="{{ url(path: '/dashboard') }}" class="mx-3 text-decoration-none text-dark" title="Ir al perfil" aria-label="Perfil de usuario">
            <i class="bi bi-person-circle" aria-hidden="true"></i> Perfil
          </a>
        @else
          <a href="{{ route(name: 'login') }}" class="mx-3 text-decoration-none text-dark" title="Iniciar sesión" aria-label="Iniciar sesión">
            Iniciar sesión
          </a> |
          @if (Route::has(name: 'register'))
            <a href="{{ route(name: 'register') }}" class="mx-3 text-decoration-none text-dark" title="Registrarse" aria-label="Registrarse">
              Registrarse
            </a>
          @endif
        @endauth
      </div>
    @endif
  </div>
</header>
```

- `components/layouts/nav.blade.php`: Barra de navegación principal, con enlaces a Inicio, Tienda y Sobre nosotros, y un acceso directo al carrito en caso de que el usuario esté autenticado.

```
<nav class="bg-white py-2 border-bottom">
  <div class="container d-flex justify-content-between align-items-center">
    <!-- Enlaces -->
    <div>
      <a href="{{ route(name: 'user.index') }}" class="me-3 p-1 text-decoration-none text-dark active" title="Ir a la página de inicio" aria-label="Inicio">Inicio</a>
      <a href="{{ route(name: 'user.tienda') }}" class="me-3 p-1 text-decoration-none text-dark" title="Ir a la tienda" aria-label="Tienda">Tienda</a>
      <a href="{{ route(name: 'user.aboutUs') }}" class="p-1 text-decoration-none text-dark" title="Saber más sobre nosotros" aria-label="Sobre Nosotros">Sobre Nosotros</a>
    </div>
    <!-- Botón del carrito/mensaje autenticación -->
    <div class="d-flex align-items-center">
      @if (auth()->check())
        <a href="{{ route(name: 'user.cart') }}" class="btn btn-outline-dark" title="Ver carrito" aria-label="Carrito de compras">
          <i class="bi bi-cart" aria-hidden="true"></i> Carrito
        </a>
      @else
        <div class="alert alert-warning text-center m-0 p-2" style="font-size: x-small; width: 100%; max-width: 300px;" role="alert" aria-live="polite">
          Inicia sesión para ver tu carrito.
        </div>
      @endif
    </div>
  </div>
</nav>
```

- **components/layouts/footer.blade.php**: Pie de página con enlaces útiles (Mapa Web, Contacto, Aviso Legal, Accesibilidad) y enlaces a redes sociales.

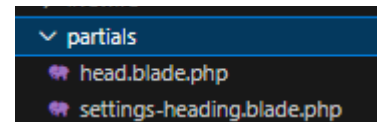
```
<footer class="bg-light mt-auto py-3 border-top">
  <div class="container text-center">
    <!-- Enlaces -->
    <div class="mb-2">
      <a href="{{ route('siteMap') }}" class="mx-3 p-1 text-decoration-none text-dark" title="Ir a la página del mapa web"
        aria-label="Mapa Web">Mapa Web</a> |
      <a href="{{ route('contact') }}" class="mx-3 p-1 text-decoration-none text-dark" title="Ir a la página de contacto"
        aria-label="Contacto">Contacto</a> |
      <a href="{{ route('legalNotice') }}" class="mx-3 p-1 text-decoration-none text-dark"
        title="Ir a aviso legal y privacidad" aria-label="Aviso legal - Privacidad">Aviso legal - Privacidad</a>
      |
      <a href="{{ route('accessibility') }}" class="mx-3 p-1 text-decoration-none text-dark"
        title="Ir a la página de accesibilidad" aria-label="Accesibilidad">Accesibilidad</a>
    </div>
    <!-- Redes -->
    <div>
      <a href="" class="me-3 text-dark text-decoration-none fs-5" title="Ir a Instagram" aria-label="Instagram">
        <i class="bi bi-instagram" aria-hidden="true"></i>
      </a>
      <a href="" class="me-3 text-dark text-decoration-none fs-5" title="Ir a Facebook" aria-label="Facebook">
        <i class="bi bi-facebook" aria-hidden="true"></i>
      </a>
      <a href="" class="text-dark text-decoration-none fs-5" title="Ir a TikTok" aria-label="TikTok">
        <i class="bi bi-tiktok" aria-hidden="true"></i>
      </a>
    </div>
  </div>
</footer>
```

- **Uso:** Para utilizar este layout en una vista, se debe utilizar la etiqueta `<x-layouts.base>`. Esta etiqueta actúa como contenedor principal de la vista y permite establecer el título de la pestaña del navegador y el encabezado visible de la sección principal, mediante los atributos `metaTitle` y `tituloSeccion`.
- Las vistas que utilizan este componente son:
 - **/user/aboutUs.blade.php**: Página de relleno.
 - **/user/cart.blade.php**: Llama al livewire `show.cart`.
 - **/user/index.blade.php**: Llama al livewire `index-page`.
 - **/user/orderConfirmation.blade.php**: Llama al livewire `order-confirmation`.
 - **/user/product.blade.php**: Llama al livewire `product-show`, y añade los comentarios de decoración. (Futura implementación)
 - **/user/tienda.blade.php**: Llama al livewire `show-store-prud`.
 - **accessibility.blade.php**: Contiene las características no accesibles de la web y enlaces a la página de contacto.

- **contact.blade.php**: Página con un formulario de contacto que no hace nada. (Futura implementación)
- **legalNotice.blade.php**: Página de relleno con texto.
- **siteMap.blade.php**: Contiene enlaces a todas las páginas de la web.

5.4.2 - HEADER.BLADE.PHP Y SIDEBAR.BLADE.PHP

Ambos layouts se crean solos al crear el proyecto, junto con otros archivos que también son utilizados en estas vistas como los de partials, es por eso que no se han eliminado y solo se les ha realizado algunas modificaciones, en especial a sidebar.blade.php.



```
<head>
    @include(view: 'partials.head')
```

Los dos archivos se crean utilizando el sistema Flux que permite una integración fluida con Livewire para funcionalidades reactivas y una experiencia dinámica en la navegación.

```
<flux:header class="lg:hidden border-b border-zinc-200 bg-zinc-50" aria-label="Menú de usuario móvil">
    <flux:sidebar.toggle class="lg:hidden icon="bars-2" inset="left" title="Abrir menú lateral"
        aria-label="Abrir menú lateral" />
    <flux:spacer />
    <flux:dropdown position="top" align="end" aria-label="Menú desplegable de perfil de usuario">
        <flux:profile :initials="auth() -> user() -> initials()" icon-trailing="chevron-down"
```

Aparte del flux, les he añadido bootstrap para que no tenga una vista demasiado diferente y además sea responsive, características accesibles como title y atributos ARIA, y algo de css.

```
<style>
    div.z-10.fixed {
        left: 255px !important;
        width: calc(100% - 255px) !important;
    }
</style>
```

```
<flux:navlist.item
    class="text-dark text-decoration-none border border-1 border-dark-subtle shadow-sm rounded"
    :href="route(name: 'user.index')" :current="request() -> routeIs(patterns: 'user.index')" wire:navigate
    title="Ir a la página principal de la tienda" aria-label="Ir a la Tienda">
    {{ __(key: 'Ir a la Tienda') }}
</flux:navlist.item>
```

También he añadido una comprobación para ver si el usuario es administrador(rol=1) o un usuario normal (rol=0), para mostrar diferentes enlaces en el aside dependiendo del rol.


```

@if (Auth::check() && Auth::user()->role == 1)
    <flux:navlist.group :heading="__(key: 'Panel de administrador')" class="grid fw-bold"
      aria-label="Sección Panel de administrador">
      <flux:navlist.item icon="home"
        class="text-dark text-decoration-none border border-1 border-dark-subtle shadow-sm rounded"
        :href="route(name: 'admin.dashboard')" :current="request() -> routeIs(patterns: 'admin.dashboard')"
        wire:navigate title="Ir al panel de administrador" aria-label="Panel de administrador">
        {{ __(key: 'Panel de administrador') }}
      </flux:navlist.item>
    </flux:navlist.group>

    <br />

    <flux:navlist.group :heading="__(key: 'Opciones de administrador')" class="grid fw-bold"
      aria-label="Opciones para administrador">
      <flux:navlist.item
        class="text-dark text-decoration-none d-flex align-items-center gap-2 border border-1 border-dark-s
        :href="route(name: 'admin.product')" :current="request() -> routeIs(patterns: 'admin.product')"
        wire:navigate title="Información de productos" aria-label="Información Productos">
        <i class="bi bi-box-seam me-2" aria-hidden="true"></i> {{ __(key: 'Información Productos') }}
      </flux:navlist.item>

```

Respecto a los elementos que traía por defecto la página como el texto, enlaces y botones del aside, los que he querido conservar los he escrito en español, y los que no los he borrado y he creado los que me han parecido reutilizando la forma en la que estaban creados.

Además le he añadido el footer para que las vistas que utilizan este layout sigan pareciendo del mismo sitio web, y he metido el código principal de la web (el slot que traía) dentro de una etiqueta main.

```

<div class="flex-1 flex flex-col">
  <!-- Contenido dinámico -->
  <main class="flex-1" tabindex="-1">
    {{ $slot }}
  </main>

  <!-- Footer fijo abajo dentro del contenido -->
  @include(view: 'components.layouts.footer')
</div>

```

Las vistas que usan estos layouts son:

- **dashboard.blade.php**: Solo sirve para redirigir dependiendo del rol.
- **/user/orders.blade.php**: Llama al livewire de see-my-orders.
- **/user/profile.blade.php**: Muestra el perfil del usuario normal (rol=0), y no llama a ningún livewire.
- **/admin/dashboard.blade.php**: Llama al livewire de admin-dashboard.
- **/admin/order.blade.php**: Llama al livewire de order-crud.
- **/admin/product.blade.php**: Llama al livewire de product-crud.
- **/admin/users.blade.php**: Llama al livewire de user-crud.

5.5 – COMPONENTES LIVEWIRE

En esta sección se describen todos los componentes Livewire implementados en el proyecto. Cada componente contiene una clase PHP ubicada en `/app/Http/Livewire` y su vista asociada en `/resources/views/livewire`. A través de estos componentes se gestiona la interacción dinámica del usuario con el sistema, como la visualización de productos, gestión del carrito, administración de pedidos, entre otras funcionalidades.

5.5.1 – ADDTOCART.PHP

FUNCIONALIDAD: Permite al usuario autenticado seleccionar una talla y una cantidad de un producto determinado, y añadirlo al carrito. Está diseñado para ser utilizado en la página de la tienda donde se muestran todos los productos y en la que se muestran los detalles del producto.

- Variables:

- **product:** instancia del modelo Product que representa el producto mostrado.
- **sizeId:** ID de la talla seleccionada por el usuario.
- **quantity:** cantidad de productos a añadir, con valor por defecto de 1.
- **message y messageType:** usados para mostrar mensajes de éxito o error tras la acción de añadir al carrito.

```
2 references  
public $product;  
2 references  
public $sizeId;  
1 reference  
public $quantity = 1;
```

```
5 references  
public $message = null;  
5 references  
public $messageType = 'success';
```

- Método addToCart():

- Verifica si el usuario está autenticado.
- Valida que exista un producto y que se haya seleccionado una talla.

```
public function addToCart(): void  
{  
    // Obtener el usuario autenticado  
    $user = auth()->user();  
  
    // Validar que el usuario esté logueado  
    if (!$user) {  
        $this->message = 'Debes iniciar sesión para añadir productos al carrito.';  
        $this->messageType = 'error';  
        return; // Detener ejecución si no hay usuario  
    }  
  
    // Validar que el producto sea válido (exista)  
    if (!$this->product) {  
        $this->message = 'Producto no válido.';  
        $this->messageType = 'error';  
        return;  
    }  
  
    // Validar que se haya seleccionado una talla  
    if (!$this->sizeId) {  
        $this->message = 'Por favor selecciona una talla.';  
        $this->messageType = 'error';  
        return;  
    }  
}
```

- Obtiene o crea un carrito asociado al usuario usando `$user->getOrCreateCart()`.
- Añade el producto al carrito con la talla y cantidad seleccionada usando `$cart->addProduct(...)`, que está en el modelo del carrito: `Cart.php`
- Muestra un mensaje de éxito o error según el resultado.

```
try {
    // Obtener o crear el carrito asociado al usuario
    $cart = $user->getOrCreateCart();

    // Añadir el producto con la talla y cantidad indicada al carrito
    $cart->addProduct(productId: $this->product->id, sizeId: $this->sizeId, quantity: $this->quantity);

    // Mensaje de éxito para el usuario
    $this->message = 'Producto añadido al carrito.';
    $this->messageType = 'success';
} catch (\Exception $e) {
    // Capturar cualquier error y mostrar mensaje de error
    $this->message = 'Error: ' . $e->getMessage();
    $this->messageType = 'error';
}
```

- Renderizado: Retorna la vista `livewire.add-to-cart`.

```
public function render(): View
{
    return view(view: 'livewire.add-to-cart');
}
```

VISTA:

- Mensaje de estado (`$message`): Se muestra un div con clase `alert` y color dinámico según `$messageType` (`success` o `danger`).

```
@if ($message)
    <div id="mensaje" class="alert alert-{{ $messageType == 'success' ? 'success' : 'danger' }} mb-4"
        role="alert" aria-label="Mensaje del sistema">
        {{ $message }}
    </div>
@endif
```

- Selección de talla: Tiene un encabezado con texto “Selecciona una talla”, y una lista de radio buttons que se enlazan con `wire:model="sizeId"`, y cuyo contenido es generado dinámicamente a partir de `$product->sizes`.

```
<p id="talla-encabezado" class="mt-3 mb-2 fw-bold" >Selecciona una talla:</p>
<div id="tallas" class="d-flex flex-wrap gap-2" role="radiogroup" aria-label="Opciones de tallas disponibles">
  @foreach ($product->sizes as $size)
    <div class="form-check">
      <input
        class="form-check-input border border-dark focus-ring"
        type="radio"
        wire:model="sizeId"
        id="size-{{ $size->id }}"
        name="size"
        value="{{ $size->id }}"
        aria-label="Talla {{ strtoupper(string: $size->name) }}"
        title="Selecciona la talla {{ strtoupper(string: $size->name) }}"
      >
      <label class="form-check-label" for="size-{{ $size->id }}">
        {{ strtoupper(string: $size->name) }}
      </label>
    </div>
  @endforeach
</div>
```

- Selección de la cantidad: Es un campo numérico enlazado con `wire:model="quantity"`, con mínimo de 1.

```
<p id="cantidad-encabezado" class="mb-1 fw-bold">Cantidad:</p>
<div class="d-flex gap-2 justify-content-between align-items-end">
  <label for="cantidad" class="visually-hidden">Introduce la cantidad</label>
  <input
    type="number"
    id="cantidad"
    name="cantidad"
    wire:model="quantity"
    min="1"
    class="form-control border border-dark"
    aria-label="Campo para introducir la cantidad"
    title="Introduce la cantidad"
  />
</div>
```

- Botón “Añadir al carrito”: Al pulsarlo se ejecuta el método `addToCart()`. Pero si el usuario no ha iniciado sesión (`auth()->check()`), el botón se deshabilita.

```
<button
  wire:click="addToCart"
  id="boton-add-cart"
  class="btn btn-primary form-control"
  {{ auth()->check() ? '' : 'disabled' }}
  aria-label="Añadir al carrito"
  title="Añadir al carrito"
>
  <i class="bi bi-cart-plus" aria-hidden="true"></i>
  Añadir al carrito
</button>
```

- Mensajes alternativos:
 - Si no hay `$product`, se muestra mensaje de advertencia “Producto no disponible”.

```
<div class="alert alert-warning" role="alert" aria-label="Producto no disponible">
  Producto no disponible.
</div>
```

- Si el usuario no está autenticado, se muestra un mensaje indicando que debe iniciar sesión para añadir productos.

```
@if (!auth()->check())
    <div id="alerta-login" class="d-flex justify-content-end mt-2">
        <div class="fw-bold alert alert-warning"
            style="font-size: x-small; color: black"
            role="alert"
            aria-label="Debes iniciar sesión para añadir productos al carrito"
            title="Debes iniciar sesión para añadir productos al carrito">
            <div>
                *Inicia sesión para poder añadir productos al carrito.
            </div>
        </div>
    </div>
@endif
```

5.5.2 – ADMIN DASHBOARD.PHP

FUNCIONALIDAD: Muestra un resumen general del sistema: productos, pedidos, usuarios, actividad reciente y alertas por bajo stock.

- Variables:

- **lowStockProducts:**
almacena productos que tienen alguna talla con stock igual o menor a 5.

```
// Lista de productos con stock bajo en alguna talla
1 reference
public $lowStockProducts = [];
```

- **mount():** Se ejecuta automáticamente al cargar el componente.
 - Filtra y carga todos los productos que tienen al menos una talla con stock ≤ 5 .
 - Solo se cargan las tallas con bajo stock, optimizando el rendimiento y visualización.

```
public function mount(): void
{
    // Filtrar productos que tengan al menos una talla con poco stock (<= 5)
    $this->lowStockProducts = Product::whereHas('sizes', callback: function (Builder<TRelatedModel> $query) {
        $query->where(column: 'product_size.stock', operator: '<=', value: 5); // Condición sobre la tabla intermedia product_size
    })
    ->with(relations: [
        'sizes' => function ($query): void {
            $query->where('product_size.stock', '<=', 5); // Cargar solo las tallas con bajo stock
        }
    ])
    ->get(); // Obtener los productos que cumplen con esa condición
}
```

- **render():** Devuelve la vista `livewire.admin-dashboard` con los siguientes datos:
 - **products:** Todos los productos de la base de datos.
 - **orders:** Todos los pedidos registrados.
 - **users:** Todos los usuarios del sistema.
 - **recentOrders:** Los 5 pedidos más recientes.
 - **recentProducts:** Los 5 productos más recientes.

```
public function render(): View
{
    return view(view: 'livewire.admin-dashboard', data: [
        'products' => Product::all(),
        'orders' => Order::all(),
        'users' => User::all(),
        'recentOrders' => Order::latest()->take(value: 5)->get(),
        'recentProducts' => Product::latest()->take(value: 5)->get(),
    ]);
}
```

VISTA:

- **Encabezado:** Muestra información del administrador autenticado: nombre, fecha de registro y enlace para editar su perfil.

```
<div
  class="flex items-center gap-6 p-6 bg-white dark:bg-neutral-900 rounded-xl border border-neutral-200 dark:border-neutral-700 shadow-md">
  
  <div>
    <h2 class="text-2xl font-semibold text-gray-900 dark:text-white">¡Hola, {{ Auth::user()->name }}!</h2>
    <p class="text-gray-600 dark:text-gray-300">Administrador desde
      {{ ucfirst(string: Auth::user()->created_at->translatedFormat('F \d\e Y')) }}</p>
    <p class="mt-2 text-sm text-indigo-600 dark:text-indigo-400 font-medium"><a
      href="{{ route(name: 'settings.profile') }}">Editar perfil</a></p>
  </div>
</div>
```

- **Tarjetas informativas:** Visualiza el resumen de los productos, pedidos y usuarios.
 - **Gestión de productos:** total y enlace al crud de productos.

```
<div
  class="bg-white dark:bg-neutral-900 p-4 rounded-xl border border-neutral-200 dark:border-neutral-700 shadow-sm">
  <h3 class="text-lg font-semibold mb-2 text-gray-900 dark:text-white">Gestión de Productos</h3>
  @if ($products->count())
    <p class="text-sm text-gray-600 dark:text-gray-300">Total de productos en la tienda:
      {{ $products->count() }}</p>
  @else
    <p>No hay productos en la tienda.</p>
  @endif

  <a href="{{ route(name: 'admin.product') }}"
    class="text-sm text-indigo-600 dark:text-indigo-400 font-medium mt-2 inline-block">Ver productos</a>
</div>
```

- Gestión de pedidos: total y enlace al crud de pedidos.

```
<div
  class="bg-white dark:bg-neutral-900 p-4 rounded-xl border border-neutral-200 dark:border-neutral-700 shadow-sm">
  <h3 class="text-lg font-semibold mb-2 text-gray-900 dark:text-white">Gestión de Pedidos</h3>
  <p class="text-sm text-gray-600 dark:text-gray-300">Total de pedidos realizados: {{ $orders->count() }}</p>
  <a href="{{ route(name: 'admin.order') }}"
    class="text-sm text-indigo-600 dark:text-indigo-400 font-medium mt-2 inline-block">Ver pedidos</a>
</div>
```

- Usuarios registrados: total y enlace al crud de usuarios.

```
<div
  class="bg-white dark:bg-neutral-900 p-4 rounded-xl border border-neutral-200 dark:border-neutral-700 shadow-sm">
  <h3 class="text-lg font-semibold mb-2 text-gray-900 dark:text-white">Usuarios Registrados</h3>
  <p class="text-sm text-gray-600 dark:text-gray-300">Total de usuarios: {{ $users->count() }}</p>
  <a href="{{ route(name: 'admin.users') }}"
    class="text-sm text-indigo-600 dark:text-indigo-400 font-medium mt-2 inline-block">Ver
    usuarios</a>
</div>
```

- Actividad reciente: Contiene una lista de los pedidos más recientes y su fecha, y los nuevos productos que se han añadido recientemente.

```
<div
  class="bg-white dark:bg-neutral-900 p-6 rounded-xl border border-neutral-200 dark:border-neutral-700 shadow-md">
  <h3 class="text-xl font-semibold mb-4 text-gray-900 dark:text-white">Actividad reciente</h3>
  <ul class="space-y-2 text-sm text-gray-700 dark:text-gray-300">
    @foreach ($recentOrders as $order)
      <li>🛒 Pedido #{{ $order->id }} realizado el {{ $order->created_at->format('d/m/Y') }}</li>
    @endforeach
    @foreach ($recentProducts as $product)
      <li>📦 Producto {{ $product->name }} añadido el {{ $product->created_at->format('d/m/Y') }}</li>
    @endforeach
  </ul>
</div>
```

- Mensaje de alerta de bajo stock:

Si hay productos con tallas en stock crítico (≤ 5), se muestra una alerta con el nombre del producto, talla y cantidad restante.

Si no hay problemas de stock, se muestra un mensaje positivo de confirmación.

```
@if ($lowStockProducts->count() > 0)
  <div class="alert alert-warning shadow-sm">
    <h5><i class="bi bi-exclamation-triangle-fill me-2"></i>Productos con poco stock:</h5>
    <ul class="mb-0">
      @foreach ($lowStockProducts as $product)
        <li>
          <strong>{{ $product->name }}</strong>
          <ul>
            @foreach ($product->sizes as $size)
              @if ($size->pivot->stock <= 5)
                <li>
                  <strong>{{ $size->name }}</strong> {{ $size->pivot->stock }} unidades restantes
                </li>
              @endif
            @endforeach
          </ul>
        </li>
      @endforeach
    </ul>
  </div>
@else
  <div class="alert alert-success shadow-sm">
    <i class="bi bi-check-circle-fill me-2"></i>Todos los productos tienen suficiente stock.
  </div>
@endif
```

5.5.3 – INDEXPAGE.PHP

FUNCIONALIDAD: Muestra productos destacados, promociones y enlaces a secciones clave del sitio, como el catálogo o el carrito.

- Variables:

- **categories:** Lista de categorías únicas de productos.
- **newProducts:** Últimos productos añadidos.
- **discountedProducts:** Productos con descuento.

```
public $categories = [];  
1 reference  
public $newProducts = [];  
1 reference  
public $discountedProducts = [];
```

- **mount():** Se ejecuta automáticamente al iniciar el componente y realiza las siguientes tareas:

- **Carga de categorías únicas:** Extrae las categorías distintas de la base de datos desde el modelo Product, elimina los valores nulos, las capitaliza, las ordena alfabéticamente y reinicia los índices del array para facilitar su uso en la vista.

```
public function mount(): void  
{  
    $this->categories = Product::whereNotNull(columns: 'category')  
        ->distinct()  
        ->pluck(column: 'category')  
        ->map(callback: fn($cat): string => ucfirst(string: $cat))  
        ->sort()  
        ->values();  
}
```

- **Carga de productos recientes:** Recupera los 10 productos más recientes mediante el método latest() y los guarda en la propiedad \$newProducts para mostrarlos en el carrusel.

```
$this->newProducts = Product::latest()->take(value: 10)->get();
```

- **Carga de productos en oferta:** Filtra los productos con un valor de discount mayor que 0, obtiene los primeros 10, y les calcula un campo adicional final_price que representa el precio con el descuento aplicado. Estos productos se almacenan en \$discountedProducts.

```
$this->discountedProducts = Product::where(column: 'discount', operator: '>', value: 0)  
    ->take(value: 10)  
    ->get()  
    ->map(callback: function (Product $product): Product {  
        $product->final_price = round(num: $product->price * (1 - $product->discount / 100), precision: 2);  
        return $product;  
    });
```


- getProductDescription(Product \$product): Genera una descripción aleatoria personalizada para cada producto, según su categoría.
 - Las frases incluyen menciones dinámicas a las tallas disponibles usando pluck('name')->join(', ').

```
public function getProductDescription($product): string
{
    $isShoe = strtolower(string: $product->category) === 'zapatos'; // Verifica si el producto es de categoría "zapatos"

    // Frases para productos de calzado
    $shoeTexts = [
        "¡Descubre el nuevo modelo de zapatos que está marcando tendencia! Hechos para acompañarte con estilo y comodidad todo el día.",
        "¿Buscas un calzado único? Estos zapatos son la combinación perfecta de diseño moderno y calidad.",
        "¡Tu próxima pisada con estilo empieza aquí! Disponible en tallas " . $product->sizes->pluck('name')->join(', ') . ". ",
        "Dale un giro a tu estilo con este calzado nuevo. Comodidad y diseño que se adapta a ti.",
        "¿A la moda y cómodo? Estos zapatos tienen todo lo que necesitas para tus pasos diarios.",
        "Elige tu talla favorita y camina con seguridad. Disponible en: " . $product->sizes->pluck('name')->join(', ') . ". ",
    ];

    // Frases para otras prendas (ropa en general)
    $clothingTexts = [
        "Renueva tu armario con esta prenda imprescindible. Ideal para combinar con cualquier look.",
        "¿Moda que habla por ti? Esta novedad en " . strtolower(string: $product->category) . " te hará destacar.",
        "Diseño, estilo y comodidad en una sola prenda. Disponible en tallas " . $product->sizes->pluck('name')->join(', ') . ". ",
        "Perfecta para cualquier ocasión. Dale un toque fresco y actual a tu outfit.",
        "Te encantará cómo se ve y se siente esta nueva prenda. ¡No puede faltar en tu colección!",
        "Confeccionada con materiales de calidad. Disponible en: " . $product->sizes->pluck('name')->join(', ') . ". ",
    ];
}
```

- Si el producto pertenece a la categoría "zapatos", se selecciona una frase aleatoria del array \$shoeTexts.
- Si es cualquier otra categoría, se usa una frase del array \$clothingTexts.

```
return $isShoe
    ? $shoeTexts[array_rand(array: $shoeTexts)]
    : $clothingTexts[array_rand(array: $clothingTexts)];
```

- render(): Devuelve la vista livewire.index-page.

```
public function render(): View
{
    return view(view: 'livewire.index-page');
}
```

VISTA:

- Carrusel de Novedades: Utiliza Bootstrap para mostrar el carrusel, y en él se muestran los productos más recientes. Cada diapositiva incluye una imagen destacada, el nombre del producto, una descripción generada dinámicamente, y un botón para ver el producto completo.

```
<div class="d-flex justify-content-center position-relative my-4">
  <div id="carouselNovedades" class="carousel slide w-75" data-bs-ride="carousel">
    <div class="carousel-inner">
      @foreach ($newProducts as $index => $product)
        <div class="carousel-item @if ($index == 0) active @endif">
          <div class="row g-0 align-items-center mb-5">
            <!-- Imagen con cartel de Novedades -->
            <div
              class="col-12 col-md-6 position-relative d-flex justify-content-center align-items-center">
              name }}" />
            </div>
          </div>
        </div>
      @endforeach
    </div>
  </div>
```



```

        <div
            class="col-12 col-md-6 d-flex flex-column justify-content-center gap-2 align-items-center text-center p-4">
                <h3 class="fw-bold">{{ $product->name }}</h3>
                <p class="text-muted fs-5 fst-italic">{{ $this->getProductDescription($product) }}</p>
                <a href="{{ route(name: 'productos.show', parameters: $product->id) }}" class="btn btn-primary mt-3">Ver
                    Producto</a>
            </div>
        </div>
    @endforeach
</div>
</div>

```

- Sección Informativa: Muestra cuatro tarjetas con íconos representativos (estilo, precios, tallas, envío) y un pequeño texto explicativo de los valores diferenciales de la tienda. Está puesto de relleno para la página de inicio.

```
<h2 class="mb-4 text-center fw-bold">Lo que nos hace únicos</h2>
<div class="row my-5 text-center">
  <div class="col-md-3 mb-4">
    <i class="bi bi-palette-fill" style="font-size: 60px;"></i>
    <h3 class="fw-bold h5 mt-3">Variedad de Estilos</h3>
    <p>Desde casual hasta elegante, para todos los gustos y ocasiones.</p>
  </div>
  <div class="col-md-3 mb-4">
    <i class="bi bi-tags" style="font-size: 60px;"></i>
    <h3 class="fw-bold h5 mt-3">Precios Accesibles</h3>
    <p>Moda de calidad que cuida tu bolsillo sin sacrificar estilo.</p>
  </div>
  <div class="col-md-3 mb-4">
    <i class="bi bi-rulers" style="font-size: 60px;"></i>
    <h3 class="fw-bold h5 mt-3">Tallas para Todos</h3>
    <p>Variedad en tallas y colores para que encuentres tu prenda ideal.</p>
  </div>
  <div class="col-md-3 mb-4">
    <i class="bi bi-truck" style="font-size: 60px;"></i>
    <h3 class="fw-bold h5 mt-3">Envíos Rápidos</h3>
    <p>Compra fácil y recibe tu pedido en tiempo récord.</p>
  </div>
</div>
```

- **Galería de Ofertas:** Muestra tarjetas con productos que tienen descuento. Cada tarjeta incluye imagen, nombre, descripción troncada, tallas disponibles y precio original tachado con el nuevo precio destacado en rojo.

```
<h2 class="mb-5 text-center fw-bold">Ofertas</h2>
<div class="row row-cols-2 row-cols-md-3 row-cols-lg-4 g-4">
  <foreach ($discountedProducts as $product)
    <div class="col">
      <a href="{{ route(name: 'productos.show', parameters: $product->id) }}" class="text-decoration-none text-dark">
        <div class="card h-100 shadow-sm">
          name }}"
            style="object-fit: cover; height: 180px;" />
          <div class="card-body d-flex flex-column">
            <p class="h5 card-title">{{ $product->name }}</p>
            <p class="card-text text-truncate" style="max-height: 4.5rem;>
              {{ $product->description }}</p>
            <p class="card-text">Tallas disponibles:
              {{ $product->sizes->pluck('name')->join(', ') . ', ' . ' ' }}</p>
            <p class="mb-0">
              <span
                class="text-muted text-decoration-line-through">{{ number_format(num: $product->price, decimals: 2) }}€</span>
              <span class="fw-bold text-danger ms-4 fs-4 ms-2">
                {{ number_format(num: $product->price * (1 - $product->discount / 100), decimals: 2) }}€
              </span>
            </p>
          </div>
        </div>
      </a>
    </div>
  </foreach>
</div>
```

Además se usa un <a> envolviendo cada tarjeta para que al pulsarla redirija a la vista donde se muestran los detalles del producto.

5.5.4 – ORDERCONFIRMATION.PHP

FUNCIONALIDAD: Permite al usuario confirmar su pedido con los productos previamente seleccionados desde el carrito. Incluye resumen detallado del pedido, cálculo del total (considerando descuentos), selección del método de pago (simulado) y opciones para confirmar o cancelar. También envía un correo de notificación al administrador con los detalles del pedido realizado.

- Variables:
 - **selectedItems:** guarda los ids de los productos seleccionados por el usuario desde el carrito.
 - **cartItems:** guarda las instancias completas de los productos seleccionados.
 - **totalPrice:** guarda el precio total de los productos seleccionados.
- **mount():** Carga los ítems seleccionados desde la sesión, obtiene las instancias completas de los ítems del carrito actual del usuario autenticado, y calcula el total de la compra al iniciar el componente.

```
public $selectedItems = [];  
3 references  
public $cartItems;  
1 reference  
public $totalPrice = 0;
```

```
public function mount(): void  
{  
    // Recuperamos los IDs de los productos seleccionados desde la sesión  
    $this->selectedItems = session(key: 'selectedItems', default: []);  
  
    // Cargamos los productos seleccionados del carrito del usuario autenticado  
    $this->cartItems = auth()->user()  
        ->cart  
        ->items()  
        ->whereIn('id', $this->selectedItems)  
        ->get();  
  
    // Calculamos el total del pedido  
    $this->updateTotal();  
}
```

- **Método updateTotal():** Recorre los productos seleccionados, calcula el precio total aplicando descuentos si tiene, y multiplica el precio final por la cantidad de unidades seleccionadas.

```

public function updateTotal(): void
{
    $total = 0; // Inicializamos el total en 0

    foreach ($this->cartItems as $item) {
        // Verificamos si el ítem está entre los seleccionados
        if (in_array(needle: $item->id, haystack: $this->selectedItems)) {

            // Precio base del producto (sin descuento)
            $price = $item->product->price;

            // Si el producto tiene un descuento, se aplica
            if ($item->product->discount && $item->product->discount > 0) {
                $discount = $item->product->discount;
                $price = $price - (($price * $discount / 100)); // Precio con descuento aplicado
            }

            // Sumamos al total la cantidad multiplicada por el precio (con o sin descuento)
            $total += $item->quantity * $price;
        }
    }

    // Asignamos el total calculado a la propiedad del componente
    $this->totalPrice = $total;
}

```

- Método `confirmOrder()`: Crea un nuevo pedido en la base de datos, envía un correo electrónico al administrador (juditquirosviolero@gmail.com) con los detalles del pedido usando la clase `EmailPedidos`, y por último redirige al usuario al dashboard con un mensaje de éxito.

```

public function confirmOrder(): mixed|RedirectResponse
{
    $order = Order::createFromCartItems(user: auth()->user(), cartItems: $this->cartItems);

    Mail::to(users: 'juditquirosviolero@gmail.com')->send(mailable: new EmailPedidos(pedido: $order));

    session()->flash(key: 'success', value: '¡Pedido confirmado con éxito!');

    return redirect()->route(route: 'user.index');
}

```

- Método `cancelOrder()`: Redirige al usuario de vuelta al carrito si decide no completar la compra.

```

public function cancelOrder(): mixed|RedirectResponse
{
    return redirect()->route(route: 'user.cart');
}

```

- `render()`: Devuelve la vista `order-confirmation`.

```

public function render(): View
{
    return view(view: 'livewire.order-confirmation');
}

```

VISTA:

- Mostrar los productos que hay en el carrito: Se recorre con un foreach los elementos que hay en un carrito, para poder mostrarlos juntos con sus datos individualmente en tarjetas.

```
@foreach ($cartItems as $item)
<div class="col-12 mb-3">
  <div class="card p-3" role="group" aria-label="Producto {{ $item->product->name }}">
```

- Dentro de cada tarjeta se muestra:
 - Imagen del producto (primera imagen asociada).

```
<div class="col-md-4 col-12 my-3">
  product->name }} - {{ $item->product->category }}, para {{ $item->product->gender }}"
    aria-label="Imagen del producto {{ $item->product->name }}"
    title="Imagen del producto {{ $item->product->name }}"
    class="img-fluid rounded shadow-sm d-block mx-auto" style="max-height: 250px;"
  </div>
```

- Nombre, descripción, color, estilo, categoría y género.

```
<div class="col-md-8 col-12">
  <h3 class="h5" title="Nombre del producto">{{ $item->product->name }}</h3>
  <p class="text-muted" title="Descripción del producto">
    {{ $item->product->description }}</p>
```

```
<div class="col-6 col-md-4" role="list"
  aria-label="Características básicas del producto">
  <p role="listitem"><strong>Género:</strong>
    <span>{{ $item->product->gender }}</span>
  </p>
  <p role="listitem"><strong>Estilo:</strong>
    <span>{{ $item->product->style }}</span>
  </p>
  <p role="listitem"><strong>Categoría:</strong>
    <span>{{ $item->product->category }}</span>
  </p>
  <p role="listitem"><strong>Color:</strong> <span>{{ $item->product->color }}</span>
  </p>
</div>
```

- Talla seleccionada y cantidad.

```
<div class="col-6 col-md-4" role="list"
  aria-label="Talla, cantidad y precios del producto">
  <p role="listitem">
    <strong>Talla:</strong>
    <span title="Talla seleccionada">
      {{ $item->product->sizes->firstWhere('id', $item->size_id)->name }}
    </span>
  </p>
  <p role="listitem">
    <strong>Cantidad:</strong>
    <span title="Cantidad seleccionada">{{ $item->quantity }}</span>
  </p>
```

- Precio individual con descuento si aplica (tachado el precio original).

```
<p role="listitem">
  <strong>Precio Unidad:</strong>
  @if ($item->product->discount && $item->product->discount > 0)
    <span class="text-decoration-line-through text-secondary me-2"
      title="Precio original">
      {{ number_format(num: $item->product->price, decimals: 2) }} €
    </span>
    <span class="text-danger fw-bold" title="Precio con descuento">
      {{ number_format(num: $item->product->price - ($item->product->price * $item->product->discount) / 100, decimals: 2) }} €
    </span>
  @else
    {{ number_format(num: $item->product->price, decimals: 2) }} €
  @endif
</p>
```

- Total individual por producto.

```
<p role="listitem">
  <strong>Total:</strong>
  @if ($item->product->discount && $item->product->discount > 0)
    {{ number_format(num: ($item->product->price - ($item->product->price * $item->product->discount) / 100) * $item->quantity, decimals: 2) }}
    €
  @else
    {{ number_format(num: $item->product->price * $item->quantity, decimals: 2) }} €
  @endif
</p>
```

- Total general del pedido: Calculado y mostrado al final en un <p> destacado.

```
<!-- Total general -->
<div class="text-center my-4" role="contentinfo" aria-label="Total general a pagar">
  <p class="fw-bold fs-4">Total a pagar: {{ number_format(num: $totalPrice, decimals: 2) }} €</p>
</div>
```

- Método de pago: Es un desplegable con tres opciones: Tarjeta de Crédito, PayPal y Transferencia Bancaria. No se almacena ni se usa funcionalmente (por ahora es solo decorativo), aunque se podría implementar en un futuro.

```
<div class="col-md-6 col-12 mb-3 d-flex align-items-center">
  <label for="paymentMethod" class="w-25 fw-bold" id="paymentMethodLabel">Método de pago:</label>
  <select class="form-control" id="paymentMethod" aria-labelledby="paymentMethodLabel"
    title="Selecciona método de pago">
    <option value="creditCard">Tarjeta de Crédito</option>
    <option value="paypal">PayPal</option>
    <option value="bankTransfer">Transferencia Bancaria</option>
  </select>
</div>
```

- Botones:
 - Confirmar y pagar: ejecuta confirmOrder()
 - Cancelar: ejecuta cancelOrder()

```
<button class="btn btn-success" wire:click="confirmOrder" aria-label="Confirmar y pagar pedido"
  title="Confirmar y pagar pedido">
  <i class="bi bi-credit-card" aria-hidden="true"></i> Confirmar y pagar
</button>
<button class="btn btn-danger" wire:click="cancelOrder" aria-label="Cancelar pedido"
  title="Cancelar pedido">
  <i class="bi bi-x-square" aria-hidden="true"></i> Cancelar
</button>
```

5.5.5 – ORDERCRUD.PHP

FUNCIONALIDAD: Permitir la gestión de pedidos en el panel administrativo del ecommerce: permite visualizar, filtrar, actualizar, eliminar y reembolsar pedidos, así como eliminar productos de un pedido.

- Variables:

- **orders**: Colección de pedidos cargados.
- **totalreturn**: Array para llevar la cuenta de totales devueltos por pedido.
- **message**: Mensaje que se mostrará al usuario-
- **messageType**: Tipo de mensaje (success, warning, error).
- **showMessage**: Controla si se muestra el mensaje o no.
- **statusFilter**: Filtro para el estado del pedido.
- **userName**: Filtro para buscar pedidos por nombre de usuario.
- **startDate**: Fecha inicial para filtrar pedidos por rango de fechas.
- **endDate**: Fecha final para filtrar pedidos por rango de fechas.

```
public $orders;
5 references
public $totalreturn = [];

1 reference
public $message = null;
1 reference
public $messageType = 'success';
1 reference
public $showMessage = false;

3 references
public $statusFilter = '';
3 references
public $userName = '';
3 references
public $startDate = '';
3 references
public $endDate = '';
```

- **mount()**: Carga los pedidos al inicializar el componente. Muestra un mensaje si no hay pedidos.

```
public function mount(): void
{
    $this->loadOrders();

    if ($this->orders->isEmpty()) {
        $this->setMessage(message: 'No hay ningún Pedido', messageType: 'warning');
    }
}
```

- **loadOrders()**: Consulta todos los pedidos con sus relaciones necesarias.

```
public function loadOrders(): void
{
    $this->orders = Order::with(relations: ['user', 'items.product', 'items.size'])->get();
}
```

- **filter()**: Aplica filtros a la consulta de pedidos por estado (pendiente, pagado, enviado, cancelado, reembolsado), usuario o por rango de fechas. Si no se encuentran resultados con los filtros aplicados, se muestra un mensaje tipo flash.

```

public function filter(): void
{
    $query = Order::with(relations: ['user', 'items.product', 'items.size']);

    if (!empty($this->statusFilter)) {
        $query->where(column: 'status', operator: $this->statusFilter);
    }

    if (!empty($this->userName)) {
        $query->whereHas(relation: 'user', callback: function (Builder<TRelatedModel> $q)... {
            // Busca usuarios cuyo nombre contenga el texto introducido (búsqueda parcial)
            $q->where(column: 'name', operator: 'like', value: '%' . $this->userName . '%');
        });
    }

    if (!empty($this->startDate) && !empty($this->endDate)) {
        // Filtro por rango de fechas creado_at, incluyendo ambos extremos
        $query->whereBetween(column: 'created_at', values: [$this->startDate, $this->endDate]);
    }

    // Ejecuta la consulta con todos los filtros aplicados
    $this->orders = $query->get();

    // Si no se encontraron pedidos con los filtros, muestra un mensaje de error temporal
    if ($this->orders->isEmpty()) {
        session()->flash(key: 'error', value: 'No se encontraron pedidos con los filtros aplicados');
    }
}

```

- resetFilters(): Limpia los filtros y recarga todos los pedidos.

```

public function resetFilters()
{
    $this->statusFilter = '';
    $this->userName = '';
    $this->startDate = '';
    $this->endDate = '';
    $this->loadOrders();
}

```

- updateStatus(): Actualiza el estado de un pedido específico.

```

public function updateStatus($orderId, $status): void
{
    $order = Order::find(id: $orderId);
    if ($order) {
        $order->status = $status;
        $order->save();
    }
}

```

- removeItem(): Su función principal es eliminar un producto de un pedido, actualizar el total de la compra, y si el estado del pedido es pagado o reembolsado devuelve el stock. Además si tras la eliminación no quedan productos ni importe pendiente de devolución, se borra el pedido por completo.


```

public function removeItem($ItemId): void
{
    $item = OrderItem::find(id: $ItemId);

    if ($item) {
        $order = $item->order;

        // Si el pedido está pagado o reembolsado, devuelve el stock al inventario
        if ($order->status === 'pagado' || $order->status === 'reembolsado') {
            \DB::table('product_size')
                ->where(column: 'product_id', operator: $item->product_id)
                ->where(column: 'size_id', operator: $item->size_id)
                ->increment(column: 'stock', amount: $item->quantity); // Suma la cantidad devuelta al stock

            // Inicializa el total de devolución para ese pedido si no existe
            if (!isset($this->totalreturn[$order->id])) {
                $this->totalreturn[$order->id] = 0;
            }

            // Acumula el importe que se debe devolver para ese pedido
            $this->totalreturn[$order->id] += $item->price * $item->quantity;
        }

        // Elimina el ítem del pedido
        $item->delete();

        // Recalcula y actualiza el total del pedido basado en los ítems restantes
        $order->total = $order->items()->sum(\DB::raw('price * quantity'));
        $order->save();

        // Si no quedan ítems en el pedido y no hay devolución pendiente, elimina el pedido
        if ($order->items()->count() === 0 && (!isset($this->totalreturn[$order->id]) || $this->totalreturn[$order->id] == 0)) {
            $order->delete();
            $this->setMessage(message: 'Pedido #' . $order->id . ' eliminado porque no tiene productos.', messageType: 'warning');
        }
    }

    // Refresca la lista de pedidos para reflejar los cambios
    $this->loadOrders();
}

```

- `returned()`: Marca el pedido como reembolsado, y elimina el pedido si no quedan productos. Además se muestra un mensaje con el resultado de la operación.

```

public function returned($orderId): void
{
    $order = Order::find(id: $orderId);

    if ($order) {
        $order->status = 'reembolsado';
        $order->save();

        // Si no quedan productos, elimina el pedido
        if ($order->items()->count() === 0) {
            $order->delete();
            $this->setMessage(message: 'Pedido #' . $order->id . ' eliminado porque no tiene productos.', messageType: 'warning');
        } else {
            // Si aún tiene productos, solo notifica que se procesó la devolución
            $this->setMessage(message: 'Se ha procesado la devolución del dinero del Pedido #' . $order->id . '.', messageType: 'success');
        }
    }

    // Vuelve a cargar el componente para actualizar la vista
    $this->mount();
}

```


- `setMessage()`:
Configura un mensaje a mostrar en la vista.

```
public function setMessage($message, $messageType = 'success'): void
{
    $this->message = $message;
    $this->messageType = $messageType;
    $this->showMessage = true;
}
```

- `render()`: Devuelve la vista asociada al componente.

```
public function render(): View
{
    return view(view: 'livewire.order-crud');
}
```

VISTA:

- Formulario para filtrar:
 - Campos de búsqueda: Permite filtrar los pedidos por estado del pedido, nombre del usuario (parcial o completo), y fecha de creación. Usa `wire:model.defer` para evitar actualizaciones en tiempo real y solo enviar los datos al hacer submit.

```
<form wire:submit.prevent="filter" aria-label="Formulario de filtro de pedidos">
  <div class="row g-3 align-items-end">
    <div class="col-md-3">
      <label for="statusFilter" class="form-label">Estado</label>
      <select id="statusFilter" name="statusFilter" class="form-select" wire:model.defer="statusFilter" title="Filtrar por estado" aria-label="Filtrar por estado">
        <option value="">-- Todos --</option>
        <option value="pendiente">Pendiente</option>
        <option value="pagado">Pagado</option>
        <option value="enviado">Enviado</option>
        <option value="cancelado">Cancelado</option>
        <option value="reembolsado">Reembolsado</option>
      </select>
    </div>
    <div class="col-md-3">
      <label for="userName" class="form-label">Nombre de Usuario</label>
      <input type="text" id="userName" name="userName" class="form-control" wire:model.defer="userName"
        placeholder="Ej: Juan Pérez" title="Filtrar por nombre de usuario" aria-label="Nombre de Usuario">
    </div>
    <div class="col-md-3">
      <label for="startDate" class="form-label">Desde</label>
      <input type="date" id="startDate" name="startDate" class="form-control" wire:model.defer="startDate"
        title="Fecha de inicio" aria-label="Fecha de inicio">
    </div>
    <div class="col-md-3">
      <label for="endDate" class="form-label">Hasta</label>
      <input type="date" id="endDate" name="endDate" class="form-control" wire:model.defer="endDate"
        min="{{ $startDate }}" title="Fecha final" aria-label="Fecha final">
    </div>
  </div>
</form>
```

- Botones: Tiene 2 botones uno para filtrar, que ejecuta el método `filter()` del componente. Y otro para borrar filtros, que llama a `resetFilters()` para limpiar todos los campos y volver a mostrar todos los pedidos.

```
<div class="col-12 text-end mt-3">
  <button type="submit" class="btn btn-primary me-2" aria-label="Filtrar pedidos">
    <i class="bi bi-search"></i> Filtrar
  </button>
  <button type="button" class="btn btn-secondary" wire:click="resetFilters" aria-label="Borrar filtros">
    <i class="bi bi-x-circle"></i> Borrar Filtros
  </button>
</div>
</form>
```

- Mensajes de estado: Si en el componente se establece un mensaje con `setMessage()`, se muestra como alerta Bootstrap con la clase correspondiente (success, warning, etc.).

```
@if ($showMessage)
<div class="alert alert-{{ $messageType === 'success' ? 'success' : ($messageType === 'warning' ? 'warning' : 'danger') }}" mb-4" role="alert">
    {{ $message }}
</div>
@endif
```

- Listado de Pedidos: Cada pedido se muestra como una tarjeta/tabla y cada tarjeta contiene:
 - Cabecera con el id del pedido, el nombre del usuario que ha hecho el pedido, precio total del pedido, su estado.

```
@foreach ($orders as $order)
<div class="card mb-4 shadow-sm">
    <div class="card-header d-flex flex-wrap justify-content-between align-items-center bg-light">
        <div>
            <strong>Pedido #{{ $order->id }}</strong> - {{ $order->user->name }}
            <span class="ms-3">Total: <strong>€{{ number_format(num: $order->total, decimals: 2) }}</strong></span>
        </div>
        <div>
            <label for="status-{{ $order->id }}" class="me-2 fw-bold">Estado:</label>
            <select id="status-{{ $order->id }}" name="status-{{ $order->id }}" class="form-select form-select-sm d-inline-block w-auto"
                wire:change="updateStatus({{ $order->id }}, $event.target.value)"
                title="Cambiar estado del pedido #{{ $order->id }}" aria-label="Estado del pedido {{ $order->id }}">
                <option value="pendiente" {{ $order->status === 'pendiente' ? 'selected' : '' }}>Pendiente</option>
                <option value="pagado" {{ $order->status === 'pagado' ? 'selected' : '' }}>Pagado</option>
                <option value="enviado" {{ $order->status === 'enviado' ? 'selected' : '' }}>Enviado</option>
                <option value="cancelado" {{ $order->status === 'cancelado' ? 'selected' : '' }}>Cancelado</option>
                <option value="reembolsado" {{ $order->status === 'reembolsado' ? 'selected' : '' }}>Reembolsado</option>
            </select>
        </div>
    </div>
</div>
```

- Cuerpo donde se muestran los productos que contiene el pedido, junto con la talla seleccionada, la cantidad y su precio. Además cada producto de un pedido tiene un botón de eliminarlo del pedido.

```
<div class="card-body">
    @foreach ($order->items as $item)
        <div class="row align-items-center mb-3 pb-3 border-bottom">
            <div class="col-md-3">
                <h6 class="mb-0" title="Nombre del producto">{{ $item->product->name }}</h6>
            </div>
            <div class="col-md-2">
                <span class="text-muted">Talla:</span> {{ $item->size->name }}
            </div>
            <div class="col-md-2">
                <span class="text-muted">Cantidad:</span> {{ $item->quantity }}
            </div>
            <div class="col-md-2">
                <span class="text-muted">Precio:</span> €{{ number_format(num: $item->price, decimals: 2) }}
            </div>
            <div class="col-md-3 text-end">
                <button class="btn btn-outline-danger btn-sm" wire:click="removeItem({{ $item->id }})"
                    title="Eliminar producto del pedido" aria-label="Eliminar producto {{ $item->product->name }}">
                    <i class="bi bi-trash3"></i> Eliminar Producto
                </button>
            </div>
        </div>
    @endforeach
</div>
```

- Además si el estado del pedido es pagado o reembolsado, al eliminar un producto del pedido (como si alguien realizara una devolución) se muestra en la vista un botón para devolver el dinero del producto eliminado, y el dinero a devolver.

```
@if (isset($totalreturn[$order->id]))
<div class="mt-2 d-flex justify-content-between">
  <span><strong>Total a devolver:</strong> €{{ number_format(num: $totalreturn[$order->id], decimals: 2) }}</span>
  <button class="btn btn-warning btn-sm" wire:click="returned('{{ $order->id }}'"
    title="Devolver dinero del pedido" aria-label="Devolver dinero del pedido {{ $order->id }}">
    <i class="bi bi-coin"></i> Devolver dinero
  </button>
</div>
@endif
```

5.5.6 – PRODUCTCRUD.PHP

FUNCIONALIDAD: Proporciona un CRUD completo para la gestión de productos en el panel de administración de un ecommerce. Permite crear, editar, listar y eliminar productos, así como gestionar tallas, stocks y múltiples imágenes por producto.

- Variables:
 - **products:** colección de productos cargados desde la base de datos.
 - **product_id, name, description, color, gender, style, category, price, discount:** campos principales del formulario.
 - **images:** imágenes nuevas subidas desde el formulario.
 - **imagesDB:** imágenes ya guardadas en la base de datos asociadas al producto.
 - **newCategory, newSize:** inputs para crear nuevas categorías y tallas.
 - **categories:** array con todas las categorías únicas disponibles.
 - **sizes:** array asociativo [size_id => stock] para el stock por talla.
 - **availableSizes:** tallas filtradas por tipo de producto (ropa/calzado).
 - **allSizes:** todas las tallas disponibles en la tabla sizes.
 - **view:** controla la vista actual (list o form).

```
public $products, $product_id, $name,
  $description, $color, $gender, $style,
  $category, $price, $discount;

10 references
public $images = [];
4 references
public $imagesDB = [];

7 references
public $newCategory = '';
7 references
public $newSize = '';
4 references
public $categories = [];
7 references
public $sizes = [];
6 references
public $availableSizes;
3 references
public $allSizes;

5 references
public $view = 'list';
```

- `mount()`: Inicializa productos, tallas y categorías desde la base de datos. También aplica el filtrado inicial de tallas.

```
public function mount(): void
{
    $this->products = Product::all(); // Carga todos los productos
    $this->allSizes = Size::all(); // Carga todas las tallas posibles

    $this->filterSizes(); // Filtra tallas según categoría actual (inicial)

    // Obtiene todas las categorías únicas existentes en productos para el filtro o selector
    $this->categories = Product::distinct()->pluck(column: 'category')->toArray();

    $this->availableSizes = Size::all();
}
```

- `filter()`: Permite buscar productos por ID o nombre. Si no se encuentra coincidencia, muestra todos los productos con un mensaje de error.

```
public function filter(): void
{
    if (empty($this->product_id) && empty($this->name)) {
        session()->flash(key: 'error', value: 'Introduce algún dato de búsqueda');
        $this->products = Product::all();
        return;
    }

    $query = Product::query();

    // Filtra por ID exacto si se proporciona
    if (!empty($this->product_id)) {
        $query->where(column: 'id', operator: $this->product_id);
    }

    // Filtra por nombre con búsqueda parcial usando LIKE
    if (!empty($this->name)) {
        $query->where(column: 'name', operator: 'like', value: '%' . $this->name . '%');
    }

    $this->products = $query->get();

    // Si no hay resultados, muestra mensaje y recarga todos los productos
    if ($this->products->isEmpty()) {
        session()->flash(key: 'error', value: 'Producto no encontrado');
        $this->products = Product::all();
    }
}
```

- `resetFilters()`: Limpia los filtros aplicados y recarga todos los productos.

```
public function resetFilters(): void
{
    $this->name = '';
    $this->product_id = '';
    $this->products = Product::all();
}
```

- `resetInputs()`: Limpia todos los campos del formulario, incluyendo imágenes y tallas.

```
public function resetInputs(): void
{
    $this->product_id = $this->name = $this->description = $this->color = $this->gender = $this->style = $this->category = $this->newSize = $this->newCategory = '';
    $this->price = null;
    $this->sizes = [];
    $this->images = []; // Limpia imágenes cargadas nuevas
    $this->imagesDB = []; // Limpia imágenes guardadas en BD
}
```

- `showCreateForm()`: Activa la vista de formulario vacío para crear un nuevo producto.

```
public function showCreateForm(): void
{
    $this->resetInputs();
    $this->view = 'form';
}
```

- `filterSizes()`: Filtra tallas disponibles según la categoría (calzado = numéricas, ropa = XS-XL).

```
public function filterSizes(): void
{
    if (strtolower(string: $this->category) === 'calzado') {
        // Para calzado, filtra solo tallas numéricas
        $this->availableSizes = $this->allSizes->filter(function ($size): bool {
            return is_numeric(value: $size->name);
        });
    } else {
        // Para ropa, filtra tallas estándar XS, S, M, L, XL
        $this->availableSizes = $this->allSizes->filter(function ($size): bool {
            return in_array(needle: strtolower(string: $size->name), haystack: ['xs', 's', 'm', 'l', 'xl']);
        });
    }
}
```

- `addNewCategory()`: Permite agregar una nueva categoría al listado si no existe. Se limpia el campo después de guardarla.

```
public function addNewCategory(): void
{
    // Limpia espacios y convierte el texto a formato "Título" (primera letra mayúscula)
    $name = trim(string: mb_convert_case(string: $this->newCategory, mode: MB_CASE_TITLE));

    // Si el nombre no está vacío y no existe ya en el array de categorías
    if ($name && !in_array(needle: $name, haystack: $this->categories)) {
        // Añade la nueva categoría al listado de categorías
        $this->categories[] = $name;

        // Asigna la nueva categoría seleccionada para usar en el formulario
        $this->category = $name;

        // Muestra un mensaje flash para informar al usuario que la categoría fue añadida
        session()->flash(key: 'message', value: 'Nueva categoría añadida.');
```

```
    }

    // Limpia el campo input de nueva categoría para que quede vacío
    $this->newCategory = '';
}
```

- `addNewSize()`: Permite crear una talla si no existe en la base de datos. Se añade automáticamente al array `sizes` con stock 0.

```
public function addNewSize(): void
{
    // Limpia espacios y convierte el texto a MAYÚSCULAS para uniformidad
    $name = trim(string: mb_strtoupper(string: $this->newSize));

    // Si el nombre no está vacío
    if ($name) {
        // Busca o crea una nueva talla en la tabla sizes con ese nombre
        $size = Size::firstOrCreate(attributes: ['name' => $name]);

        // Comprueba que la talla no esté ya en el array de tallas disponibles
        if (!collect(value: $this->availableSizes)->contains(key: 'id', operator: $size->id)) {
            // Si no está, la añade a la lista de tallas disponibles
            $this->availableSizes[] = $size;
        }

        // Inicializa el stock de esta talla nueva a 0 en el array sizes (para el formulario)
        $this->sizes[$size->id] = 0;
    }

    // Limpia el campo input de nueva talla para que quede vacío
    $this->newSize = '';
}
```

- `removePreviewImage($key)`: Elimina una imagen cargada en tiempo real del array `images`, antes de ser subida.

```
public function removePreviewImage($key): void
{
    if (isset($this->images[$key]) && is_object(value: $this->images[$key])) {
        unset($this->images[$key]);
        // Reindexar el array para evitar huecos y mantenerlo limpio
        $this->images = array_values(array: $this->images);
    }
}
```

- `deleteImage($id)`: Elimina una imagen existente tanto de la base de datos como del disco, y actualiza `imagesDB`.

```
public function deleteImage($id): void
{
    $image = Image::find(id: $id);

    if ($image) {
        Storage::disk(name: 'public')->delete(paths: $image->url);
        $image->delete();

        $this->imagesDB = Image::where(column: 'product_id', operator: $this->product_id)->get();
    }
}
```

- `saveImages($productId)`: Este es un método privado que se llama en store y en update. Lo que hace es validar y guardar todas las imágenes nuevas asociadas al producto, dentro de `storage/app/public/products`. Además crea un registro en la tabla `images` para cada imagen.

```
private function saveImages($productId): void
{
    $validImages = collect(value: $this->images)
        ->filter(callback: fn(TValue $img): bool => $img instanceof \Illuminate\Http\UploadedFile)
        ->values();

    if ($validImages->isEmpty()) {
        return;
    }

    $this->validate(rules: [
        'images.*' => 'image|max:2048',
    ]);

    foreach ($this->images as $image) {
        $path = $image->store('products', 'public');
        Image::create(attributes: [
            'url' => $path,
            'product_id' => $productId,
        ]);
    }

    $this->images = [];
    $this->dispatch(event: 'clearFileInput');
}
```

- `edit($id)`: Carga los datos del producto seleccionado, incluyendo imágenes y tallas con su stock, para edición. Cambia la vista a form.

```
public function edit($id): void
{
    $product = Product::with(relations: 'images')->findOrFail(id: $id);
    $this->product_id = $product->id;
    $this->fill(values: $product->toArray());

    $this->imagesDB = $product->images;

    $this->sizes = $product->sizes->pluck('pivot.stock', 'id')->toArray();

    $this->availableSizes = Size::all();
    $this->categories = Product::distinct()->pluck(column: 'category')->toArray();
    $this->view = 'form';
}
```


- `store()`: Valida y guarda un nuevo producto. Si se han ingresado nuevas categorías o tallas, también se guardan. Asocia las tallas con sus respectivos stocks mediante la tabla pivote `product_size`. Guarda imágenes y reinicia el formulario tras guardar.

```

public function store(): void
{
    $this->validate(rules: [
        'name' => 'required',
        'price' => 'required|numeric',
        'discount' => 'nullable|numeric|min:0', // nullable para permitir vacío
        'sizes' => [
            'required',
            function ($attribute, $value, $fail): void {
                $hasStock = false;
                foreach ($value as $stock) {
                    if ($stock > 0) {
                        $hasStock = true;
                        break;
                    }
                }
                if (!$hasStock) {
                    $fail('Debes asignar stock a al menos una talla.');

```


- `update()`: Valida y actualiza un producto existente. Si se añaden nuevas categorías o tallas, también se guardan. Sincroniza las tallas con su stock mediante la tabla pivote `product_size`. Guarda nuevas imágenes si las hay, reinicia el formulario y vuelve a la vista del listado.

```
public function update(): void
{
    $this->validate(rules: [
        'name' => 'required',
        'price' => 'required|numeric',
        'discount' => 'nullable|numeric|min:0', // nullable para permitir vacío
        'sizes' => [
            'required',
            function ($attribute, $value, $fail): void {
                $hasStock = false;
                foreach ($value as $stock) {
                    if ($stock > 0) {
                        $hasStock = true;
                        break;
                    }
                }
                if (!$hasStock) {
                    $fail('Malamente');
                }
            }
        ]
    ],
    );

    if (!empty($this->newCategory)) {
        $this->category = trim(string: $this->newCategory);
    }

    if (!empty($this->newSize)) {
        $size = Size::firstOrCreate(attributes: ['name' => trim(string: $this->newSize)]);
        $this->sizes[$size->id] = 0;
    }

    $product = Product::findOrFail(id: $this->product_id);
    $product->update($this->validateData());

    $syncData = [];
    foreach ($this->sizes as $sizeId => $stock) {
        if ($stock > 0) {
            $syncData[$sizeId] = ['stock' => $stock];
        }
    }

    $product->sizes()->sync($syncData);

    // Guardar imágenes asociadas SOLO si hay imágenes nuevas
    if (!empty($this->images)) {
        $this->saveImages(productId: $product->id);
    }

    $this->resetInputs();
    session()->flash(key: 'message', value: 'Producto actualizado.');
```

- `cancel()`: Cancela la creación o edición de un producto. Limpia todos los campos del formulario y vuelve a la vista del listado.

```
public function cancel(): void
{
    $this->resetInputs();
    $this->view = 'list';
}
```

- `delete($id)`: Elimina un producto existente de la base de datos según su ID. Muestra un mensaje de confirmación tras la eliminación.

```
public function delete($id): void
{
    Product::findOrFail(id: $id)->delete();
    session()->flash(key: 'message', value: 'Producto eliminado.');
```

- `validateData()`: Devuelve los datos del formulario ya validados y formateados (como nombre, color, estilo y género en formato título) para ser usados en la creación o actualización del producto.

```
private function validateData(): array
{
    return [
        'name' => trim(string: mb_convert_case(string: $this->name, mode: MB_CASE_TITLE)),
        'description' => $this->description,
        'color' => trim(string: mb_convert_case(string: $this->color, mode: MB_CASE_TITLE)),
        'gender' => trim(string: mb_convert_case(string: $this->gender, mode: MB_CASE_TITLE)),
        'style' => trim(string: mb_convert_case(string: $this->style, mode: MB_CASE_TITLE)),
        'category' => $this->category,
        'price' => $this->price,
        'discount' => $this->discount,
    ];
}
```

- `render()`: Devuelve la vista Livewire product-crud.

```
public function render(): View
{
    return view(view: 'livewire.product-crud');
```

VISTA: Se divide en dos secciones principales, controladas por la propiedad `$view`: `list` que es la vista de listado de productos con filtrado y acciones (editar, eliminar) y `form` que es la vista de formulario para crear o actualizar productos, incluyendo la gestión de imágenes

Listado de Productos (`$view === 'list'`):

- Filtro de Productos: Filtra por nombre o ID usando inputs con `wire:model="name"` y `wire:model="product_id"`. Tiene 2 botones uno para aplicar los filtros y otro para borrarlos.

```
<form wire:submit.prevent="filter" class="row g-3 align-items-end" aria-describedby="filtro-desc">
  <div id="filtro-desc" class="visually-hidden">
    Formulario para filtrar productos por nombre o ID.
  </div>
```

```

<div class="col-md">
  <label for="filter-name" class="form-label">Nombre</label>
  <input type="text" wire:model="name" id="filter-name" placeholder="Buscar por nombre"
    class="form-control" aria-describedby="filter-name-desc">
  <small id="filter-name-desc" class="visually-hidden">Escribe el nombre para buscar</small>
</div>

<div class="col-md">
  <label for="filter-id" class="form-label">ID</label>
  <input type="number" wire:model="product_id" id="filter-id" placeholder="Buscar por ID"
    class="form-control" aria-describedby="filter-id-desc">
  <small id="filter-id-desc" class="visually-hidden">Escribe el ID del producto para
    buscar</small>
</div>

<div class="col-auto">
  <button type="submit" class="btn btn-primary" aria-label="Filtrar productos">
    <i class="bi bi-search" title="Buscar"></i> Filtrar
  </button>
</div>

<div class="col-auto">
  <button type="button" wire:click="resetFilters" class="btn btn-secondary"
    aria-label="Borrar filtros">
    <i class="bi bi-x-circle" title="Borrar filtros"></i> Borrar Filtros
  </button>
</div>
</form>

```

- Mensajes Flash: Muestra mensajes de éxito o error después de acciones (crear, editar, eliminar).

```

@if (session()->has(key: 'message'))
  <div class="alert alert-success mb-4" role="alert">
    {{ session(key: 'message') }}
  </div>
@endif
@if (session()->has(key: 'error'))
  <div class="alert alert-danger mb-4" role="alert">
    {{ session(key: 'error') }}
  </div>
@endif

```

- Botón para crear un producto nuevo, y supuesta paginación: La paginación no está implementada por falta de tiempo, pero se podría implementar a futuro para mejorar la vista de los productos.

```

<button wire:click="showcreateForm" class="btn btn-success" aria-label="Crear nuevo producto">
  Crear nuevo producto
</button>

```

```

<!-- NO funciona (No está implementado es de decoración 😊) -->
<div>
  <label for="show-count" class="visually-hidden">Número de productos a mostrar</label>
  <select id="show-count" class="form-select form-select-sm"
    aria-label="Seleccionar número de productos a mostrar">
    <option value="5">Mostrar 5 primeros productos</option>
    <option value="10">Mostrar 10 primeros productos</option>
    <option value="20">Mostrar 20 primeros productos</option>
    <option value="50">Mostrar 50 primeros productos</option>
  </select>
</div>

```

- Tabla de Productos: Muestra todos los productos con sus atributos y dos botones: editar y eliminar. Al pulsar eliminar, aparece un mensaje de confirmación ya que la acción no se puede deshacer. Si no hay productos, se muestra el mensaje "No hay productos".

```

<table class="table table-bordered table-striped w-100 align-items-center" role="table"
  aria-describedby="productos-heading">
  <thead class="thead-light">
    <tr>
      <th scope="col">ID</th>
      <th scope="col">Nombre</th>
      <th scope="col">Descripción</th>
      <th scope="col">Características</th>
      <th scope="col">Tallas</th>
      <th scope="col">Stock</th>
      <th scope="col">Rebaja</th>
      <th scope="col">Precio</th>
      <th scope="col">Acciones</th>
    </tr>
  </thead>
  <tbody>
    @forelse ($products as $product)
      <tr>
        <td>{{ $product->id }}</td>
        <td>{{ $product->name }}</td>
        <td>{{ $product->description }}</td>
        <td>
          <p>Categoría: {{ $product->category }}</p>
          <p>Color: {{ $product->color }}</p>
          <p>Género: {{ $product->gender }}</p>
          <p>Estilo: {{ $product->style }}</p>
        </td>
        <td>
          @foreach ($product->sizes as $size)
            <div>{{ $size->name }}</div>
          @endforeach
        </td>
        <td>
          @foreach ($product->sizes as $size)
            <div>{{ $size->pivot->stock }}</div>
          @endforeach
        </td>
        <td>
          @if ($product->discount <= 0)
            <td>No tiene rebaja</td>
          @else
            <td>{{ $product->discount }}%</td>
          @endif
        <td>
          @if ($product->discount > 0)
            <span style="text-decoration: line-through; "
              aria-label="Precio original">
              {{ number_format(num: $product->price, decimals: 2) }}€
            </span>
            <span class="fw-bold text-danger" aria-label="Precio con descuento">
              {{ number_format(num: $product->price - ($product->price * $product->discount) / 100, decimals: 2) }}€
            </span>
          @else
            {{ number_format(num: $product->price, decimals: 2) }}€
          @endif
        </td>
        <td>
          <button wire:click="edit('{{ $product->id }}')" class="btn btn-warning btn-sm mt-2"
            aria-label="Editar producto {{ $product->name }}" title="Editar producto">
            <i class="bi bi-pencil" aria-hidden="true"></i>
          </button>
          <button wire:click="delete('{{ $product->id }}')" class="btn btn-danger btn-sm mt-2"
            onclick="confirm('¿Estás seguro de eliminar este producto?') || event.stopImmediatePropagation()"
            aria-label="Eliminar producto {{ $product->name }}" title="Eliminar producto">
            <i class="bi bi-trash" aria-hidden="true"></i>
          </button>
        </td>
      </tr>
    @empty
      <tr>
        <td colspan="9" class="text-center">No hay Productos</td>
      </tr>
    @endforelse
  </tbody>
</table>

```

Formulario de Creación/Edición (\$view !== 'list')

- Encabezado dinámico: Dependiendo de qué botón se pulse, el de editar o crear, el encabezado tendrá escrito Actualizar o Crear.

```
<h2 class="mb-5 my-4 text-center" role="heading" aria-level="2">
  {{ $product_id ? 'Actualizar' : 'Crear' }} Productos
</h2>
```

- Gestión de Imágenes: Las imágenes guardadas en la base de datos se muestran con un botón para ver en grande (usa un modal), y otro para eliminarlas.

```
<label for="images" class="form-label">Imágenes</label>
{{-- Imágenes guardadas en BD --}}
@if (count(value: $imagesDB) > 0)
  <div class="col-md-6 d-flex flex-wrap gap-3 mb-3" role="list"
    aria-label="Imágenes guardadas">
    @foreach ($imagesDB as $index => $image)
      <div style="position: relative; width: 150px; height: 150px;" role="listitem">
        

        <button type="button"
          class="btn btn-sm btn-dark position-absolute top-0 end-0 m-1"
          style="background-color: rgba(0, 0, 0, 0.5); border: none;"
          data-bs-toggle="modal" data-bs-target="#imageModal{{ $index }}"
          title="Ver imagen en tamaño grande"
          aria-label="Ver imagen {{ $index + 1 }} en tamaño grande">
          <i class="bi bi-fullscreen text-light"></i>
        </button>

        <button type="button"
          class="btn btn-sm btn-danger position-absolute bottom-0 end-0 m-1"
          style="background-color: rgba(0, 0, 0, 0.5); border: none;"
          wire:click="deleteImage({{ $image->id }})" title="Eliminar imagen"
          aria-label="Eliminar imagen {{ $index + 1 }}">
          <i class="bi bi-trash text-light"></i>
        </button>
      </div>
    @endforeach

    {{-- Modal de imagen grande --}}
    <div class="modal fade" id="imageModal{{ $index }}" tabindex="-1"
      aria-labelledby="imageModalLabel{{ $index }}" aria-hidden="true">
      <div class="modal-dialog modal-dialog-centered modal-lg">
        <div class="modal-content">
          <div class="modal-body p-0">
            
          </div>
        </div>
      </div>
    </div>
  @endif
```

- Input de nuevas imágenes: Se pueden subir más de una imagen siempre y cuando al elegir las imágenes en el explorador se seleccionen más de una. Cuando se crea un nuevo producto el campo es obligatorio y tiene que haber al menos 1 imagen subida, pero al editar deja de ser obligatorio.

```
<input type="file" wire:model="images" id="images" class="form-control form-control-sm"
  multiple {{ !$product_id ? 'required' : '' }} aria-describedby="imagesHelp"
  aria-label="Subir nuevas imágenes del producto">

<small id="imagesHelp" class="form-text ">Puedes subir una o más imágenes {{ !$product_id ? '. Campo obligatorio' : '' }}</small>
```

- Previsualización de imágenes seleccionadas: Usa temporaryUrl() para mostrar previews, y tienen un botón para eliminarlas que llama al método del componente "removePreviewImage({{ \$key }})".

```
@if (count(value: $images) > 0)
  <div class="d-flex flex-wrap gap-3 mt-3" role="list"
    aria-label="Vista previa de nuevas imágenes seleccionadas">
    @foreach ($images as $key => $tempImage)
      @if (is_object(value: $tempImage) && method_exists(object_or_class: $tempImage, method: 'temporaryUrl'))
        <div style="position: relative; width: 150px; height: 150px; role="listitem">
          
          <button type="button"
            class="btn btn-sm btn-danger position-absolute bottom-0 end-0 m-1"
            wire:click="removePreviewImage({{ $key }})"
            title="Quitar imagen"
            aria-label="Quitar imagen previa {{ $key + 1 }}">
            <i class="bi bi-trash text-light"></i>
          </button>
        </div>
      @endif
    @endforeach
  </div>
@endif
```

- Otros Campos del Formulario: Hay campos para añadir todos los atributos de producto:
 - Nombre: Campo obligatorio para el nombre del producto.

```
<div class="col-md-6">
  <label for="name" class="form-label">Nombre</label>
  <input type="text" id="name" wire:model="name" class="form-control form-control-sm"
    placeholder="Nombre" required aria-required="true" aria-describedby="nameHelp">
  <small id="nameHelp" class="form-text ">Nombre del producto. Campo obligatorio</small>
</div>
```

- Descripción: Breve resumen del producto, también obligatorio.

```
<div class="col-md-6">
  <label for="description" class="form-label">Descripción</label>
  <input type="text" id="description" wire:model="description"
    class="form-control form-control-sm" placeholder="Descripción" required aria-required="true"
    aria-describedby="descriptionHelp">
  <small id="descriptionHelp" class="form-text ">Descripción breve del producto. Campo obligatorio</small>
</div>
```

- Color: Campo obligatorio que indica el color principal del producto.

```
<div class="col-md-6">
  <label for="color" class="form-label">Color</label>
  <input type="text" id="color" wire:model="color" class="form-control form-control-sm"
    placeholder="Color" required aria-required="true" aria-describedby="colorHelp">
  <small id="colorHelp" class="form-text">Color principal del producto. Campo obligatorio</small>
</div>
```

- Género: Es un campo obligatorio, donde se indica el género objetivo del producto (por ejemplo, hombre, mujer, unisex).

```
<div class="col-md-6">
  <label for="gender" class="form-label">Género</label>
  <input type="text" id="gender" wire:model="gender" class="form-control form-control-sm"
    placeholder="Género" required aria-required="true" aria-describedby="genderHelp">
  <small id="genderHelp" class="form-text">Género para el que está pensado el
    producto. Campo obligatorio</small>
</div>
```

- Estilo: Campo obligatorio que indica el estilo o tipo de diseño (casual, deportivo, etc.)

```
<div class="col-md-6">
  <label for="style" class="form-label">Estilo</label>
  <input type="text" id="style" wire:model="style" class="form-control form-control-sm"
    placeholder="Estilo" required aria-required="true" aria-describedby="styleHelp">
  <small id="styleHelp" class="form-text">Estilo del producto. Campo obligatorio</small>
</div>
```

- Precio: Campo numérico obligatorio con decimales, que representa el precio del producto.

```
<div class="col-md-6 row mt-3">
  <div class="col-md-6">
    <label for="price" class="form-label">Precio</label>
    <input type="number" id="price" wire:model="price" class="form-control form-control-sm"
      placeholder="Precio" step="0.01" required aria-required="true"
      aria-describedby="priceHelp">
    <small id="priceHelp" class="form-text">Precio en moneda local. Campo obligatorio</small>
  </div>
</div>
```

- Descuento: Campo opcional para indicar un descuento en porcentaje (entre 0 y 90).

```
<div class="col-md-6">
  <label for="discount" class="form-label">Descuento</label>
  <input type="number" id="discount" wire:model="discount"
    class="form-control form-control-sm" placeholder="Descuento" step="5"
    max="90" min="0" aria-describedby="discountHelp">
  <small id="discountHelp" class="form-text">Porcentaje de descuento (0-90%)</small>
</div>
```

- Selección y Creación de Categorías: Para añadir la categoría del producto se utiliza un campo <select> que carga dinámicamente las categorías existentes desde la base de datos.


```

<div class="col-md-6">
  <label for="category" class="form-label">Categoría</label>
  <select id="category" wire:model="category" wire:change="filterSizes"
    class="form-select form-select-sm" required aria-required="true"
    aria-describedby="categoryHelp">
    <option value="">-- Selecciona una categoría --</option>
    @foreach ($categories as $cat)
      <option value="{{ $cat }}">{{ $cat }}</option>
    @endforeach
  </select>
  <small id="categoryHelp" class="form-text">Categoría del producto. Campo obligatorio</small>
</div>

```

Por eso, se proporciona un input adicional para permitir la creación de una nueva categoría directamente desde el formulario, ya sea porque no existe ninguna categoría registrada, o porque se desea añadir una nueva categoría sin abandonar el formulario actual.

```

<div class="col-md-6">
  <label for="newCategory" class="form-label">Otra categoría:</label>
  <div class="input-group input-group-sm">
    <input type="text" id="newCategory" wire:model="newCategory" class="form-control"
      placeholder="Nueva Categoría" aria-describedby="newCategoryHelp">
    <button class="btn btn-outline-dark" type="button" wire:click="addNewCategory"
      aria-label="Añadir nueva categoría">
      Añadir
    </button>
  </div>
  <small id="newCategoryHelp" class="form-text">Escribe una nueva categoría y presiona
    Añadir</small>
</div>

```

- **Asignación de Tallas y Stock:** Al igual que con las categorías, se incluye un campo adicional que permite registrar una nueva talla. Las tallas disponibles se cargan desde la base de datos y se muestran junto a campos numéricos para añadir individualmente su stock. El sistema exige que al menos una talla tenga stock asignado para poder guardar el producto.

```

<h5 class="my-4" role="heading" aria-level="3">Asignar Stock por Talla</h5>
@error('sizes')
  <div class="text-danger" role="alert">{{ '*Debes asignar stock a al menos una talla.' }}</div>
@enderror
<small id="sizeHelp" class="form-text mb-1">Es OBLIGATORIO que al menos una talla tenga Stock</small>
<div class="d-flex flex-wrap gap-3" role="group" aria-labelledby="sizesLabel">
  @foreach ($availableSizes as $size)
    <div class="d-flex align-items-center mb-2">
      <label for="size-{{ $size->id }}" class="form-label mb-0 me-2"
        style="min-width: fit-content;" id="sizesLabel">{{ $size->name }}</label>
      <input type="number" id="size-{{ $size->id }}"
        wire:model="sizes.{{ $size->id }}" class="form-control form-control-sm me-4"
        placeholder="Stock" min="0" style="width: 80px;"
        aria-describedby="Campo de stock para la talla {{ $size->name }}">
    </div>
  @endforeach
  <div class="d-flex align-items-center mb-2">
    <label for="newSize" class="form-label mb-0 me-2" style="min-width: fit-content;">Otra
      talla:</label>
    <div class="input-group input-group-sm">
      <input type="text" id="newSize" wire:model="newSize"
        class="form-control form-control-sm" placeholder="Ej: XS, M, XL..."
        aria-describedby="newSizeHelp">
      <button class="btn btn-outline-dark" type="button" wire:click="addNewSize"
        aria-label="Añadir nueva talla">
        Añadir
      </button>
    </div>
  </div>
</div>

```


- Botones: El formulario cuenta con los botones para Crear/Actualizar que llaman a su correspondiente método del componente, y un botón para cancelar la acción y volver para atrás.

```
<div class="col-12 d-flex gap-2">
  <button type="submit" class="btn btn-primary btn-sm"
    aria-label="{{ $product_id ? 'Actualizar producto' : 'Crear producto' }}">
    {{ $product_id ? 'Actualizar' : 'Crear' }}
  </button>
  <button type="button" wire:click="cancel" class="btn btn-secondary btn-sm"
    aria-label="Cancelar acción">Cancelar</button>
</div>
```

5.5.7 - PRODUCTSHOW.PHP

FUNCIONALIDAD: Mostrar la información detallada de un producto seleccionado, incluyendo sus imágenes, descripción, precio (con o sin descuento) y un botón para añadirlo al carrito.

- Variables:
 - **product:** Producto que será mostrado en la vista. Se carga al montar el componente, incluyendo sus imágenes relacionadas.
- `mount($id)`: Carga desde la base de datos el producto correspondiente, incluyendo sus imágenes (`with('images')`).

```
public function mount($id): void
{
    $this->product = Product::with(relations: 'images')->findOrFail(id: $id);
}
```

- `render()`: Devuelve la vista asociada al componente (`livewire.product-show`) para su renderizado en el navegador.

```
public function render(): View
{
    // Devuelve la vista 'livewire.product-show' con los datos del producto
    return view(view: 'livewire.product-show');
}
```

VISTA:

- Botón de volver atrás: Usa url()->previous() para regresar a la página anterior.

```
<a href="{{ url()->previous() }}" class="btn btn-secondary mb-3" style="width: 100px;" title="Volver atrás"
  aria-label="Volver a la página anterior">
  <i class="bi bi-arrow-left"></i> Volver
</a>
```

- Imágenes del producto: Se muestra de formas diferentes dependiendo de si hay una o más imágenes.
 - Si hay solo una imagen: se muestra en formato img individual.

```
@if ($product && $product->images->count() > 0)
  @if ($product->images->count() === 1)
    {{-- Solo una imagen --}}
    name }} - {{ $product->category }}, para {{ $product->gender }}"
      aria-label="Imagen del producto {{ $product->name }}"
      title="Imagen del producto {{ $product->name }}" class="img-fluid rounded shadow-sm" />
```

- Si hay múltiples imágenes: se utiliza un carrusel de Bootstrap personalizado con controles de navegación (flechas izquierda y derecha).

```
@else
  {{-- Carrusel Bootstrap --}}
  <div id="carouselProductImages{{ $product->id }}" class="carousel slide" data-bs-ride="carousel">
    <div class="carousel-inner rounded shadow-sm">
      @foreach ($product->images as $index => $image)
        <div class="carousel-item {{ $index === 0 ? 'active' : '' }}">
          name }}"
            aria-label="Imagen {{ $index + 1 }} del producto {{ $product->name }}"
            title="Imagen {{ $index + 1 }} del producto {{ $product->name }}" />
        </div>
      @endforeach
    </div>
  </div>
```

```
<!-- Flecha izquierda -->
<button class="carousel-control-prev" type="button"
  style="background-color: rgba(255, 255, 255, 0.7);"
  data-bs-target="#carouselProductImages{{ $product->id }}" data-bs-slide="prev">
  <span class="carousel-control-prev-icon" style="filter: invert(1);"
    aria-hidden="true"></span>
</button>

<!-- Flecha derecha -->
<button class="carousel-control-next" type="button"
  style="background-color: rgba(255, 255, 255, 0.7);"
  data-bs-target="#carouselProductImages{{ $product->id }}" data-bs-slide="next">
  <span class="carousel-control-next-icon" style="filter: invert(1);"
    aria-hidden="true"></span>
</button>
</div>
@endif
```

- Si no hay imágenes se muestra un mensaje (aunque no debería ocurrir).

```
@else
    <p>No hay imágenes disponibles para este producto.</p>
@endif
```

- Detalles del producto: Se muestra el nombre del producto, su categoría, y género, la descripción, el precio (si hay descuento se muestra el precio original tachado y el nuevo precio en rojo y destacado), y se llama al livewire “<livewire:add-to-cart>”.

```
<div class="col-md-8">
    <h3>{{ $product->name }}</h3>
    <p class="text-muted">Categoría: {{ $product->category }} / {{ $product->gender }}</p>

    <!-- Descripción larga -->
    <p>{{ $product->description }}</p>

    <!-- Precio -->
    @if ($product->discount > 0)
        <p>
            <span
                class="text-muted text-decoration-line-through">{{ number_format(num: $product->price, decimals: 2) }}€</span>
            <span
                class="fw-bold fs-5 text-danger ms-2">{{ number_format(num: $product->price - ($product->price * $product->discount) / 100, decimals: 2) }}€</span>
        </p>
    @else
        <p class="fw-bold fs-5">{{ number_format(num: $product->price, decimals: 2) }}€</p>
    @endif

    <livewire:add-to-cart :product="$product" />
</div>
```

5.5.8 – SEEMYORDERS.PHP

FUNCIONALIDAD: Muestra todos los pedidos realizados por el usuario autenticado, incluyendo los productos, tallas, cantidades, precios y estado del pedido.

- Variables:
 - **orders:** Contiene la colección de pedidos pertenecientes al usuario actual, con todos los datos cargados.

```
public $orders;
```

- mount(): Obtiene todos los pedidos del usuario autenticado usando Auth::user()->orders(), y utiliza with('items.product', 'items.size') para cargar de forma anticipada los productos y las tallas asociadas a cada ítem del pedido, optimizando las consultas a la base de datos.

```
public function mount(): void
{
    $this->orders = Auth::user()
        ->orders()
        // Cargamos en la misma consulta los productos y tallas de cada ítem del pedido para optimizar
        ->with(relations: 'items.product', callback: 'items.size')
        ->get();
}
```

- `render()`: Devuelve la vista `livewire.see-my-orders`, que se encargará de mostrar todos los pedidos del usuario.

```
public function render(): View
{
    // Devuelve la vista 'livewire.see-my-orders' que mostrará los pedidos
    return view(view: 'livewire.see-my-orders');
}
```

VISTA:

- Mostrar los pedidos: Se comprueba si el usuario tiene pedidos (`$orders->count()`).
 - Si tiene se itera sobre cada uno de ellos, y se muestra los detalles más relevantes como el id del pedido, la fecha, el estado, precio y los productos con su talla, precio individual y cantidad.

```
@if ($orders->count())
    @foreach ($orders as $order)
        <div class="card mb-3">
            <div class="card-header">
                <strong>Pedido #{{ $order->id }}</strong> |
                Fecha: {{ $order->created_at->format('d/m/Y') }} |
                Estado: <span class="badge bg-info">{{ $order->status }}</span>
            </div>
            <div class="card-body">
                @foreach ($order->items as $item)
                    <div class="row mb-2">
                        <div class="col-md-2">
                            product->name }}" class="img-fluid">
                        </div>
                        <div class="col-md-10">
                            <h5>{{ $item->product->name }}</h5>
                            <p>{{ $item->product->description }}</p>
                            <p>
                                <strong>Cantidad:</strong> {{ $item->quantity }} |
                                <strong>Precio:</strong> ${{ $item->price }} |
                                <strong>Talla:</strong> {{ $item->size->name ?? 'N/A' }}
                            </p>
                        </div>
                    </div>
                @endforeach
                <div class="text-end">
                    <strong>Total:</strong> ${{ $order->total }}
                </div>
            </div>
        </div>
    @endforeach
```

- Y si no tiene pedidos, se muestra un mensaje informativo con un alert de bootstrap.

```
@else
    <div class="alert alert-info">
        Aún no tienes pedidos realizados.
    </div>
@endif
```

5.5.9 – SHOWCART.PHP

FUNCIONALIDAD: Gestiona la visualización, selección, modificación y eliminación de productos del carrito de compras del usuario autenticado. También permite proceder con los productos seleccionados para la creación de un pedido.

- Variables:

- **cartItems:** Lista de items del carrito del usuario con relación a productos e imágenes.
- **totalPrice:** Precio total de los productos seleccionados.
- **selectedItems:** IDs de los productos seleccionados para la compra.
- **itemToDelete:** ID del producto marcado para eliminación (si se usa en un modal de confirmación).
- **message:** Mensaje de estado mostrado al usuario.
- **messageType:** Tipo de mensaje (success, error, warning).
- **showMessage:** Booleano para controlar la visibilidad del mensaje.

```
public $cartItems;
1 reference
public $totalPrice = 0;
7 references
public $selectedItems = [];
0 references
public $itemToDelete = null;
1 reference
public $message = null;
1 reference
public $messageType = 'success';
1 reference
public $showMessage = false;
```

- **mount():** Se ejecuta al cargar el componente. Obtiene o crea el carrito del usuario autenticado, carga los items del carrito con productos e imágenes asociadas, y si el carrito está vacío, muestra un mensaje de advertencia. También llama al método de “updateTotal()”.

```
public function mount(): void
{
    $user = auth()->user();
    $cart = $user->cart;

    // Si el usuario no tiene carrito, se crea uno vacío
    if (!$cart) {
        $cart = $user->cart()->create();
    }

    // Carga los items del carrito con la relación de producto e imágenes
    $this->cartItems = $cart->items()->with('product.images')->get();

    // Si el carrito está vacío, mostramos mensaje de advertencia
    if ($this->cartItems->isEmpty()) {
        $this->setMessage(message: 'Todavía no hay productos en el carrito', messageType: 'warning');
    }

    // Calcula el total de productos seleccionados
    $this->updateTotal();
}
```

- `itemSelection($itemId)`: Guarda o saca del array de `$selectedItems` las tarjetas que estén o no seleccionadas, y actualiza el total tras cada cambio llamando a la función `updateTotal()`.

```
public function itemSelection($itemId): void
{
    // Si el item ya estaba seleccionado, lo quitamos
    if (in_array(needle: $itemId, haystack: $this->selectedItems)) {
        $this->selectedItems = array_filter(array: $this->selectedItems, callback: function ($id) use ($itemId): bool {
            return $id != $itemId;
        });
    } else {
        // Si no estaba seleccionado, lo añadimos
        $this->selectedItems[] = $itemId;
    }

    // Actualizamos el total teniendo en cuenta los productos seleccionados
    $this->updateTotal();
}
```

- `updateTotal()`: Recalcula el precio total según los productos seleccionados, y aplica descuentos si los hay.

```
public function updateTotal(): void
{
    $total = 0; // Inicializamos el total en 0

    foreach ($this->cartItems as $item) {
        // Verificamos si el ítem está entre los seleccionados
        if (in_array(needle: $item->id, haystack: $this->selectedItems)) {

            // Precio base del producto (sin descuento)
            $price = $item->product->price;

            // Si el producto tiene un descuento, se aplica
            if ($item->product->discount && $item->product->discount > 0) {
                $discount = $item->product->discount;
                $price = $price - ($price * $discount / 100); // Precio con descuento aplicado
            }

            // Sumamos al total la cantidad multiplicada por el precio (con o sin descuento)
            $total += $item->quantity * $price;
        }
    }

    // Asignamos el total calculado a la propiedad del componente
    $this->totalPrice = $total;
}
```

- `confirmDeletion($itemId)`: Elimina el producto del carrito, refresca la lista de items y el total y muestra un mensaje confirmando la eliminación del producto del carrito.

```
public function confirmDeletion($itemId): void
{
    $cartItem = auth()->user()->cart->items()->find($itemId);
    if ($cartItem) {
        $cartItem->delete();
        // Actualizamos la lista de items del carrito y el total
        $this->cartItems = auth()->user()->cart->items();
        $this->updateTotal();
        $this->setMessage(message: 'Producto eliminado del carrito', messageType: 'success');
    }

    // Si el carrito queda vacío, mostramos mensaje de advertencia
    if ($this->cartItems->isEmpty()) {
        $this->setMessage(message: 'Todavía no hay productos en el carrito', messageType: 'warning');
    }
}
```

- `updateCartItem($itemId, $quantity, $size)`: Actualiza la cantidad y talla de un ítem del carrito, refresca el total, y muestra mensaje de confirmación o error.

```
public function updateCartItem($itemId, $quantity, $size): void
{
    $cartItem = auth()->user()->cart->items()->find($itemId);
    if ($cartItem) {
        $cartItem->quantity = $quantity;
        $cartItem->size_id = $size;
        $cartItem->save();

        // Refrescamos los items y el total para reflejar los cambios
        $this->cartItems = auth()->user()->cart->items;
        $this->updateTotal();
        $this->setMessage(message: 'Producto actualizado', messageType: 'success');
    } else {
        $this->setMessage(message: 'No se ha podido actualizar el producto', messageType: 'error');
    }
}
```

- `setMessage($message, $messageType = 'success')`: Establece el mensaje de estado que se mostrará en la vista.

```
public function setMessage($message, $messageType = 'success'): void
{
    $this->message = $message;
    $this->messageType = $messageType;
    $this->showMessage = true;
}
```

- `makeOrder()`: Verifica si hay productos seleccionados, guarda en sesión los IDs seleccionados, y redirige a la ruta `user.orderConfirmation`.

```
public function makeOrder(): mixed|RedirectResponse
{
    if (empty($this->selectedItems)) {
        $this->setMessage(message: 'Debes seleccionar al menos un producto', messageType: 'warning');
        return;
    }

    // Guardamos en sesión los IDs de los productos seleccionados para usar en la confirmación
    session()->put(key: 'selectedItems', value: $this->selectedItems);

    // Redirigimos a la ruta de confirmación de pedido
    return redirect()->route(route: 'user.orderConfirmation');
}
```

- `render()`: Devuelve la vista `livewire.show-cart`.

```
public function render(): View
{
    return view(view: 'livewire.show-cart');
}
```


VISTA:

- Mensajes de estado:

```
@if ($message)
<div class="alert alert-{{ $messageType == 'success' ? 'success' : ($messageType == 'warning' ? 'warning' : 'danger') }}" mb-4"
    role="alert" aria-live="assertive" aria-atomic="true" title="Mensaje de estado">
    {{ $message }}
</div>
@endif
```

- Mostrar los productos: Se itera sobre \$cartItems, y se van mostrando cada producto que está guardado en el carrito, mostrandose de forma visual como una tarjeta que contiene:
 - Un checkbox para seleccionar el producto.

```
<div class="col-auto">
    <!-- Checkbox -->
    <input type="checkbox" wire:click="itemSelection({{ $item->id }})"
        {{ in_array(needle: $item->id, haystack: $selectedItems) ? 'checked' : '' }}
        aria-label="Seleccionar producto {{ $item->product->name }}"
        title="Seleccionar producto {{ $item->product->name }}">
</div>
```

- Una imagen del producto.

```
product->name }} - {{ $item->product->category }}, para {{ $item->product->gender }}"
    aria-label="Imagen del producto {{ $item->product->name }}"
    title="Imagen del producto {{ $item->product->name }}"
    class="img-fluid rounded shadow-sm" style="max-height: 250px;">
```

- El nombre y descripción.

```
<div class="col" role="region"
    aria-label="Detalles del producto {{ $item->product->name }}">
    <h3 class="h5">{{ $item->product->name }}</h3>
    <p class="text-muted">{{ $item->product->description }}</p>
```

- Talla y cantidad seleccionada.

```
<label for="size-{{ $item->id }}"
    title="Etiqueta para talla del producto {{ $item->product->name }}">
    Talla
</label>
<select wire:model="size" class="form-control"
    id="size-{{ $item->id }}"
    wire:change="updateCartItem({{ $item->id }}, {{ $item->quantity }}, $event.target.value)"
    aria-label="Seleccionar talla para {{ $item->product->name }}"
    title="Seleccionar talla para {{ $item->product->name }}">
    @foreach ($item->product->sizes as $size)
        <option value="{{ $size->id }}"
            {{ $size->id == $item->size_id ? 'selected' : '' }}>
            {{ $size->name }}
        </option>
    @endforeach
</select>
```



```

<label for="quantity-{{ $item->id }}"
  title="Etiqueta para cantidad del producto {{ $item->product->name }}">
  Cantidad
</label>
<input type="number" wire:model="quantity" min="1"
  class="form-control" id="quantity-{{ $item->id }}"
  value="{{ $item->quantity }}"
  wire:change="updateCartItem({{ $item->id }}, $event.target.value, {{ $item->size_id }})"
  aria-label="Cantidad para {{ $item->product->name }}"
  title="Cantidad para {{ $item->product->name }}">

```

- Precio con o sin descuento.

```

<div class="row">
  <div class="col-md-6">
    <p class="fs-6 fw-semibold">
      aria-label="Precio unidad del producto {{ $item->product->name }}"
      Precio Unidad:
      @if ($item->product->discount)
        <span class="text-muted text-decoration-line-through">
          aria-label="Precio original tachado"
          {{ number_format(num: $item->product->price, decimals: 2) }} €
        </span>
        <span class="text-danger fw-bold fs-5">
          aria-label="Precio con descuento"
          {{ number_format(num: $item->product->price * (1 - $item->product->discount / 100), decimals: 2) }} €
        </span>
      @else
        {{ number_format(num: $item->product->price, decimals: 2) }} €
      @endif
    </p>
  </div>
</div>

```

- Precio total del producto.

```

<div class="col-md-6">
  <p class="fs-6 fw-semibold">
    aria-label="Precio total del producto {{ $item->product->name }}"
    Precio Total:
    {{ number_format(num: $item->product->price * (1 - $item->product->discount / 100) * $item->quantity, decimals: 2) }} €
  </p>
</div>

```

- Botones para ver detalles del producto o para eliminar del carrito.

```

<div class="col-auto d-flex justify-content-around flex-column" role="group"
  aria-label="Acciones para el producto {{ $item->product->name }}">
  <a href="{{ route(name: 'productos.show', parameters: $item->product->id) }}"
    class="btn btn-outline-primary mb-2" aria-label="Ver Producto" title="Ver Producto">
    <i class="bi bi-eye" aria-hidden="true"></i> Ver Producto
  </a>
  <a wire:click="confirmDeletion({{ $item->id }})" class="btn btn-danger mt-2"
    role="button" tabindex="0"
    aria-label="Eliminar producto {{ $item->product->name }} del carrito"
    title="Eliminar producto {{ $item->product->name }} del carrito">
    <i class="bi bi-cart-dash" aria-hidden="true"></i> Eliminar
  </a>
</div>

```

- Conteo de productos seleccionados y su precio final:

```
<p class="text-right me-5 m-0" aria-label="Cantidad de productos seleccionados">
|   <strong>Cantidad de productos seleccionados: {{ count(value: $selectedItems) }}</strong>
</p>
<p class="text-right me-5 m-0" aria-label="Precio total de productos seleccionados">
|   <strong>Total: {{ number_format(num: $totalPrice, decimals: 2) }} €</strong>
</p>
```

- Botón para realizar el pedido:

```
<button class="btn btn-primary me-3" wire:click="makeOrder" aria-label="Realizar pedido"
|   title="Realizar pedido">
|   Realizar pedido
</button>
```

5.5.10 – SHOWSTOREPROD.PHP

FUNCIONALIDAD: Permite filtrar productos en la tienda donde se muestran todos los productos que hay en la base de datos. Presenta un formulario de búsqueda avanzada y dinámico, y muestra los resultados directamente sin recargar la página, gracias al uso de Livewire.

- Variables:
 - **gender:** género del producto.
 - **categories:** categorías seleccionadas.
 - **color:** color del producto.
 - **style:** estilo del producto.
 - **priceMin:** precio mínimo.
 - **priceMax:** precio máximo.
 - **sizes:** tallas seleccionadas.
 - **onlyOffers:** si se deben mostrar sólo productos con descuento.
 - **prodFiltrado:** resultados filtrados.
 - **availableSizes:** todas las tallas disponibles.
 - **gendersList, categoriesList, colorsList, stylesList:** valores únicos de productos para generar filtros dinámicos.

```
public $gender;
4 references
public $categories = [];
4 references
public $color;
4 references
public $priceMin;
4 references
public $priceMax;
4 references
public $sizes = [];
1 reference
public $availableSizes;
4 references
public $style;

// Listas para mostrar opciones de filtros dinámicos
1 reference
public $gendersList = [];
1 reference
public $categoriesList = [];
1 reference
public $colorsList = [];
1 reference
public $stylesList = [];

0 references
public $orderBy;

3 references
public $onlyOffers = false;

6 references
public $prodFiltrado;
```

- `mount()`: Inicializa las listas dinámicas para los filtros (géneros, categorías, colores, estilos) a partir de los valores únicos en la base de datos, y carga todas las tallas desde la tabla `sizes`.

```
public function mount(): void
{
    // Obtenemos valores distintos de cada atributo para los filtros
    $this->gendersList = Product::distinct()->pluck(column: 'gender')->toArray();
    $this->categoriesList = Product::distinct()->pluck(column: 'category')->toArray();
    $this->colorsList = Product::distinct()->pluck(column: 'color')->toArray();
    $this->stylesList = Product::distinct()->pluck(column: 'style')->toArray();

    // Todas las tallas disponibles (tabla Size)
    $this->availableSizes = Size::all();
}
```

- `filter()`: Construye dinámicamente una consulta aplicando los filtros seleccionados por el usuario. Se puede filtrar por género, categorías, color, estilo, tallas, precio (mínimo y máximo), y descuento.

```
public function filter(): void
{
    $query = Product::query();

    $this->prodFiltrado = null;

    // Aplicamos filtro por género
    if (!empty($this->gender)) {
        $query->where(column: 'gender', operator: $this->gender);
    }

    // Filtro por categorías (pueden ser múltiples)
    if (!empty($this->categories)) {
        $query->whereIn(column: 'category', values: $this->categories);
    }

    // Filtro por color
    if (!empty($this->color)) {
        $query->where(column: 'color', operator: $this->color);
    }

    // Filtro por estilo
    if (!empty($this->style)) {
        $query->where(column: 'style', operator: $this->style);
    }

    // Precio mínimo
    if (!empty($this->priceMin)) {
        $query->where(column: 'price', operator: '>=', value: $this->priceMin);
    }

    // Precio máximo
    if (!empty($this->priceMax)) {
        $query->where(column: 'price', operator: '<=', value: $this->priceMax);
    }

    // Filtro por tallas (relación many-to-many)
    if (!empty($this->sizes)) {
        $query->whereHas(relation: 'sizes', callback: function (Builder<TRelatedModel> $q) {
            $q->whereIn(column: 'name', values: $this->sizes);
        });
    }

    // Mostrar solo productos en oferta
    if ($this->onlyOffers) {
        $query->where(column: 'discount', operator: '>', value: 0);
    }
}
```

Además, se comprueba si no se ha aplicado ningún filtro y lanza un mensaje con `session()->flash()` para avisar al usuario, o si no hay ningún producto con las características seleccionadas.

```
// Si no hay filtros aplicados, mostramos error y limpiamos resultados
if (
    empty($this->gender) && empty($this->categories) && empty($this->color) && empty($this->style) &&
    empty($this->priceMin) && empty($this->priceMax) && empty($this->sizes) && !$this->onlyOffers
) {
    session()->flash(key: 'error', value: 'Introduce algún dato para filtrar');
    $this->prodFiltrado = collect(); // Mejor usar colección vacía que array
    return;
}

// Ejecutamos la consulta y guardamos los productos filtrados
$this->prodFiltrado = $query->with(relations: 'images')->get();

// Si no hay resultados, mostramos mensaje de error
if ($this->prodFiltrado->isEmpty()) {
    session()->flash(key: 'error', value: 'No se ha encontrado ningún producto con esas características.');
```

- `resetFilters()`: Limpia todos los campos de filtro y reinicia el listado.

```
public function resetFilters(): void
{
    $this->gender = null;
    $this->categories = [];
    $this->color = null;
    $this->priceMin = null;
    $this->priceMax = null;
    $this->sizes = [];
    $this->style = null;
    $this->onlyOffers = false;
    $this->prodFiltrado = null;
}
```

- `render()`: Devuelve la vista `livewire.show-cart`, pasando los productos filtrados si existen, o todos los productos por defecto.

```
public function render(): View
{
    $products = $this->prodFiltrado ?? Product::with(relations: 'images')->get();
    return view(view: 'livewire.show-store-prod', data: compact('products'));
}
```

VISTA:

- Formulario de filtros (en `<aside>`): Exceptuando los filtros de precio y descuento, el resto de los filtros se generan dinámicamente a partir de los valores existentes en la base de datos de productos. Es decir, se recorren los registros almacenados y se extraen los valores únicos de cada campo para escribir los filtros correspondientes que son:

```
<aside class="col-md-3 mb-4">
    <form class="p-5 rounded border bg-light" wire:submit.prevent="filter"
        aria-label="Formulario de filtro de productos">
```

- Género (radio)

```
<h3 class="h5">Género</h3 class="h5">
@foreach ($gendersList as $genero)
    <div class="form-check">
        <input class="form-check-input" type="radio" wire:model="gender"
            id="gender-{{ $genero }}" name="gender" value="{{ $genero }}"
            aria-label="Género {{ $genero }}" title="Género {{ $genero }}">
        <label class="form-check-label" for="gender-{{ $genero }}">{{ ucfirst(string: $genero) }}</label>
    </div>
@endforeach
```

- Categorías (checkboxes múltiples)

```
<h3 class="h5">Categoría</h3 class="h5">
<div class="row">
    @foreach ($categoriesList as $categoria)
        <div class="col-6 col-md-4 col-lg-3 me-5 pe-5">
            <div class="form-check">
                <input class="form-check-input" type="checkbox" wire:model="categories"
                    id="cat-{{ $categoria }}" name="categories[]" value="{{ $categoria }}"
                    aria-label="Categoría {{ $categoria }}" title="Categoría {{ $categoria }}">
                <label class="form-check-label"
                    for="cat-{{ $categoria }}">{{ ucfirst(string: $categoria) }}</label>
            </div>
        </div>
    @endforeach
</div>
```

- Color (select)

```
<h3 class="h5">Color</h3 class="h5">
<select class="form-select" wire:model="color" name="color" id="color"
    aria-label="Filtrar por color" title="Selecciona un color">
    <option value="">Todos</option>
    @foreach ($colorsList as $colorItem)
        <option value="{{ $colorItem }}">{{ ucfirst(string: $colorItem) }}</option>
    @endforeach
</select>
```

- Talla (checkboxes múltiples)

```
<h3 class="h5">Talla</h3 class="h5">
<div class="row">
    @foreach ($availableSizes as $size)
        <div class="col-6 col-md-4 col-lg-3 me-3">
            <div class="form-check">
                <input class="form-check-input" type="checkbox" wire:model="sizes"
                    id="size-{{ $size->name }}" name="sizes[]" value="{{ $size->name }}"
                    aria-label="Talla {{ strtoupper(string: $size->name) }}"
                    title="Talla {{ strtoupper(string: $size->name) }}">
                <label class="form-check-label"
                    for="size-{{ $size->name }}">{{ strtoupper(string: $size->name) }}</label>
            </div>
        </div>
    @endforeach
</div>
```

- Estilo (radio)

```
<h3 class="h5">Estilo</h3 class="h5">
@foreach ($stylesList as $styleItem)
    <div class="form-check">
        <input class="form-check-input" type="radio" wire:model="style"
            id="style-{{ $styleItem }}" name="style" value="{{ $styleItem }}"
            aria-label="Estilo {{ $styleItem }}" title="Estilo {{ $styleItem }}">
        <label class="form-check-label"
            for="style-{{ $styleItem }}">{{ ucfirst(string: $styleItem) }}</label>
    </div>
@endforeach
```

- Precio mínimo y máximo (inputs numéricos)

```
<h3 class="h5">Precio</h3 class="h5">
<input type="number" wire:model="priceMin" class="form-control" placeholder="Precio mínimo"
    id="priceMin" name="priceMin" min="0" step="0.01" aria-label="Precio mínimo"
    title="Precio mínimo">
<input type="number" wire:model="priceMax" class="form-control mt-2" placeholder="Precio máximo"
    id="priceMax" name="priceMax" min="0" step="0.01" aria-label="Precio máximo"
    title="Precio máximo">
```

- Checkbox para mostrar solo ofertas

```
<h3 class="h5">Ofertas</h3 class="h5">
<div class="form-check">
    <input class="form-check-input" type="checkbox" wire:model="onlyOffers" id="onlyOffers"
        name="onlyOffers" aria-label="Solo productos en oferta"
        title="Mostrar solo productos en oferta">
    <label class="form-check-label" for="onlyOffers">Mostrar solo productos en oferta</label>
</div>
```

- Botones de "Filtrar" y "Borrar filtros"

```
<button type="submit" class="btn btn-primary" aria-label="Filtrar" title="Filtrar">Filtrar</button>
<button type="reset" class="btn btn-outline-dark"
    onmouseover="this.classList.replace('btn-outline-dark','btn-outline-secondary')"
    onmouseout="this.classList.replace('btn-outline-secondary','btn-outline-dark')"
    wire:click="resetFilters" aria-label="Borrar filtros" title="Borrar filtros">
    Borrar filtros
</button>
```

- Resultados de productos (en <section>): Muestra los productos (filtrados o todos por defecto) en tarjetas de Bootstrap, cada una con:

- Imagen del producto

```
name }}">
```

- Nombre y descripción

```
<h3 class="h5" class="card-title">{{ $product->name }}</h3 class="h5">
<p class="card-text">{{ $product->description }}</p>
```

- Precio, con precio original tachado si hay descuento

```
@if ($product->discount > 0)
<p>
  <span class="text-muted text-decoration-line-through">
    {{ number_format(num: $product->price, decimals: 2) }}€
  </span>
  <span class="fw-bold fs-5 text-danger ms-2">
    {{ number_format(num: $product->price - ($product->price * $product->discount) / 100, decimals: 2) }}€
  </span>
</p>
@else
  <p class="fw-bold fs-5">{{ number_format(num: $product->price, decimals: 2) }}€</p>
@endif
```

- Llama al livewire Addtocart.php, para añadir ese producto al carrito.

```
<livewire:add-to-cart :product="$product" :key="'add-to-cart-' . $product->id . '-' . $loop->index" />
```

- Botón para ver los detalles del producto: redirige a user.product.

```
<a href="{{ route(name: 'productos.show', parameters: $product->id) }}"
  class="btn btn-outline-primary w-100 mt-3"
  aria-label="Ver detalles del producto {{ $product->name }}"
  title="Ver detalles del producto {{ $product->name }}">Ver detalles del
  Producto</a>
```

5.5.11 – USERCRUD.PHP

FUNCIONALIDAD: Permite la gestión completa de usuarios del sistema (crear, leer, actualizar, eliminar) con filtros dinámicos por nombre y rol. También permite alternar entre vista de lista y formulario de edición/creación.

- Variables:
 - **users, user_id, name, email, role:** datos del usuario.
 - **filterName:** Texto a filtrar por nombre.
 - **filterRole:** Filtro por tipo de rol.
 - **roles:** Mapea los roles con sus nombres para los select.
 - **view:** Determina qué vista se muestra (list o form).

```
public $users, $user_id, $name, $email, $role;
3 references
public $filterName = '';
3 references
public $filterRole = '';

// Roles definidos para mostrar en un select (clave => valor)
0 references
public $roles = [
  '0' => 'Usuario',
  '1' => 'Administrador',
];

// Controla la vista actual, puede ser 'list' o 'form'
5 references
public $view = 'list';
```


- `mount()`: Carga inicial de todos los usuarios al iniciar el componente, llamando a la función `loadUsers()`.

```
public function mount(): void
{
    $this->loadUsers();
}
```

- `filter()`: Filtra usuarios por nombre (LIKE) y por rol (exacto). Si no hay resultado manda un mensaje de error.

```
public function filter(): void
{
    $query = User::query();

    if (!empty($this->filterName)) {
        // Busca usuarios cuyo nombre contenga el texto del filtro (consulta LIKE)
        $query->where(column: 'name', operator: 'like', value: '%' . $this->filterName . '%');
    }
}
```

```
    if ($this->filterRole !== '') {
        // Filtra por rol exacto
        $query->where(column: 'role', operator: $this->filterRole);
    }

    // Ejecuta la consulta y guarda los resultados en $this->users
    $this->users = $query->get();

    if ($this->users->isEmpty()) {
        // Si no hay resultados, muestra mensaje de error y recarga todos usuarios
        session()->flash(key: 'error', value: 'No se encontraron usuarios con esos filtros.');
```

- `resetFilters()`: Limpia todos los filtros y recarga todos los usuarios.

```
public function resetFilters(): void
{
    $this->filterName = '';
    $this->filterRole = '';
    $this->loadUsers();
}
```

- `resetInputs()`: Limpia los campos del formulario.

```
public function resetInputs(): void
{
    $this->user_id = null;
    $this->name = '';
    $this->email = '';
    $this->role = '';
}
```

- `showCreateForm()`: Cambia la vista a formulario para crear un nuevo usuario.

```
public function showCreateForm(): void
{
    $this->resetInputs();
    $this->view = 'form';
}
```


- `edit($id)`: Carga un usuario específico por ID para edición.

```
public function edit($id): void
{
    $user = User::findOrFail(id: $id);
    $this->user_id = $user->id;
    $this->fill(values: $user->toArray());
    $this->view = 'form';
}
```

- `store()`: Valida y guarda un nuevo usuario en la base de datos.

```
public function store(): void
{
    $this->validate(rules: [
        'name' => 'required|string|max:255',
        'email' => 'required|email|unique:users,email',
        'role' => 'required|in:0,1',
    ]);

    User::create(attributes: $this->validateData());

    session()->flash(key: 'message', value: 'Usuario creado correctamente.');
```

- `update()`: Valida y actualiza un usuario existente.

```
public function update(): void
{
    $this->validate(rules: [
        'name' => 'required|string|max:255',
        'email' => 'required|email|unique:users,email,' . $this->user_id,
        'role' => 'required|in:0,1',
    ]);

    $user = User::findOrFail(id: $this->user_id);
    $user->update($this->validateData());

    session()->flash(key: 'message', value: 'Usuario actualizado correctamente.');
```

- `cancel()`: Cancela la creación o edición y regresa a la vista de lista.

```
public function cancel(): void
{
    $this->resetInputs();
    $this->view = 'list';
}
```

- `delete($id)`: Elimina un usuario por su ID.

```
public function delete($id): void
{
    User::findOrFail(id: $id)->delete();
    session()->flash(key: 'message', value: 'Usuario eliminado correctamente.');
```

- loadUsers(): Método privado reutilizable para cargar todos los usuarios.

```
private function loadUsers(): void
{
    $this->users = User::all();
}
```

- validateData(): Método privado que retorna los datos validados para crear/editar.

```
private function validateData(): array
{
    return [
        'name' => $this->name,
        'email' => $this->email,
        'role' => $this->role,
    ];
}
```

- render(): Devuelve la vista user-crud.blade.php.

```
public function render(): View
{
    return view(view: 'livewire.user-crud');
}
```

VISTA:

Vista list:

- Filtros por nombre y rol con botones para aplicar y borrar filtros.

```
<div class="card mb-4">
  <div class="card-header bg-light">
    <strong>Filtrar Usuarios</strong>
  </div>
  <div class="card-body">
    <form wire:submit.prevent="filter" class="row g-3 align-items-end">
      <div class="col-md">
        <label for="filterName" class="form-label">Nombre</label>
        <input type="text" id="filterName" name="filterName" wire:model.defer="filterName" class="form-control" placeholder="Ej: María">
      </div>

      <div class="col-md">
        <label for="filterRole" class="form-label">Rol</label>
        <select id="filterRole" name="filterRole" wire:model.defer="filterRole" class="form-select">
          <option value="">-- Todos --</option>
          @foreach ($roles as $key => $label)
            <option value="{{ $key }}">{{ $label }}</option>
          @endforeach
        </select>
      </div>

      <div class="col-auto">
        <button type="submit" class="btn btn-primary">
          <i class="bi bi-search"></i> Filtrar
        </button>
      </div>

      <div class="col-auto">
        <button type="button" wire:click="resetFilters" class="btn btn-secondary">
          <i class="bi bi-x-circle"></i> Borrar Filtros
        </button>
      </div>
    </form>
  </div>
</div>
```

- Mensajes de estado:

```
@if (session()-has(key: 'message'))|
| <div class="alert alert-success">{{ session(key: 'message') }}</div>
@endif

@if (session()-has(key: 'error'))
| <div class="alert alert-danger">{{ session(key: 'error') }}</div>
@endif
```

- Tabla responsiva con los campos id, nombre, email, rol ("Administrador" o "Usuario"), y los botones para editar o eliminar. Al pulsar el botón de eliminar debería salir un alert preguntando si está seguro de realizar esa acción.

```
<table class="table table-bordered table-striped w-100 align-items-center">
  <thead class="table-light">
    <tr>
      <th>ID</th>
      <th>Nombre</th>
      <th>Email</th>
      <th>Rol</th>
      <th>Acciones</th>
    </tr>
  </thead>
  <tbody>
    @forelse($users as $user)
      <tr>
        <td>{{ $user->id }}</td>
        <td>{{ $user->name }}</td>
        <td>{{ $user->email }}</td>
        <td>{{ $user->role == 1 ? 'Administrador' : 'Usuario' }}</td>
        <td>
          <button wire:click="edit('{{ $user->id }}')" class="btn btn-sm btn-warning">
            <i class="bi bi-pencil"></i> Editar
          </button>
          <button wire:click="delete('{{ $user->id }}')" class="btn btn-sm btn-danger"
            onclick="confirm('¿Estás seguro de eliminar este usuario?') || event.stopImmediatePropagation()">
            <i class="bi bi-trash"></i> Eliminar
          </button>
        </td>
      </tr>
    @empty
      <tr>
        <td colspan="5" class="text-center">No hay usuarios</td>
      </tr>
    @endforelse
  </tbody>
</table>
```

Vista form:

- Formulario para editar: Contiene los campos nombre, email y rol, siendo el rol un select donde solo hay dos opciones o usuario o administrador.

```
<form wire:submit.prevent="update">
  <div class="mb-3">
    <label for="name" class="form-label">Nombre:</label>
    <input type="text" id="name" name="name" wire:model.defer="name" class="form-control">
    @error('name')
    <span class="text-danger">{{ $errors->first('name') }}</span>
    @enderror
  </div>
```

```

<div class="mb-3">
  <label for="email" class="form-label">Email:</label>
  <input type="email" id="email" name="email" wire:model.defer="email" class="form-control">
  @error('email')
    <span class="text-danger">{{ $errors->first('email') }}</span>
  @enderror
</div>

<div class="mb-3">
  <label for="role" class="form-label">Rol:</label>
  <select id="role" name="role" wire:model.defer="role" class="form-select">
    <option value="">Selecciona un rol</option>
    <option value="0">Usuario</option>
    <option value="1">Administrador</option>
  </select>
  @error('role')
    <span class="text-danger">{{ $errors->first('role') }}</span>
  @enderror
</div>

```

- Botones: para guardar los cambios o cancelar y volver a la lista.

```

<button type="submit" class="btn btn-primary">Guardar Cambios</button>
<button type="button" wire:click="cancel" class="btn btn-secondary ms-2">Cancelar</button>
</form>

```

5.6 - GESTIÓN DE USUARIOS Y ROLES

5.6.1 - ROLES DEFINIDOS

A la hora de crear la tabla de usuario en la base de datos se establecieron dos roles principales dentro del sistema:

```
$table->tinyInteger(column: 'role')->default(value: 0)->comment(comment: '0 para usuarios. 1 para Admin');
```

- Administrador (admin): valor asignado 1. Tiene acceso a casi todas las funcionalidades del sistema, incluyendo la gestión de productos, pedidos y usuarios.
- Usuario (user): valor asignado 0. Tiene acceso limitado a las funcionalidades del sistema.

El campo role es un entero que permite distinguir qué tipo de usuario ha iniciado sesión y aplicar restricciones en consecuencia.

5.6.2 - MIDDLEWARE PARA EL CONTROL DE ACCESO

Para controlar el acceso a unas vistas u otras dependiendo si el rol del usuario es administrador o no, se ha implementado un middleware personalizado que restringe el acceso a ciertas rutas según el rol del usuario autenticado.

```
public function handle(Request $request, Closure $next, $role)
{
    if (!Auth::check()) {
        return redirect()->route(route: 'login');
    }

    if (Auth::user()->role == $role) {
        return $next($request);
    }

    abort(code: 403, message: 'Acceso no autorizado');
}
```

- Este middleware hace:
 - Si el usuario no está autenticado, lo redirige a la ruta de login para que inicie sesión.

```
if (!Auth::check()) {
    return redirect()->route(route: 'login');
}
```

- Si el usuario está autenticado y el valor del campo role coincide con el rol que se pasa como parámetro desde el archivo web.php, permite que continúe hacia la ruta que estaba intentando acceder (es decir, lo deja pasar al siguiente middleware o controlador).

```
if (Auth::user()->role == $role) {
    return $next($request);
}
```

- Si el usuario está autenticado pero su rol no coincide con el rol requerido, lanza un error 403 (Acceso no autorizado) y muestra directamente una página de error 403.

```
abort(code: 403, message: 'Acceso no autorizado');
```

- Este middleware se encuentra en:
 - app/Http/Middleware/UserControl.php

Para que funcione también tenemos que meternos en el archivo bootstrap/app.php (antes había que hacerlo en la carpeta de Kernel pero en Laravel 11 ya no existe), y registrar el middleware:

```
->withMiddleware(callback: function (Middleware $middleware): void {
    // Registramos el middleware personalizado por nombre
    $middleware->alias(alias: [
        'role' => UserControl::class,
    ]);
})
```

5.6.3 – REDIRECCIÓN POST-LOGIN

Después de que un usuario inicia sesión correctamente, el sistema determina su rol y lo redirige automáticamente a la ruta correspondiente, garantizando así una navegación adecuada según su nivel de acceso.

- Esta lógica de redirección se gestiona desde la ruta /dashboard, que está definida en el archivo web.php de la siguiente manera:
 - Usuarios con rol 1 (administradores) son redirigidos al panel de administración.
 - Usuarios con rol 0 (usuarios normales) son redirigidos a su perfil de usuario.

```
Route::get(uri: '/dashboard', action: function (): mixed|RedirectResponse {  
    if (Auth::check()) {  
        // Si el rol es 1 (admin), redirige al dashboard admin; sino, perfil usuario normal  
        return redirect()->route(route: Auth::user()->role == 1 ? 'admin.dashboard' : 'user.profile');  
    }  
})->middleware(middleware: ['auth', 'verified']->name(name: 'dashboard');
```

5.7 – DEFINICIÓN DE RUTAS

Las rutas controlan el acceso a vistas públicas, funcionalidades protegidas por autenticación, y vistas restringidas por roles (usuario normal o administrador), así como las rutas asociadas al sistema de autenticación, configuración de perfil y verificación en dos pasos (2FA).

5.7.1 – RUTAS PÚBLICAS

- / : Lleva a la vista user.index que es la página de inicio de la tienda.
- /tienda: Lleva a la vista user.tienda, donde se muestra el listado de productos.
- /sobreNosotros: Lleva a la vista user.aboutUs, que contiene información sobre la tienda.
- /productos/{id}: Lleva a la vista user.product y muestra el detalle de un producto según su ID.
- /aviso-legal: Lleva a la vista legalNotice, con la información legal del sitio.

- /mapaWeb: Lleva a la vista siteMap, que muestra el mapa del sitio.
- /contacto: Lleva a la vista contact, donde se encuentra el formulario o datos de contacto.
- /accesibilidad: Lleva a la vista accessibility, con información sobre accesibilidad del sitio.

```
Route::view(uri: '/', view: 'user.index')->name(name: 'user.index');

Route::view(uri: '/tienda', view: 'user.tienda')->name(name: 'user.tienda');

Route::view(uri: '/sobreNosotros', view: 'user.aboutUs')->name(name: 'user.aboutUs');

Route::get(uri: '/productos/{id}', action: function ($id): View {
    $product = Product::findOrFail(id: $id);
    return view(view: 'user.product', data: compact(var_name: 'product'));
})->name(name: 'productos.show');

Route::view(uri: '/aviso-legal', view: 'legalNotice')->name(name: 'legalNotice');

Route::view(uri: '/mapaWeb', view: 'siteMap')->name(name: 'siteMap');

Route::view(uri: '/contacto', view: 'contact')->name(name: 'contact');

Route::view(uri: '/accesibilidad', view: 'accessibility')->name(name: 'accessibility');
```

5.7.2 – RUTAS PROTEGIDAS PARA USUARIOS AUTENTICADOS

- /carrito: Lleva a la vista user.cart, que muestra el carrito de compra del usuario.
- /confirmacionPedido: Lleva a la vista user.orderConfirmation, donde se confirma el pedido.
- /configuracion : Redirige automáticamente a /configuracion/perfil.
- /configuracion/perfil : Lleva a la vista settings.profile (Livewire Volt), para configurar el perfil.
- /configuracion/contraseña : Lleva a la vista settings.password (Livewire Volt), para cambiar la contraseña.

- /configuracion/apariencia : Lleva a la vista settings.appearance (Livewire Volt), para personalizar la apariencia.

```
Route::middleware(middleware: ['auth'])->group(callback: function (): void {  
  
    Route::view(uri: '/carrito', view: 'user.cart')->name(name: 'user.cart');  
    Route::view(uri: '/confirmacionPedido', view: 'user.orderConfirmation')->name(name: 'user.orderConfirmation');  
  
    Route::redirect(uri: 'configuracion', destination: 'settings/profile');  
  
    // Rutas de configuración de usuario con Volt  
    Volt::route(uri: 'configuracion/perfil', componentName: 'settings.profile')->name(name: 'settings.profile');  
    Volt::route(uri: 'configuracion/contraseña', componentName: 'settings.password')->name(name: 'settings.password');  
    Volt::route(uri: 'configuracion/apariencia', componentName: 'settings.appearance')->name(name: 'settings.appearance');  
});
```

5.7.3 – RUTAS PARA USUARIOS NORMALES (ROL = 0)

- /usuario/perfil: Lleva a la vista user.profile, que muestra el perfil del usuario autenticado.
- /usuario/pedidos: Lleva a la vista user.orders, donde se muestra el historial de pedidos del usuario.

```
Route::middleware(middleware: ['auth', 'role:0'])->group(callback: function (): void {  
    Route::view(uri: '/usuario/perfil', view: 'user.profile')->name(name: 'user.profile');  
  
    Route::view(uri: '/usuario/pedidos', view: 'user.orders')->name(name: 'user.orders');  
});
```

5.7.4 – RUTAS PARA ADMINISTRADORES (ROL = 1)

- /admin/panel: Lleva a la vista admin.dashboard, que es el panel principal del administrador.
- /admin/productos: Lleva a la vista admin.product, para la gestión CRUD de productos.
- /admin/pedidos: Lleva a la vista admin.order, donde se gestionan los pedidos.
- /admin/usuarios: Lleva a la vista admin.users, para gestionar los usuarios registrados.


```
Route::middleware(middleware: ['auth', 'role:1'])->group(callback: function (): void {
    Route::view(uri: '/admin/panel', view: 'admin.dashboard')->name(name: 'admin.dashboard');
    Route::view(uri: '/admin/productos', view: 'admin.product')->name(name: 'admin.product');
    Route::view(uri: '/admin/pedidos', view: 'admin.order')->name(name: 'admin.order');
    Route::view(uri: '/admin/usuarios', view: 'admin.users')->name(name: 'admin.users');
});
```

5.7.6 – OTRAS RUTAS

- /dashboard: Redirige a una vista u otra dependiendo del rol.

```
Route::get(uri: '/dashboard', action: function (): mixed|RedirectResponse {
    if (Auth::check()) {
        // Si el rol es 1 (admin), redirige al dashboard admin; sino, perfil usuario normal
        return redirect()->route(route: Auth::user()->role == 1 ? 'admin.dashboard' : 'user.profile');
    }
})->middleware(middleware: ['auth', 'verified'])->name(name: 'dashboard');
```

- /2fa: No está en el archivo /routes/web.php sino en el archivo /routes/auth.php, y lleva a la vista auth.second-verification, para completar la verificación en dos pasos (2FA) tras el login.

```
Route::middleware(middleware: 'auth')->group(callback: function (): void {
    Volt::route(uri: '2fa', componentName: 'auth.second-verification')->name(name: '2fa');
```

- Rutas del archivo /routes/auth.php: Se crean solas al crear el proyectos de laravel, por lo que no las he modificado, solo he añadido la vista de la segunda autenticación.

```
Route::middleware(middleware: 'guest')->group(callback: function (): void {
    Volt::route(uri: 'login', componentName: 'auth.login')
        ->name(name: 'login');

    Volt::route(uri: 'register', componentName: 'auth.register')
        ->name(name: 'register');

    Volt::route(uri: 'forgot-password', componentName: 'auth.forgot-password')
        ->name(name: 'password.request');

    Volt::route(uri: 'reset-password/{token}', componentName: 'auth.reset-password')
        ->name(name: 'password.reset');
});
```

```
Route::middleware(middleware: 'auth')->group(callback: function (): void {
    Volt::route(uri: 'verify-email', componentName: 'auth.verify-email')
        ->name(name: 'verification.notice');

    Route::get(uri: 'verify-email/{id}/{hash}', action: VerifyEmailController::class)
        ->middleware(middleware: ['signed', 'throttle:6,1'])
        ->name(name: 'verification.verify');

    Volt::route(uri: 'confirm-password', componentName: 'auth.confirm-password')
        ->name(name: 'password.confirm');

    // Verificación en 2 pasos
    Volt::route(uri: '2fa', componentName: 'auth.second-verification')->name(name: '2fa');
});

Route::post(uri: 'logout', action: App\Livewire\Actions\Logout::class)
    ->name(name: 'logout');
```

5.8 – SISTEMA DE CORREO ELECTRÓNICO

Los correos electrónicos se utilizan para notificar a los usuarios y administradores sobre eventos importantes, como la realización de un pedido o la verificación en dos pasos (2FA). Los correos son gestionados mediante clases que extienden Mailable y usan vistas Blade para mostrar el contenido del email.

5.8.1 – NUEVO PEDIDO

Esta clase (app/Mail/EmailPedidos.php) genera un correo dirigido al administrador notificando que un usuario ha realizado un nuevo pedido. Su contenido es el siguiente:

- Variable pública: \$pedido que almacena la información del pedido que se enviará en el email. `public $pedido;`
- Método __construct(\$pedido): Asigna el pedido que se pasa a la variable de la clase.

```
public function __construct($pedido)
{
    $this->pedido = $pedido;
}
```

- Método `build()`: Se construye el correo que se quiere mandar. Este método devuelve:
 - El asunto de email
 - La vista blade asociada que contiene la plantilla del email.
 - Los datos (el pedido y el usuario que realiza el pedido) que se envían a la vista para mostrar en el email.

```
public function build(): EmailPedidos
{
    return $this->subject(subject: 'Nuevo pedido recibido')
        ->view(view: 'emails.email-pedidos')
        ->with(key: [
            'pedido' => $this->pedido,
            'usuario' => $this->pedido->user,
        ]);
}
```

Luego en la vista (`resources/views/emails/email-pedidos.blade.php`) se muestra el mensaje que contiene: Nombre y email del usuario, ID y fecha del pedido, y un botón que enlaza al panel de administración para ver los pedidos.

```
<h2>Nuevo pedido recibido!</h2>
<p>El usuario <strong>{{ $usuario->name }}</strong> ({{ $usuario->email }}) ha realizado un pedido.</p>

<p><strong>ID del pedido:</strong> {{ $pedido->id }}</p>
<p><strong>Fecha:</strong> {{ $pedido->created_at->format('d/m/Y H:i') }}</p>

<p>
    <a href="{{ url(path: '/admin/pedidos/') }}"
        style="padding:10px 15px; background:#0d6efd; color:white; text-decoration:none; border-radius:5px;"
        Ver en la web
    </a>
</p>
```

5.8.2 – CÓDIGO DE VERIFICACIÓN

Esta clase (`app\Mail\twoFACode.php`) genera un correo con un código único de verificación que se envía tras el login para confirmar la identidad del usuario. Su contenido es el siguiente:

- Variable pública: `$code` que almacena código que se enviará en el email.
- Método `__construct($code)`: Asigna el código que se pasa a la variable de la clase.

```
public function __construct($code)
{
    $this->code = $code;
}
```

- Método `build()`: Se construye el correo que se quiere mandar. Este método devuelve:
 - El asunto de email
 - La vista blade asociada que contiene la plantilla del email.
 - El código que se envían a la vista para mostrar en el email.

```
public function build(): twoFACode
{
    return $this->subject(subject: 'Código de Verificación')
        ->view(view: 'emails.twoFACode')
        ->with(key: [
            'code' => $this->code,
        ]);
}
```

Luego en la vista (`resources/views/emails/email-pedidos.blade.php`) se muestra el mensaje que contiene: un mensaje informativo indicando que se trata del código de verificación, y el código destacado en negrita para facilitar su visualización.

```
<h2>Código de Verificación.</h2>
<p>Su código de verificación es:</p>
<p><strong>{{ $code }}</strong></p>
```

5.9 - AUTENTICACIÓN Y VERIFICACIÓN EN 2 PASOS

Laravel ya incluye por defecto un sistema de autenticación que se encarga del registro, inicio y cierre de sesión, recuperación de contraseña, verificación de correo, etc. En este proyecto, ese sistema se ha personalizado para incluir una verificación en dos pasos (2FA) justo después del login.

Todas las vistas y componentes relacionados con la autenticación están ubicados en la carpeta `resources/views/livewire/auth`. Laravel genera por defecto estos archivos, que combinan tanto la lógica PHP como la parte visual (blade) en un solo archivo. A estos archivos les hice cambios mínimos para adaptarlos: traduje los textos al español, mejoré la accesibilidad con etiquetas como `aria-label` y `title`, y sustituí el icono predeterminado de Laravel por uno de Bootstrap.

```
@section('title', content: 'Registrarse')
<x-auth-header
    :title="__(key: 'Crear una cuenta')"
    :description="__(key: 'Introduce tus datos para crear tu cuenta')"
    title="Crear una cuenta"
    aria-label="Formulario para crear una cuenta"
/>
```

La mayoría de los cambios los he hecho en la pantalla del login para añadir el 2FA.

Al iniciar sesión correctamente, el sistema genera un código aleatorio de 6 dígitos y lo envía al correo del usuario (en este caso, el administrador). Ese código se guarda en sesión junto con el ID del usuario, y se redirige a una nueva vista llamada `second-verification.blade.php`.

```
public function login(): void
{
    // ...

    // Genera código 2FA de 6 dígitos aleatorio
    $twoFACode = rand(min: 100000, max: 999999);

    $user = Auth::user();

    // Guarda el id del usuario y código en sesión para validar en 2FA
    session(key: [
        '2fa:user:id' => $user->id,
        '2fa:user:code' => (string) $twoFACode,
    ]);

    // Enviar correo con el código al administrador
    Mail::to(users: $this->email)->send(mailable: new twoFACode(code: $twoFACode));

    RateLimiter::clear(key: $this->throttleKey());
    Session::regenerate();

    redirect()->to(path: route(name: '2fa'));
}
```

Esta nueva vista se encuentra en la misma ruta que el resto (`resources/views/livewire/auth`) y mantiene el mismo estilo visual que el login, ya que reutilicé parte del código y los componentes para que tuviera coherencia con el diseño general de las vistas de autenticación.

En esta pantalla, el usuario debe introducir el código de 6 cifras recibido por email. Si el código es correcto, la verificación se completa y se redirige al usuario al dashboard; si no, se muestra un mensaje de error sin recargar la página.

```
<?php

use Livewire\Attributes\Layout;
use Livewire\Volt\Component;

new #[Layout(name: 'components.layouts.auth')] class extends Component {
    public $codigo;

    public function mount(): void
    {
        // Validar sesión 2FA
        if (!session()->has(key: '2fa:user:code') || !session()->has(key: '2fa:user:id')) {
            Auth::logout();
            session()->invalidate();
            session()->regenerateToken();

            $this->redirect(url: route(name: 'login'));
        }
    }
}
```

```

public function verification(): mixed|RedirectResponse
{
    $this->validate(
        rules: [
            'codigo' => 'required|digits:6',
        ],
        messages: [
            'codigo.required' => 'Por favor, introduce el código de verificación.',
            'codigo.digits' => 'El código debe tener exactamente 6 dígitos.',
        ],
    );

    $codigoGuardado = session(key: '2fa:user:code');

    if ($this->codigo !== $codigoGuardado) {
        // Código introducido no coincide con el guardado:
        $this->addError(name: 'codigo', message: 'El código introducido es incorrecto.');
```

```

    } else {
        // Código correcto:
        session()->forget(keys: ['2fa:user:id', '2fa:user:code']); // Limpia datos temporales 2FA
        session(key: ['2fa:verified' => true]); // Marcar 2FA como completado

        // Redirige al destino (dashboard o ruta que quieras)
        return redirect()->intended(default: route(name: 'dashboard'));
    }
}

};
?>
<div class="flex flex-col gap-6">
    @section(section: 'title', content: 'Segunda Verificación')
    <x-auth-header title="Verificación en 2 pasos"
        description="Le hemos enviado un email. Por favor introduzca el código que contiene el mensaje para continuar"
        aria-label="Formulario para iniciar sesión en tu cuenta" />

    <x-auth-session-status class="text-center" :status="session(key: 'status')" role="alert" aria-live="polite" />

    <form wire:submit.prevent="verification" class="flex flex-col gap-6" aria-label="Formulario de Verificación">
        <div>
            <label for="codigo" class="block text-sm font-medium text-gray-700">Código</label>
            <input type="text" id="codigo" name="codigo" wire:model="codigo" required autofocus
                class="mt-1 block w-full border border-gray-300 rounded-md shadow-sm p-2"
                placeholder="Introduce el código: XXXXXX" title="Introduce el código de verificación"
                aria-label="Código de verificación" />
            <span style="font-size: smaller; font-weight: lighter;"> Campo obligatorio</span>

            @error('codigo')
                <span class="text-red-600 text-sm">{{ $message }}</span>
            @enderror
        </div>

        <div class="flex items-center justify-end">
            <button type="submit" class="w-full btn btn-outline-dark py-2 px-4 rounded hover:bg-blue-700">
                {{ __(key: 'Enviar Código') }}
            </button>
        </div>
    </form>
</div>

```

Este proceso asegura una capa extra de seguridad al sistema de autenticación sin afectar la experiencia del usuario, ya que la implementación es fluida y visualmente coherente con el resto de la interfaz.

5.10 – PRÁCTICAS DE ACCESIBILIDAD IMPLEMENTADAS

Para garantizar una experiencia inclusiva, se han aplicado diversas buenas prácticas de accesibilidad en el desarrollo del ecommerce:

- **Etiquetado semántico:** Todos los campos de formulario están asociados correctamente a sus etiquetas (label) mediante los atributos for, id y name, facilitando así la navegación mediante lectores de pantalla.
- **Atributos de accesibilidad:** Se han añadido atributos como title, aria-label y alt en los elementos interactivos e imágenes, mejorando la comprensión del contenido por parte de tecnologías asistivas.
- **Contraste adecuado en formularios:** Los campos de entrada cuentan con un borde negro por defecto, y al recibir el foco presentan un borde de 1px con alto contraste, garantizando la visibilidad para personas con baja visión.
- **Jerarquía de encabezados:** Se ha respetado la jerarquía de los encabezados HTML para mantener una estructura clara y accesible del contenido.
- **Identificadores únicos:** Todos los elementos interactivos cuentan con ids únicos que mejora su accesibilidad y facilita su manipulación mediante estilos o scripts. Además, permite una identificación precisa de los elementos en caso de implementar pruebas automatizadas o manuales.
- **Navegación mediante teclado:** Todos los elementos interactivos han sido configurados para que el orden de tabulación y el foco sigan una secuencia lógica y natural, garantizando que sean completamente accesibles mediante el teclado.
- **Indicadores de foco visual:** Se ha aplicado CSS específico para que los elementos que reciben el foco mediante teclado se resalten de forma clara, ayudando especialmente a personas con baja visión o daltonismo a no perder de vista el elemento activo.
- **Diseño completamente responsive:** Gracias al uso de Bootstrap y css personalizado todos los elementos de la interfaz se adaptan correctamente a distintos tamaños de pantalla y resoluciones.

6. PRUEBAS

Para asegurar el correcto funcionamiento del ecommerce, en un primer momento se planteaba realizar **pruebas unitarias y funcionales** además de las de accesibilidad. Sin embargo, por motivos de tiempo y priorización de funcionalidades esenciales, en esta fase del proyecto solo se llevarán a cabo las pruebas de accesibilidad.

Como idea de implementación a futuro se contempla la posibilidad de ampliar el sistema de validaciones automatizadas mediante:

- Pruebas unitarias: se alojarían en la carpeta `/tests/Unit` del proyecto Laravel. Los archivos de los test se pueden generar con el comando:

```
php artisan make:test NombreDelTest --unit
```

- Pruebas funcionales: también se podrían desarrollar dentro del entorno de Laravel, en la carpeta `/tests/Feature`, aunque sería necesario investigar con mayor profundidad cómo se estructuran y ejecutan este tipo de pruebas en Laravel.

No obstante, desde un principio se consideró realizar las pruebas funcionales con herramientas externas, según lo aprendido durante el periodo de formación dual en la empresa. En concreto, se valoraron dos opciones:

- **Cypress con Cucumber**, que permite desarrollar pruebas funcionales automatizadas directamente desde Visual Studio Code, simulando interacciones reales del usuario.
- **Selenium con Cucumber**, que posibilita realizar pruebas funcionales automatizadas en Java, utilizando el entorno de desarrollo Eclipse, también con enfoque en comportamiento (BDD).

6.1 – DE ACCESIBILIDAD

El proyecto incluirá **pruebas de accesibilidad web**, siguiendo como referencia la norma **UNE-EN 301549:2022**, que establece los criterios para que un sitio web sea accesible para todas las personas, incluyendo aquellas con discapacidades.

La evaluación se centrará en los puntos **9 (Web)**, **11 (Software)** y **12 (Servicios de Apoyo)**, ya que el sitio no tendrá integradas tecnologías específicas de accesibilidad (punto 5), no tendrá opciones de voz (punto 6), ni vídeos (punto 7), y no incorporará documentos descargables (punto 10), y por lo tanto no hará falta que se examinen.

6.1.1 – HERRAMIENTAS

Las pruebas se realizarán principalmente mediante herramientas y extensiones de Chrome como:

- **Lighthouse**
- **Siteimprove**
- **taba11y**
- **Live CSS**
- **HeadingsMap**

Además, aquellas comprobaciones no cubiertas por las extensiones se realizarán de forma manual.

Para facilitar el proceso, se seguirá un **manual de accesibilidad en desarrollo**, basado en lo aprendido dentro del equipo de trabajo y facilitado por una compañera de empresa.

6.1.2 - DOCUMENTACIÓN

Todo el proceso de pruebas quedará documentado en varios formatos:

- Una imagen llamada **resultados_prueba** de un fichero excel que recogerá los resultados de accesibilidad con su puntuación.
- Dos documentos .txt:
 - FALLOS_EPIGRAFES.txt: Es técnico y está orientado al **programador**, con los fallos detectados y posibles soluciones.
 - NO_ACCESIBLE.txt: Indica las características no accesibles de la web. Tiene una redacción más formal y orientada a informar al usuario final de las limitaciones detectadas.
- Carpetas con **capturas de pantalla** e **informes** generados por las herramientas utilizadas.

Esta documentación permitirá tener trazabilidad del trabajo realizado, además de servir como guía de mejora para futuras versiones del sitio.

7. COSTES/PRESUPUESTO

El principal coste ha sido el tiempo dedicado a la planificación, diseño, desarrollo y pruebas del ecommerce.

Gracias al uso de herramientas de código abierto como Laravel, Livewire y XAMPP (que incluye Apache, PHP y MySQL), el desarrollo se ha realizado sin necesidad de licencias ni gastos en infraestructura.

Todo se ha trabajado localmente por lo que los recursos utilizados han sido principalmente mi equipo personal y acceso a internet, sin contratar servicios externos ni herramientas de pago.

En caso de llevar el proyecto a producción, se deberá considerar la contratación de un hosting y registro de dominio. Estos servicios, podrían tener un coste mensual aproximado entre 5 y 10 euros para un plan básico adecuado al tamaño y funcionalidad del ecommerce. Además, se podría necesitar inversión en certificados SSL y mantenimiento continuo.

Aunque el proyecto tiene un carácter académico y no ha implicado costes reales, en un entorno profesional sería necesario realizar una estimación económica basada en el tiempo invertido y el coste de la mano de obra.

Si se estima que para llevar a cabo el desarrollo completo del ecommerce, se han dedicado aproximadamente 100 horas de trabajo. Considerando una tarifa media de 15 €/hora para un desarrollador web junior o freelance, el coste total del desarrollo ascendería a unos 1.500 €.

En conclusión, el desarrollo del ecommerce no ha supuesto costes económicos directos gracias al uso de herramientas de código abierto y recursos propios. Sin embargo, si se valorara el tiempo invertido como mano de obra profesional, el coste estimado ascendería a unos 1.500 €. En caso de llevar el proyecto a producción, habría que añadir los gastos asociados a hosting, dominio y mantenimiento.

8. CONCLUSIONES

Este proyecto ha permitido desarrollar una tienda online funcional utilizando Laravel y Livewire, aplicando buenas prácticas de desarrollo web, control de roles, gestión dinámica de productos, pedidos y usuarios. Se han implementado características avanzadas como filtros por atributos, carrito de compras con total actualizado, sistema de ofertas, subida de imágenes múltiples y verificación en dos pasos (2FA).

Además, la documentación técnica generada durante el proceso permite una futura escalabilidad y mantenimiento del sistema. Este proyecto sienta las bases para futuros desarrollos profesionales o académicos en el ámbito del comercio electrónico.

No obstante, me hubiera gustado contar con más tiempo para implementar todas las ideas que se tenían previstas, así como para solucionar algunos errores pendientes, realizar más pruebas y llevar a cabo mejoras adicionales que habrían enriquecido aún más la funcionalidad y experiencia del usuario.

9. BIBLIOGRAFÍA

Durante el desarrollo de este proyecto se ha consultado a varias fuentes digitales, que han servido para resolver dudas técnicas, entender el funcionamiento de herramientas y adquirir conocimientos necesarios para la implementación y optimización del ecommerce. Por ejemplo:

- [Documentación oficial de Laravel](#)
- [Documentación oficial de Livewire](#)
- [Documentación oficial de Middleware](#)
- [MySQL 8.0 Reference Manual](#)
- [Bootstrap 5 Documentation](#)

- Videos de YouTube:
 - Lista de reproducción: [Laravel 11 E-Commerce Project](#)

- [Stack Overflow](#) (para buscar solución a algunos errores)
- [ChatGPT](#) (que depende del día me ayudaba con los errores, o se cargaba media web)

Además, se ha recurrido a documentación dada por los maestros durante el curso para aprender x temas, y a la hecha por una misma para algunas tareas durante el curso, que han servido como referencia para recordar conceptos y procedimientos de programación y diseño que no se tenían totalmente frescos.

Asimismo, se utilizaron otras páginas web y foros especializados de los cuales no recuerdo la URL, que facilitaron la búsqueda de soluciones a errores puntuales y la implementación correcta de funcionalidades diversas.

10. GLOSARIO

Laravel: Framework PHP para el desarrollo de aplicaciones web siguiendo el patrón MVC.

Livewire: Librería de Laravel que permite crear interfaces dinámicas sin necesidad de usar JavaScript.

CRUD: Siglas de Create, Read, Update, Delete; operaciones básicas de persistencia de datos.

Blade: Motor de plantillas de Laravel que permite separar lógica y presentación.

Middleware: Filtro que intercepta peticiones HTTP para aplicar reglas o condiciones.

2FA (Autenticación en dos pasos): Mecanismo de seguridad que solicita un segundo factor de autenticación.

Eloquent: ORM (Mapeo Objeto-Relacional) incluido en Laravel para trabajar con bases de datos de forma intuitiva.

Frontend: Parte visible de una aplicación web que interactúa con el usuario.

Backend: Lógica del servidor que gestiona datos, reglas de negocio y seguridad.

Dashboard: Panel de administración para gestionar usuarios, productos, pedidos y otras funcionalidades del ecommerce.

Flux: Componente de navegación utilizado en Volt, el motor de plantillas de Laravel, que permite crear listas de navegación dinámicas y accesibles con variantes de diseño, empleando tecnologías como Tailwind CSS y Vite para la gestión del frontend.

Volt: Motor de plantillas basado en PHP para Laravel que facilita la creación de vistas reutilizables y limpias.

Tailwind CSS: Framework CSS para diseño de interfaces con clases utilitarias que permite un desarrollo rápido y personalizado.

Vite: Herramienta de construcción y servidor de desarrollo frontend moderno que acelera el proceso de compilación y recarga en caliente.

UNE-EN 301549:2022: norma que establece los criterios para que un sitio web sea accesible para todas las personas, incluyendo aquellas con discapacidades.

11. ANEXOS

11.1 – PROTOTIPO DE LA WEB

Como parte del proceso de diseño previo al desarrollo, se ha creado un prototipo de baja fidelidad utilizando la herramienta **Balsamiq Mockups**. Este prototipo tiene como objetivo representar de forma visual la estructura general y la disposición de los elementos clave de la tienda online, antes de su implementación final en Laravel + Livewire.

El diseño busca ofrecer una experiencia de usuario intuitiva, clara y accesible tanto para usuarios comunes como para administradores del sistema. Incluye pantallas como la página principal, listado de productos, detalle del producto, carrito de compras, formulario de registro/login, y panel de administración.

El prototipo completo se adjunta en el archivo **Prototipo_Ecommerce.pdf**.

Este documento ha servido de base para tomar decisiones durante el desarrollo, validar ideas y anticipar posibles mejoras en la interfaz antes de escribir el código.

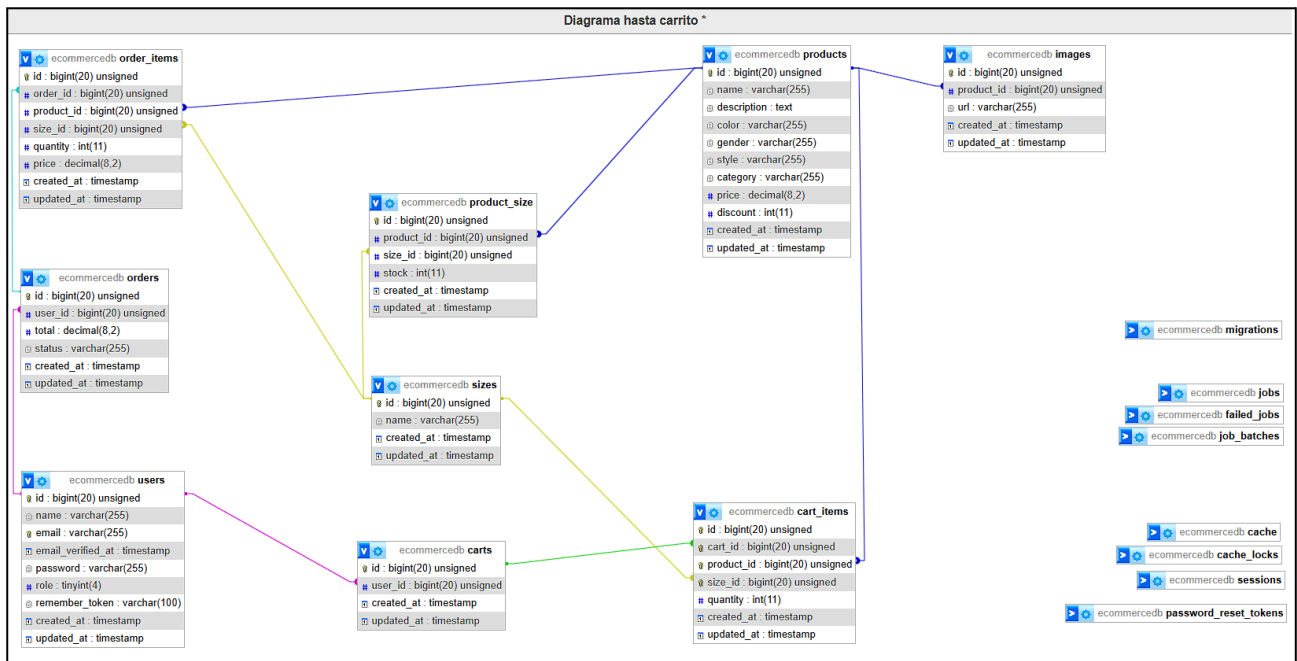
11.2 – BASE DE DATOS

Al crear un nuevo proyecto con Laravel, se generan automáticamente algunas tablas por defecto relacionadas con la autenticación y la gestión de usuarios. Sin embargo, para el desarrollo del ecommerce se han diseñado e implementado tablas adicionales que responden a las necesidades específicas del sistema, como la gestión de productos, tallas, imágenes, carrito, pedidos y sus relaciones.

A continuación, se presenta el modelo de base de datos utilizado, incluyendo las tablas principales y sus relaciones.

11.2.1 - DIAGRAMA RELACIONAL

El siguiente diagrama muestra la estructura relacional de la base de datos utilizada en el proyecto. En él se pueden observar las tablas principales del sistema, así como las relaciones entre ellas (uno a muchos y muchos a muchos).



Este esquema facilita la comprensión de la lógica de datos detrás del ecommerce, permitiendo identificar de forma clara cómo se vinculan los productos con sus imágenes, tallas disponibles, carritos de usuario, pedidos realizados, etc.

11.2.2 - GRAFO

El grafo de relaciones representa visualmente las conexiones entre las tablas de la base de datos desde una perspectiva más simplificada. Este enfoque se centra en mostrar cómo se enlazan los elementos del sistema, destacando las dependencias clave:

- users (id, name, email, email_verified_at, password, role, remember_token, created_at, updated_at)
- products (id, name, description, color, gender, style, category, price, discount, created_at, updated_at)
- images (id, product_id, url, created_at, updated_at)
- product_size (id, product_id, size_id, stock, created_at, updated_at)
- sizes (id, name, created_at, updated_at)
- carts (id, users_id, created_at, updated_at)
- cart_items (id, cart_id, product_id, size_id, quantity, created_at, updated_at)
- orders (id, users_id, total, status, created_at, updated_at)
- order_items (id, order_id, product_id, size_id, quantity, price, created_at, updated_at)

Este recurso es útil tanto para el desarrollo como para futuras tareas de mantenimiento, ya que ofrece una visión rápida y global de la estructura del proyecto.